Scaling Multi-Cloud

The Engineer’s Guide to Docker, Jenkins, and Infrastructure as Code

Tarun Pradeep B

2026-10-12

Contents

Reader and Verification Notes **7**

Chapter 1: Stop Deploying Snowflakes: Design the Platform Before the Pipeline **8**
- 03:07: The Server Nobody Can Rebuild . . . 8
- Four Lifecycles, Four Owners . . . 8
- Set Targets Before Choosing Services . . . 9
- Draw Trust Boundaries Around Identities . . . 10
- Hands-On: Create the Platform Repository . . . 11
- Acceptance Gates: Make the Architecture Falsifiable . . . 13
- References . . . 14

Chapter 2: Terraform Without Guesswork: Build a Reproducible Foundation **15**
- The Apply That Removed the Emergency Fix . . . 15
- Configuration, State, and Reality . . . 15
- Terraform Executes a Graph, Not a Script . . . 16
- Providers, Resources, and Data Sources . . . 16
- Hands-On: A Guarded Terraform Root . . . 16
- Credentials Are Execution Inputs . . . 20
- Lock Dependencies and Review Execution . . . 20
- Key Takeaways . . . 21
- References . . . 21

Chapter 3: State Is Infrastructure: Lock It, Protect It, Recover It **22**
- Two Applies, One Broken Production Environment . . . 22
- Remote Storage Alone Is Not Enough . . . 22
- Bootstrap Without a Circular Dependency . . . 23
- Give the Writer Only Its State . . . 26
- Migrate and Prove Coordination . . . 27
- State Recovery Runbook . . . 27
- Key Takeaways . . . 28
- References . . . 28

Chapter 4: AWS from Zero: Provision a Network You Can Explain **29**
- The Database That Answered the Internet . . . 29
- A Route Is a Decision . . . 29
- Lab and Production Are Different Architectures . . . 29
- Provision the Lab Root . . . 30
- Verify Behavior, Not Just Objects . . . 32
- Diagnose the First Broken Hop . . . 33
- Key Takeaways . . . 33
- References . . . 33

Chapter 5: Modules with Boundaries: Reuse Infrastructure Without Coupling Every-

thing **34**
A Development Tag That Replaced Production . . . 34
Root Modules Are Operational Boundaries . . . 34
Workspaces Are State Namespaces, Not Security Walls . . . 34
Keep Modules Small Enough to Explain . . . 35
Repository Contract . . . 35
Provider and State Isolation . . . 37
Acceptance and Recovery . . . 38
Review a Module as a Product Contract . . . 38
Key Takeaways . . . 39
References . . . 39

Chapter 6: Build Once, Run Consistently: Containerize the Application **40**
Friday's Different Runtime . . . 40
Define the Application Contract . . . 40
Separate Build Tools from Runtime . . . 41
Local Development Without Public Databases . . . 42
Build, Test, and Record the Artifact . . . 43
Promote, Do Not Rebuild . . . 43
A Dependency Change Is a New Release . . . 44
Key Takeaways . . . 44
References . . . 45

Chapter 7: Containers Die; Data Must Survive: Understand Docker Storage **46**
The Image Update That Deleted Orders . . . 46
Three Storage Lifecycles . . . 46
Establish the Data Contract . . . 47
Permissions Must Match the Image . . . 48
Prove Container Replacement Is Safe . . . 48
Backup Is an Application Operation . . . 48
Upgrades and Destruction Need Their Own Workflow . . . 49
Detect an Empty Mount Before Writing . . . 49
Key Takeaways . . . 50
References . . . 50

Chapter 8: Jenkins That Survives Monday: Docker Volume Persistence on EC2 **51**
The Spot Termination That Erased CI . . . 51
Own the Controller's Data . . . 51
Select the Disk Before Formatting . . . 51
Mount Reliably and Fail Closed . . . 52
Match Ownership to the Image . . . 53
Demonstrate Replacement Survival . . . 53
Backup and Recovery Are Separate . . . 54
Retention Is a Cloud Resource Setting . . . 54
Key Takeaways . . . 55
References . . . 55

Chapter 9: Secure the Build Factory: Separate Controllers, Agents, and Credentials **56**
A Dependency Script Finds the Deployment Keys . . . 56
Controller and Agent Are Different Trust Domains . . . 56
Configuration as Code Must Be Versioned . . . 56
Short-Lived Credentials Still Need Separation . . . 58
Credentials Belong to Stages and Identities . . . 59
State and Recovery Controls . . . 59
Prove the Boundary . . . 59
Limit Agent Network Authority . . . 60

Key Takeaways . . . 60
References . . . 60

Chapter 10: From Commit to Trusted Image: Write the Jenkinsfile 61
Latest Was Not the Release . . . 61
Define the Pipeline's Contract . . . 61
A Complete Declarative Pipeline . . . 61
Understand the Scanner Gate . . . 64
Publication Is Not Deployment . . . 64
Boundaries and Failure Handling . . . 64
Bind the Published Digest to the Tested Image . . . 65
Key Takeaways . . . 65
References . . . 65

Chapter 11: Terraform Through CI: Approve the Change You Actually Apply 66
Approval Covered a Different Plan . . . 66
Plan Files Are Sensitive Execution Artifacts . . . 66
Separate Review from Authority . . . 66
A Plan, Approve, Apply Pipeline . . . 67
Handle Staleness Explicitly . . . 68
Partial Apply Is an Incident Condition . . . 69
Protect the Approval System . . . 69
Test the Process . . . 69
Plan Review Must Explain Replacements . . . 70
Key Takeaways . . . 70
References . . . 70

Chapter 12: Kubernetes Beyond "It Runs": Orchestrate the Application 71
The Pods That Never Came Back . . . 71
Deployments Own Replica Intent . . . 71
The Deployment Contract . . . 71
Probes Have Different Jobs . . . 74
Requests Are Scheduling Inputs . . . 74
Apply and Inspect . . . 74
Termination Is Part of Correctness . . . 75
State and Ownership . . . 75
Troubleshoot from Ownership to Traffic . . . 75
Key Takeaways . . . 76
References . . . 76

Chapter 13: AWS Kubernetes: Build an EKS Platform with Terraform 77
The Cluster Only Its Creator Understood . . . 77
Split Cloud Provisioning from Workload Deployment . . . 77
The Module Is Community Maintained . . . 77
IRSA Connects a Specific Service Account . . . 79
Access Is Not One Permission . . . 80
Upgrades Are Platform Releases . . . 80
Recovery and Cleanup . . . 80
Version the Entire Platform Envelope . . . 81
Key Takeaways . . . 81
References . . . 81

Chapter 14: Traffic Has Boundaries: TLS, Network Policies, and FortiGate 82
One Container, Every Database . . . 82
North-South and East-West Are Design Categories . . . 82
Establish Default Deny . . . 82

TLS Is an Identity Boundary . . . 84
FortiGate Must Be on the Actual Route . . . 84
Test Packet Paths . . . 85
State and Recovery . . . 85
Make Dependency Changes Reviewable . . . 86
Key Takeaways . . . 86
References . . . 86

Chapter 15: Stateful Kubernetes: Storage, Databases, and Safe Migrations 87
The Database Could Not Follow the Pod . . . 87
PV, PVC, and StorageClass Have Different Owners . . . 87
Provision with Scheduling Context . . . 87
A Single-Instance Database Lab . . . 88
Verify Claim and Zone . . . 90
Managed Services Are a Tradeoff . . . 90
Migrate Schemas Safely . . . 90
Backup and Recovery . . . 90
Cleanup and Ownership . . . 91
Storage Expansion Is an Operational Change . . . 91
Key Takeaways . . . 91
References . . . 92

Chapter 16: Release Without Roulette: Deployments, Promotion, and Rollback 93
Green Pods, Unusable Service . . . 93
Package Configuration Without Rebuilding . . . 93
Environment Values . . . 93
Rolling Updates Need Headroom . . . 95
Choose a Traffic Strategy . . . 95
Release Gates . . . 95
Rollback Has a Compatibility Boundary . . . 96
Verify User Behavior . . . 96
State and Authority . . . 96
A Release Needs a Decision Window . . . 97
Key Takeaways . . . 97
References . . . 97

Chapter 17: Scale What You Can Measure: Reliability and Observability 98
Fifty Replicas, One Exhausted Database . . . 98
Define the Service's Evidence . . . 98
Capacity Is a Budget Across Dependencies . . . 98
HPA Requires Requests and Metrics . . . 99
PDBs Protect Selected Maintenance . . . 100
Alerts Need Actionable Semantics . . . 100
Controlled Load Testing . . . 100
Recovery and Telemetry State . . . 101
Tune with Evidence . . . 101
Protect the Database During Scale-Up . . . 101
Key Takeaways . . . 102
References . . . 102

Chapter 18: Azure as a Second Home: Port the Platform Without Pretending Clouds Match 103
Renaming Resources Did Not Port the Network . . . 103
Map Responsibilities, Not Resource Names . . . 103
A Deliberate AKS Foundation . . . 103
Separate Cluster and Workload Identities . . . 105

Registry and Release Contracts . . . 105
State and Security Boundaries . . . 106
Validate Differences Explicitly . . . 106
Upgrade and Cleanup . . . 106
Model Azure Egress and Administration Separately . . . 106
Key Takeaways . . . 107
References . . . 107

Chapter 19: GCP Without a Rewrite: Test the Limits of Portability 108
The Friday Migration Mandate . . . 108
Build a Capability Map . . . 108
Autopilot Changes the Node Ownership Contract . . . 109
Authenticate Without Exported Keys . . . 110
Deploy the Application Contract . . . 110
Data Portability Has a Recovery Cost . . . 111
Test the Limits, Then Record Them . . . 111
Cleanup and Upgrade Responsibility . . . 111
Build Adapter Tests Around Failure Semantics . . . 111
Key Takeaways . . . 112
References . . . 112

Chapter 20: Connect Clouds Carefully: Identity, Routing, and Data Boundaries 113
The Tunnel Was Up; Traffic Went Nowhere . . . 113
Allocate Addresses Before Connecting . . . 113
Choose the Routing Protocol Explicitly . . . 113
AWS Static-Routing Example . . . 114
DNS Is a Separate Cross-Cloud System . . . 115
FortiGate and Return Paths . . . 115
Data Gravity and Cost . . . 115
Architecture Checklist . . . 115
Prove the Path and Its Failure Behavior . . . 116
Rotation Can Interrupt the Data Path . . . 116
Key Takeaways . . . 117
References . . . 117

Chapter 21: Failover Is a Data Problem: Engineer Disaster Recovery 118
DNS Moved; Writes Still Failed . . . 118
Active-Passive Has One Writer . . . 118
Measure RPO and RTO Before the Incident . . . 118
Fence Before Promotion . . . 119
Recovery Checklist . . . 119
Inspect the Secondary . . . 119
Promote the Approved Standby . . . 120
Deploy the Recovery Application . . . 120
Update Independently Controlled DNS . . . 120
Verify Service, Then Declare Recovery . . . 121
Failback Is Another Migration . . . 121
Preserve the Recovery Decision . . . 122
Key Takeaways . . . 122
References . . . 122

Chapter 22: Break It Before Production: The Multi-Cloud Capstone 123
Green Dashboards, Missing Packets . . . 123
State the Experiment Before Creating the Fault . . . 123
The Game Day Record . . . 123
Experiment One: Lose a Worker Node . . . 124

Experiment Two: Block the Database Dependency 124
Experiment Three: Lose Jenkins 125
Experiment Four: Make State Unavailable 126
Verify Alerts Actually Arrive 126
Recovery and Data Verification 126
Readiness Is a Reviewable Result 126
Close Every Experiment 127
Key Takeaways 127
References 127

Reader and Verification Notes

This first complete manuscript contains all 22 chapters. Examples distinguish executable lab roots, partial integration snippets, and environment-specific values requiring review. It is not a claim that cloud deployments, Jenkins plugins, multi-cloud failover, or diagnostic accuracy have been verified in a live environment.

Use isolated accounts, synthetic data, reviewed tool versions, and protected identities. Commands that format disks, delete resources, or promote databases require the stated prerequisites. Read the verification report before running companion examples.

The author is Tarun Pradeep B. All examples are original teaching material; third-party platforms and documentation remain subject to their respective terms.

Chapter 1: Stop Deploying Snowflakes: Design the Platform Before the Pipeline

03:07: The Server Nobody Can Rebuild

The order API stops responding at 03:07. Your EC2 instance has failed, and the replacement starts with an empty application directory. The engineer who configured the original machine left six months ago.

Git contains the application, but nobody knows which commit was running. PostgreSQL lived on the same server. A cron job supposedly copied backups somewhere; its credentials and destination were configured interactively.

You are rebuilding an undocumented system under pressure. Improvised fixes create undocumented dependencies.

A server is replaceable only when its configuration, artifacts, access requirements, and recovery procedures exist outside it.

Four Lifecycles, Four Owners

Our reference workload contains an HTTP order API, a background worker, and PostgreSQL. The API persists orders and work records in a database transaction; the worker claims pending work and processes it with idempotent handlers.

Separate the platform into four lifecycles:

Lifecycle	Contents	Change mechanism	Recovery mechanism
Application	API, worker, dependencies, migrations	Tested commit becomes an immutable image	Redeploy a known image digest
Infrastructure	Networks, compute, IAM, storage resources	Reviewed Terraform plan and apply	Reconcile definitions with protected state
Runtime configuration	Endpoints, feature flags, limits, secret references	Validated environment configuration	Restore approved configuration; recover secrets separately
Persistent data	Orders, work records, database files	Application transactions and controlled migrations	Restore backups and replay retained database logs

Application Code: Promote an Artifact

Build an image once and promote the same digest through environments. Rebuilding for production introduces another dependency-resolution event and weakens the relationship between tested and deployed software.

Keep database migration code with the application, but give migration execution its own release step. An application rollback does not reverse a schema change safely. Prefer additive changes first, migrate consumers, and remove obsolete structures in a later release.

Application version, configuration version, and schema version must be traceable together.

Infrastructure: Define Ownership Before Automation

Terraform owns resource configuration and its association with remote resource identities. Its state is an operational record, not a database backup and not a complete substitute for configuration.

Separate state by meaningful operational boundaries: bootstrap services, shared networking, and workload infrastructure. Give each root configuration a named owner and authorized apply path. Do not let two roots manage the same resource.

Runtime Configuration: Change Behavior Without Rebuilding

An API image should not contain a production database password or environment-specific hostname. Inject configuration when the workload starts, validate required values, and fail clearly when configuration is invalid.

Version non-secret configuration. Store only secret references in Git, with secret values held in a dedicated secret manager. Document who creates, rotates, retrieves, and recovers those secrets.

Persistent Data: Protect It from Replacement

Container filesystems, VM boot disks, persistent volumes, and backups have different failure boundaries. A Docker volume can survive container replacement while remaining vulnerable to host failure or deletion.

Define data retention and destruction rules explicitly. Production database deletion should require a separate authorized workflow, and backups must remain recoverable if the primary environment becomes inaccessible.

Replacing compute must not imply deleting business data.

Set Targets Before Choosing Services

Availability and recovery objectives describe different requirements. Availability concerns how often users receive usable service; disaster recovery concerns how much interruption and data loss the business accepts after a defined failure.

For this book, use provisional targets that must later be demonstrated:

Objective	Proposed target	Measurement
Order API availability	99.9% over a rolling 30 days	Successful eligible requests / total eligible requests
API latency	95th percentile below 300 ms at 100 requests/s	Load test with a documented request mix

Objective	Proposed target	Measurement
Production disaster RTO	60 minutes	Interruption to validated order creation and processing
Production disaster RPO	5 minutes	Gap between failure time and newest recoverable committed data
Jenkins recovery	RTO 4 hours; RPO 24 hours	Restore controller configuration and resume a test pipeline

A request-based SLO does not directly produce a downtime allowance. Under a separate time-based availability calculation, 99.9% over 30 days permits approximately 43 minutes of unavailability. Choose one definition and keep the arithmetic consistent.

RTO includes detection, decision-making, provisioning, restoration, and validation. A restore command completing in fifteen minutes does not prove a fifteen-minute RTO.

RPO describes acceptable loss measured in time. If the latest recoverable database point is 03:02 and the failure occurs at 03:07, the recovery gap is five minutes. A successful nightly backup cannot establish a five-minute RPO without another recovery mechanism.

Recovery objectives should be defined per workload and used to select recovery strategies.[1] Our initial single-cloud labs will not claim these production targets; they establish components that later recovery exercises will test.

Draw Trust Boundaries Around Identities

A private subnet is a network placement decision, not an authorization policy. Model what each identity can do after compromise.

Separate production and non-production accounts. Isolate build agents from the controller, and never expose production credentials to untrusted pull requests.

Define these boundaries:

Boundary	Allowed interaction	Required control
Internet → API entry point	Authenticated order requests over HTTPS	TLS, authentication, input validation, rate limits
Entry point → API	Forward application traffic	Restricted origin access
API/worker → PostgreSQL	Required database operations	Private connectivity, TLS, separate least-privilege roles
Build agent → registry	Publish approved build artifacts	Scoped temporary credentials where supported
Release agent → production	Apply approved release or infrastructure change	Separate roles, review gate, audit trail
Operator → administration	Approved administrative sessions	MFA, restricted access, logged emergency procedure

Network policies and firewalls reinforce these boundaries; they do not replace application authorization. A FortiGate inspection path must have documented routes and return paths. A firewall deployed outside the actual traffic path provides no protection to that flow.

Hands-On: Create the Platform Repository

Run the following from a directory where a new platform repository should be created:

```
mkdir -p platform
cd platform
git init

# Separate lifecycles so releases cannot silently absorb infrastructure changes.
mkdir -p apps/{api,worker} infra/{bootstrap,modules}
mkdir -p infra/environments/{dev,staging,prod}/{aws,azure}

# Commit configuration contracts, never recovered data or secret values.
mkdir -p config/environments deploy/{compose,kubernetes}
mkdir -p ci docs/{architecture,runbooks} tests/architecture

touch README.md
 ↪  docs/architecture/{environment-matrix,dependency-map,threat-model,acceptance-criteria}.md
touch docs/runbooks/{restore-database,restore-jenkins,emergency-access}.md
```

The resulting ownership structure is:

```
platform/
├── apps/                 # Application code and schema migration code
├── infra/
│   ├── bootstrap/        # State storage and foundational access
│   ├── modules/          # Reusable resource definitions
│   └── environments/     # Independent cloud/environment roots
├── config/environments/  # Non-secret values and secret references
├── deploy/               # Compose and Kubernetes workload definitions
├── ci/                   # Jenkinsfiles and agent configuration
├── docs/
│   ├── architecture/     # Contracts, dependencies, threats, acceptance gates
│   └── runbooks/         # Procedures executable by another engineer
└── tests/architecture/   # Boundary and recovery verification
```

Add a starting `.gitignore`:

```
# Terraform caches, plans, and state can expose infrastructure and secrets.
**/.terraform/*
*.tfstate
*.tfstate.*
*.tfplan
crash.log

# Preserve examples while excluding local secret-bearing configuration.
.env
.env.*
!.env.example

# Recovery artifacts belong in protected storage, never source control.
backups/
*.sql
*.dump
*.pem
*.key
```

Define the Environment Matrix

Copy this table into `docs/architecture/environment-matrix.md`. These are planned contracts, not deployed capabilities.

Property	Development	Staging	Production
Cloud scope	AWS sandbox	Separate AWS account	AWS primary; Azure recovery later
Data	Synthetic	Synthetic representative dataset	Customer orders
Deploy authority	Developer pipeline	Release pipeline	Approved release pipeline
Terraform state	Dedicated dev roots	Dedicated staging roots	Separate AWS/Azure production roots
Identity	Dev-scoped roles	Staging-scoped roles	Production-scoped roles; MFA administration
Capacity	Minimal, scheduled shutdown	Production-like during tests	Sized from measured demand
Recovery target	Rebuild; disposable data	Restore drill ≤ 2 hours	RTO 60 min; RPO 5 min
Deletion	Automated cleanup allowed	Owner-approved reset	Separate authorized destruction workflow

Record the Dependency Map

Copy this into `docs/architecture/dependency-map.md`:

```
# Dependency Map

Arrows mean "depends on", not packet direction.

## Bootstrap and provisioning
Terraform execution -> cloud authentication
Terraform workload roots -> protected state backend
Workload provisioning -> networking and IAM
State backend recovery -> independently accessible backup and credentials

## Build and release
Jenkins controller -> persistent home, authentication, configuration backup
Build agent -> source repository, dependency sources, image registry
Release agent -> approved image digest, environment config, deployment API

## Runtime
Public entry point -> DNS, certificate, healthy API instances
API -> secret retrieval, PostgreSQL
Worker -> secret retrieval, PostgreSQL
Database recovery -> backup storage, retained logs, decryption authorization

## Failure contracts
- Existing workloads must run when Jenkins is unavailable.
- Rebuilding Jenkins must not require a successful Jenkins pipeline.
- Restoring state storage must not require the unavailable state backend.
- Restoring PostgreSQL must not require the failed primary database.
```

Bootstrap dependencies deserve special attention. Record an independently authenticated procedure for recreating backend access and restoring state. Otherwise your recovery system depends

on the component it must recover.

Copy the Threat-Model Template

Create `docs/architecture/threat-model.md`:

```
# Architecture Threat Model

## Scope
Workload: Order API, worker, PostgreSQL, Jenkins, Terraform
Owner: <named team>
Reviewed: <date and architecture revision>
Assumptions: <identity provider, cloud scope, data classification>
Out of scope: <explicit exclusions and responsible owners>

## Protected assets
| Asset | Consequence of compromise | Owner |
|---|---|---|
| Customer orders | Disclosure, deletion, unauthorized modification | Application team |
| Terraform state | Sensitive-value exposure, ownership manipulation | Platform team |
| Deployment identity | Unauthorized production changes | Platform team |
| Database backups | Data exposure or loss of recovery capability | Database owner |

## Trust boundaries
For each flow record:
source identity -> destination -> protocol -> authorization -> audit evidence

## Threat register
| Threat | Control | Verification | Residual risk / owner |
|---|---|---|---|
| Malicious PR steals production credentials | Untrusted builds isolated from release roles |
 ↪  PR job cannot obtain production session | Agent compromise / platform |
| Runtime identity deletes backups | Backup administration separate from runtime |
 ↪  Authorization-policy test in sandbox | Backup admin compromise / database owner |
| Secret committed to Git | Secret references, scanning, rotation procedure | Seeded test
 ↪  secret blocked before merge | Scanner gaps / application |
| Order retried after timeout | Idempotency key and transactional uniqueness | Repeat request
 ↪  creates one order | External side effects / application |

## Recovery and review
Backup location and access owner: <reference>
Emergency access procedure: <runbook>
Recovery evidence: <test date and results>
Open risks: <owner, decision, deadline>
Review triggers: identity change, new public endpoint, storage change
```

Acceptance Gates: Make the Architecture Falsifiable

Finish `acceptance-criteria.md` with concrete gates:

1. **Rebuild:** another engineer can identify every required input without accessing the original host.
2. **Traceability:** each deployment records image digest, configuration revision, schema version, and infrastructure revision.
3. **Isolation:** a development identity cannot read production secrets or state.
4. **Persistence:** replacing application compute preserves committed orders.
5. **Recovery:** database and Jenkins restores are tested with timestamps and functional checks.
6. **Independence:** existing workloads continue serving when Jenkins is unavailable.

At this stage, review repository completeness and assign owners. Mark operational gates **unverified** until later labs produce evidence. An empty runbook is unfinished work, and a successful Terraform apply is not proof of recoverability.

References

[1] AWS Well-Architected Framework, “Define recovery objectives for downtime and data loss”: Documentation

Chapter 2: Terraform Without Guesswork: Build a Reproducible Foundation

The Apply That Removed the Emergency Fix

At 11:40, customers stop receiving order confirmations. During yesterday's incident, an engineer added an AWS security-group rule through the console so the API could reach PostgreSQL on port 5432. Another engineer changed the application's database endpoint manually.

Neither change reached Git. This morning's routine Terraform plan proposed removing the extra rule from a security group whose rules were managed inline. The reviewer saw a small network change and approved it.

The apply completed successfully. Database connections failed, retries saturated the API, and order processing stopped.

Terraform did not misunderstand the emergency fix. The team misunderstood ownership: configuration still declared the previous rules, and reconciliation removed the console-added access.

A successful apply proves that Terraform completed its requested operations. It does not prove that your architecture still works.

Configuration, State, and Reality

Terraform works with three distinct inputs:

Input	Responsibility
Configuration	Declares intended infrastructure and relationships
State	Associates Terraform addresses with remote objects and records attributes
Provider reads	Observe supported attributes of real infrastructure

During an ordinary plan, Terraform reads tracked objects through their providers and compares refreshed observations with configuration. It then proposes actions that reconcile the differences. State supplies identity: `aws_s3_bucket.foundation` maps to a particular AWS bucket.

Terraform does not scan an account and adopt every object. An independently created security group remains outside this configuration's ownership until explicitly imported and represented appropriately.

Drift behavior also depends on resource semantics. An inline security-group definition can reconcile its managed rule set; an individually managed rule does not imply ownership of every other rule. Read the provider's ownership model before predicting what Terraform will remove.

When console drift appears, choose deliberately: incorporate the change into configuration, reverse it, or transfer ownership through a reviewed procedure. Never assume that refreshing state makes the configuration agree with the emergency change.

```
# Inspect observed drift without proposing configuration-driven repairs.
terraform plan -refresh-only

# Compare configuration with refreshed infrastructure before authorizing writes.
terraform plan -out=reviewed.tfplan
terraform show reviewed.tfplan
```

Refresh-only apply updates state and outputs without intentionally changing remote infrastructure. It does not rewrite your HCL. Review it as a state operation, not a fix for missing declarations.[1]

Terraform Executes a Graph, Not a Script

Terraform loads the root directory's configuration files together. Naming a file `01-network.tf` does not make it execute before `02-storage.tf`.

References establish dependency edges. An object referencing its bucket identifier must wait for that bucket to exist. Independent resources can execute concurrently.

Some values are unknown until apply because their producers do not exist yet. Those unknowns propagate through dependent expressions; they are not evidence that Terraform has ignored your configuration.

Use `depends_on` for a hidden behavioral prerequisite that has no natural value reference. For example, writing an object must wait for bucket versioning even though the object needs only the bucket identifier.

Prefer expression references; use explicit dependencies narrowly. Broad module-level dependencies can make plans unnecessarily conservative and increase unknown values.[2] Ordering also does not guarantee application readiness or eliminate eventual consistency.

Providers, Resources, and Data Sources

A provider implements the API operations Terraform uses to read and manage infrastructure. `required_providers` declares its source and compatible versions; the `provider` block configures its region, authentication context, and other behavior.

A resource declares something this configuration owns. A data source reads information without taking lifecycle ownership of the queried object. In this chapter, resources manage a private S3 bucket; `aws_caller_identity` reads the execution account.

Data sources are not automatically reproducible inputs. A query for the newest machine image can resolve differently next week, despite unchanged HCL. Use stable identifiers or deliberate update procedures where deployment reproducibility requires them.

Hands-On: A Guarded Terraform Root

Create the starter under the development AWS root from Chapter 1:

```
cd platform
mkdir -p infra/environments/dev/aws/foundation
cd infra/environments/dev/aws/foundation
```

The project provisions a private, versioned, encrypted bucket and a non-secret architecture marker. It demonstrates production-oriented input and dependency controls without exposing a service publicly.

This is not the Terraform state bucket. Chapter 3 introduces the remote backend; this isolated lab uses local state and must not become a shared production apply root.

providers.tf

```
terraform {
  # Bound CLI compatibility; pin the exact CLI in your team's runner image.
  required_version = ">= 1.10.0, < 2.0.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      # Exact root-module pin: upgrades require an explicit reviewed change.
      version = "= 6.66.0"
    }
  }
}

provider "aws" {
  region = var.deployment.region

  # Fail closed when the shell authenticates to the wrong AWS account.
  allowed_account_ids = [var.deployment.account_id]

  # Credentials come from the standard credential chain, never this file.
  default_tags {
    tags = {
      Project     = var.deployment.project
      Environment = var.deployment.environment
      ManagedBy   = "Terraform"
    }
  }
}
```

The provider version is an explicit baseline, not a request for “latest.” Before adopting it, review its release notes and validate it with your organization’s CLI and policies.

The CLI range rejects unsupported major versions but does not pin one binary. Record an exact CLI version in your development tooling and CI image. The provider lock file does not lock Terraform itself.

variables.tf

```
variable "deployment" {
  description = "Reviewed deployment identity and placement."
  nullable    = false

  # One structured contract prevents unrelated account/region defaults.
  type = object({
    account_id  = string
```

```
    region      = string
    environment = string
    project     = string
  })

  validation {
    condition     = can(regex("^[0-9]{12}$", var.deployment.account_id))
    error_message = "account_id must contain exactly 12 digits."
  }

  validation {
    # This is an organization allow-list, not AWS's complete region list.
    condition = contains(
      ["ap-south-1", "ap-southeast-1"],
      var.deployment.region
    )
    error_message = "region must be ap-south-1 or ap-southeast-1."
  }

  validation {
    condition = contains(
      ["dev", "staging", "prod"],
      var.deployment.environment
    )
    error_message = "environment must be dev, staging, or prod."
  }

  validation {
    # Constrain project naming before constructing resource identifiers.
    condition     = can(regex("^[a-z][a-z0-9-]{2,19}$", var.deployment.project))
    error_message = "project must be 3-20 lowercase letters, digits, or hyphens, starting with a
↪   letter."
  }
}
```

An object type validates structure; the validation blocks enforce domain rules. `can(regex(...))` turns a nonmatching expression into a controlled validation failure.

These checks reject known invalid inputs before resource creation. They cannot prove sufficient IAM permissions, available capacity, or globally unique resource names. Input validation reduces preventable failures; it does not replace planning and service-level checks.

main.tf

```
data "aws_caller_identity" "current" {}

locals {
  # Account, region, and environment reduce collisions; S3 names remain global.
  bucket_name = join("-", [
    var.deployment.project,
    var.deployment.environment,
    data.aws_caller_identity.current.account_id,
    var.deployment.region,
    "foundation"
  ])
}

resource "aws_s3_bucket" "foundation" {
  bucket = local.bucket_name
```

```
  # Never empty stored objects automatically to make destruction succeed.
  force_destroy = false
}

resource "aws_s3_bucket_public_access_block" "foundation" {
  bucket = aws_s3_bucket.foundation.id # Implicit dependency on the bucket.

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "foundation" {
  bucket = aws_s3_bucket.foundation.id

  versioning_configuration {
    status = "Enabled" # Retain previous object versions after overwrite.
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "foundation" {
  bucket = aws_s3_bucket.foundation.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256" # Explicit SSE-S3 baseline; no secrets in HCL.
    }
  }
}

resource "aws_s3_object" "architecture_marker" {
  bucket       = aws_s3_bucket.foundation.id # Natural implicit dependency.
  key          = "architecture/foundation.json"
  content_type = "application/json"

  content = jsonencode({
    project     = var.deployment.project
    environment = var.deployment.environment
    purpose     = "Non-secret foundation metadata"
  })

  # Hidden prerequisites: the bucket ID alone conveys none of these controls.
  depends_on = [
    aws_s3_bucket_public_access_block.foundation,
    aws_s3_bucket_versioning.foundation,
    aws_s3_bucket_server_side_encryption_configuration.foundation
  ]
}
```

The bucket reference establishes existence ordering. The explicit edges establish security and persistence prerequisites before writing the marker.

Versioning is not immutable backup storage. SSE-S3 is encryption at rest, not a customer-managed key policy. Sensitive workloads may require KMS controls, TLS-enforcing bucket policies, retention enforcement, and independently protected recovery copies.

Credentials Are Execution Inputs

Authenticate locally through an approved AWS IAM Identity Center profile:

```
aws sso login --profile platform-dev
export AWS_PROFILE=platform-dev
aws sts get-caller-identity
```

Verify the returned account before planning. The provider's account allow-list adds another guardrail; it does not grant permission.

Use a development-scoped role authorized only for the lab's S3 operations and account-identity lookup. CI should obtain short-lived role sessions through supported federation or workload identity rather than reuse a developer's credentials.

Do not write access keys into HCL, variable files, shell scripts, or Git. Secret masking does not prevent exposure through state, logs, or saved plans. Provider credentials and backend credentials also have separate configuration paths; protect both.

Lock Dependencies and Review Execution

Create `terraform.tfvars`, replacing the example account with your sandbox account:

```
deployment = {
  account_id  = "123456789012"
  region      = "ap-south-1"
  environment = "dev"
  project     = "orders"
}
```

Initialize and record provider selections:

```
terraform init

# Record package checksums for the team's actual execution platforms.
terraform providers lock   -platform=linux_amd64   -platform=darwin_arm64

terraform fmt -check -recursive
terraform validate
terraform plan -out=reviewed.tfplan
terraform show reviewed.tfplan
```

Commit `.terraform.lock.hcl`. Constraints specify acceptable versions; the lock file records selected provider versions and package checksums. It does not lock remote modules, which need their own version or commit constraints.[3]

After committing the lock, CI should use `terraform init -lockfile=readonly`. Perform intentional upgrades in a separate change with `terraform init -upgrade`; an exact provider pin must first be changed.

Review creation, deletion, replacement, IAM effects, and network effects before running:

```
# Apply the reviewed saved plan rather than generating a different plan.
terraform apply reviewed.tfplan

# Confirm convergence; inspect any unexpected remaining actions.
terraform plan -detailed-exitcode
```

Detailed exit codes are 0 for no changes, 1 for an error, and 2 for proposed changes. A saved plan is sensitive and time-bound: concurrent infrastructure changes still require coordination

and fresh review.

Terraform is not a transaction across AWS resources. An apply can partially succeed before failing; inspect state and reality, correct the cause, and re-plan. Do not assume automatic rollback.

For this lab, preserve local state, exclude it and plans from Git, and keep a single authorized operator. Cleanup requires deliberately removing the marker's current and historical versions before destroying the bucket; `force_destroy = false` prevents automatic data purging.

Key Takeaways

- **Configuration, state, and observed infrastructure serve different roles.** Refreshing state does not adopt console changes into HCL.
- **Terraform executes dependencies, not file order.** References create natural edges; explicit dependencies describe hidden prerequisites.
- **Resources own lifecycle; data sources read information.** Mutable queries can weaken reproducibility.
- **Pin dependencies deliberately.** Commit provider locks and independently pin CLI and module versions.
- **Validate contracts early.** Typed objects and regex checks prevent avoidable deployment mistakes.
- **Treat credentials, state, and plans as sensitive.** Use scoped short-lived identities and verify the target account.
- **Review effects, not just success messages.** Apply approved plans, verify convergence, and test application behavior.

References

[1] HashiCorp: Documentation [2] HashiCorp: Documentation [3] HashiCorp: Documentation

Chapter 3: State Is Infrastructure: Lock It, Protect It, Recover It

Two Applies, One Broken Production Environment

At 09:12, two engineers start production applies from separate laptops. Each has a copied local state file. One replaces an application instance; the other modifies its security group and deployment outputs.

Neither execution sees the other's lock. Both operate from an incomplete picture of ownership. A shared-file synchronization tool later overwrites one state copy with the other.

The application loses connectivity. The surviving state records an instance already deleted by the first apply and omits resources created by the second. The team now has both an outage and an unreliable ownership ledger.

This is logical state corruption, even if the JSON still parses. Concurrent writes through an unsafe shared filesystem can also damage the file itself. Do not blame Terraform for coordination your backend never provided.

One infrastructure ownership boundary requires one authoritative state and one coordinated writer.

Remote Storage Alone Is Not Enough

For collaborative infrastructure, a secured remote backend is mandatory. Copies in Git, chat attachments, and synchronized folders do not provide an authoritative, protected coordination mechanism.

State maps Terraform resource addresses to remote identities and stores provider-returned attributes. It is neither application data nor an infrastructure backup. Without it, configuration alone may not identify which existing resources belong to which addresses.

State can contain plaintext passwords, tokens, and private material after decryption. Marking an output `sensitive` suppresses ordinary display; it does not encrypt that value inside state.

Read access to state is a privileged capability. Encrypt storage, require TLS, restrict identities, protect local exports, and record access. Saved plans deserve equivalent treatment.

S3 native locking uses a companion object ending in `.tflock`. Terraform coordinates lock acquisition before operations that require exclusive state access. Enable `use_lockfile = true`; locking is not enabled automatically. DynamoDB-based locking is deprecated, so new deployments should use native S3 locking.[1]

A lock protects only clients using the same coordination mechanism and state identity. Two different state keys managing the same resources remain dangerous. So do clients that disable locking or write state directly.

Bootstrap Without a Circular Dependency

A backend must exist before Terraform initializes it. Therefore, create it in a separate bootstrap root using the Chapter 2 provider requirements, then configure workload roots to use it.

Initially protect bootstrap local state on an encrypted administrative workstation with restricted access and an independently protected backup. For ongoing shared administration, migrate bootstrap state into a separately administered backend whose recovery does not depend on this bucket.

Use distinct identities for backend administration, workload execution, and recovery. The workload role may read and update one state object; it must not delete the bucket, disable versioning, or schedule deletion of its KMS key.

backend-bootstrap.tf

The following belongs in the bootstrap root, not the workload root. Supply the approved account and existing execution-role name through non-secret inputs.

```
variable "account_id" {
  type     = string
  nullable = false

  validation {
    condition     = can(regex("^[0-9]{12}$", var.account_id))
    error_message = "Supply a 12-digit AWS account ID."
  }
}

variable "state_role_name" {
  type        = string
  description = "Existing, approved workload execution role."
  nullable    = false
}

provider "aws" {
  region              = "ap-south-1"
  allowed_account_ids = [var.account_id]
}

locals {
  bucket_name = "orders-prod-state-${var.account_id}"
  state_key   = "prod/aws/orders/terraform.tfstate"
}

resource "aws_kms_key" "state" {
  description             = "Terraform state encryption"
  enable_key_rotation     = true
  deletion_window_in_days = 30

  # Enable same-account IAM delegation; this is not public access.
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid       = "EnableAccountIAM"
      Effect    = "Allow"
      Principal = {
        AWS = "arn:aws:iam::${var.account_id}:root"
      }
      Action   = "kms:*"
```

```
      Resource = "*" # In a key policy, refers to this key.
    }]
  })

  lifecycle {
    prevent_destroy = true # Guardrail, not an IAM boundary.
  }
}

resource "aws_s3_bucket" "state" {
  bucket        = local.bucket_name
  force_destroy = false

  lifecycle {
    prevent_destroy = true
  }
}

resource "aws_s3_bucket_public_access_block" "state" {
  bucket                  = aws_s3_bucket.state.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "state" {
  bucket = aws_s3_bucket.state.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "state" {
  bucket = aws_s3_bucket.state.id

  rule {
    # Object-level encryption context enables narrower KMS conditions.
    bucket_key_enabled = false

    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.state.arn
    }
  }
}

resource "aws_s3_bucket_policy" "tls_only" {
  bucket = aws_s3_bucket.state.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid       = "DenyInsecureTransport"
      Effect    = "Deny"
      Principal = "*"
      Action    = "s3:*"
      Resource = [
        aws_s3_bucket.state.arn,
```

```
        "${aws_s3_bucket.state.arn}/*"
      ]
      Condition = {
        Bool = { "aws:SecureTransport" = "false" }
      }
    }]
  })

  depends_on = [aws_s3_bucket_public_access_block.state]
}

output "state_bucket" {
  value = aws_s3_bucket.state.id
}

output "state_kms_key_arn" {
  value = aws_kms_key.state.arn
}
```

Keep the Chapter 2 `terraform.required_providers` block, but replace its provider configuration with the bootstrap provider above; do not declare two default AWS providers.

`prevent_destroy` blocks covered destruction while the declaration remains present. Removing the resource declaration, using another administrative identity, or deleting through AWS can defeat that guardrail. IAM and organizational controls must enforce the actual boundary.

Versioning preserves previous object versions; it does not prevent an administrator deleting those versions. KMS protection also fails operationally if recovery identities cannot decrypt backups. Test recovery permissions before an incident.

backend.tf

Create this file only in the workload root:

```
terraform {
  backend "s3" {
    # Replace both example identifiers with bootstrap outputs.
    bucket = "orders-prod-state-123456789012"
    key    = "prod/aws/orders/terraform.tfstate"
    region = "ap-south-1"

    encrypt      = true
    use_lockfile = true

    kms_key_id = "arn:aws:kms:ap-south-1:123456789012:key/REPLACE_WITH_KEY_ID"
    allowed_account_ids = ["123456789012"]
  }
}
```

Backend blocks cannot reference ordinary variables, locals, or resource outputs. Copy verified bootstrap outputs or provide non-secret partial configuration at initialization. Never put credentials into backend arguments: configuration can persist in working directories and plan artifacts.

Native locking is implemented by Terraform's S3 backend, not the AWS provider. Use Terraform 1.10 or later, with an exact reviewed CLI version in every execution environment.

Give the Writer Only Its State

Add `state-writer-policy.tf` to the bootstrap root. This identity policy assumes the execution role already exists in the same account.

```
resource "aws_iam_policy" "state_writer" {
  name = "orders-prod-state-writer"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "ListDedicatedBackend"
        Effect   = "Allow"
        Action   = ["s3:ListBucket"]
        Resource = aws_s3_bucket.state.arn
      },
      {
        Sid    = "ReadWriteOneState"
        Effect = "Allow"
        Action = ["s3:GetObject", "s3:PutObject"]
        Resource = [
          "${aws_s3_bucket.state.arn}/${local.state_key}"
        ]
      },
      {
        Sid    = "ManageOnlyItsLock"
        Effect = "Allow"
        Action = [
          "s3:GetObject",
          "s3:PutObject",
          "s3:DeleteObject"
        ]
        Resource = [
          "${aws_s3_bucket.state.arn}/${local.state_key}.tflock"
        ]
      },
      {
        Sid    = "UseStateKeyThroughS3"
        Effect = "Allow"
        Action = [
          "kms:Encrypt",
          "kms:Decrypt",
          "kms:GenerateDataKey"
        ]
        Resource = aws_kms_key.state.arn
        Condition = {
          StringEquals = {
            "kms:ViaService" = "s3.ap-south-1.amazonaws.com"
            "kms:EncryptionContext:aws:s3:arn" = [
              "${aws_s3_bucket.state.arn}/${local.state_key}",
              "${aws_s3_bucket.state.arn}/${local.state_key}.tflock"
            ]
          }
        }
      }
    ]
  })
}
```

```
resource "aws_iam_role_policy_attachment" "state_writer" {
  role       = var.state_role_name
  policy_arn = aws_iam_policy.state_writer.arn
}
```

Bucket listing is scoped to this dedicated backend bucket; object permissions are scoped to the exact state and lock keys. This policy intentionally excludes state deletion and historical-version access. Use only the default workspace with this example; alternate workspaces change object paths.

This is backend access, not permission to provision application infrastructure. Attach independently reviewed workload permissions to the execution identity. Other policies, permission boundaries, service-control policies, and the KMS key policy still affect effective access.

SSE-KMS requires appropriate key permissions as well as S3 permissions.[2] If enabling S3 Bucket Keys later, review the encryption-context conditions because the context changes.

Migrate and Prove Coordination

Freeze writes before migration. Back up existing local state into a restricted, encrypted location and authenticate as the intended workload identity.

```
# Run in the workload root after bootstrap completes.
umask 077
terraform init -migrate-state

# Confirm migration did not introduce ownership changes.
terraform plan -lock-timeout=60s
```

Review the migration prompts; do not use unattended force-copy as a convenience. Confirm the expected key exists, encryption uses the expected KMS key, and versioning is enabled.

In an isolated lab, hold a state lock using one execution and verify a second execution waits or fails with lock information. Do not race production applies to demonstrate locking.

Test that the writer can create and remove its lock but cannot delete state, access another state key, or administer the KMS key. Record the identity, backend key, and observed result.

Do not set `-lock=false` to get past contention. A blocked apply is protecting the ownership ledger.

State Recovery Runbook

Recover the relationship between state and infrastructure, not merely an older file.

1. Freeze CI and administrative writes. Verify no active writer remains; identify the account, region, bucket, and key.
2. Preserve the current object and incident evidence using the recovery identity. Store exports securely.
3. Inspect available state versions and select a candidate based on deployment records, resource identities, lineage, and serial.
4. Compare that candidate with actual infrastructure. An older state may omit resources created after its snapshot.
5. Restore only under an approved recovery procedure, then review a normal plan without applying it.

Example inspection commands, executed with a recovery role authorized for historical reads:

```
umask 077
export STATE_BUCKET="orders-prod-state-123456789012"
export STATE_KEY="prod/aws/orders/terraform.tfstate"

aws s3api list-object-versions   --bucket "$STATE_BUCKET" --prefix "$STATE_KEY"

# Replace VERSION_ID; candidate.json contains sensitive state.
aws s3api get-object   --bucket "$STATE_BUCKET" --key "$STATE_KEY"   --version-id VERSION_ID
 ↪  candidate.json

terraform state pull > current.json

# Stops on incompatible lineage or an older serial.
terraform state push candidate.json
```

A rejected push is a safeguard, not a reason to add `-force`. Use reviewed imports, address moves, or a formally authorized state-restoration procedure where rollback of state is actually necessary. Preserve both snapshots.

For a stale lock, verify its owner has stopped and use `terraform force-unlock LOCK_ID` only against the confirmed backend. Never delete a live lock to unblock a deadline.

Complete recovery by reviewing `terraform plan`, resolving unexpected deletions or replacements, and testing application behavior. Record lost ownership information and remediation. State recovery does not restore lost database records.

Key Takeaways

- **Use one authoritative, locked backend per ownership boundary.**
- **Treat state and plans as sensitive, even when outputs are masked.**
- **Enable native S3 locking explicitly; DynamoDB locking is deprecated.**
- **Separate bootstrap, workload, and recovery authority.**
- **Scope state writes and lock deletion independently.**
- **Versioning and KMS need tested recovery access.**
- **Never force past state or lock safeguards without investigation.**

References

[1] HashiCorp S3 backend: Documentation [2] AWS SSE-KMS: Documentation

Chapter 4: AWS from Zero: Provision a Network You Can Explain

The Database That Answered the Internet

The startup's order database disappears during lunch. An attacker connected to a PostgreSQL listener exposed through a public address, guessed an administrative password, and deleted the tables. A note demands payment for a recovery key that does not exist.

The database sat in a public subnet because the engineer copied the application server's launch settings. Its security group allowed port 5432 from everywhere to simplify testing. No restore exercise had verified the backup.

A public subnet alone does not expose a database: public addressing, routing, permissive controls, and a listening service complete the path. That distinction matters because deleting an Internet Gateway route will not fix stolen credentials or recover data. **Prevent exposure at multiple boundaries, and recover from independently protected backups.**

A Route Is a Decision

A VPC establishes a private addressing domain. A subnet occupies part of that address space in one Availability Zone. A route table tells traffic where to go; it does not authenticate the sender or filter the application protocol.

A public subnet has a route to an Internet Gateway. An IPv4 instance also needs a public address and appropriate security controls for direct internet communication. A private subnet does not have that direct route. It can still reach external destinations through a NAT gateway or service endpoints.

NAT supports outbound connections and their return traffic. It is not a firewall policy engine, does not inspect application content, and does not make unrestricted egress safe. Security groups, proxies, endpoint policies, and optional inspected routing constrain destinations and identities.

Security groups are stateful. Return traffic for allowed connections is handled without a corresponding broad inbound rule. Network ACLs are stateless subnet controls; if used, their return-port rules must be understood. Start with clearly owned security groups rather than adding complicated ACLs without a test plan.

Lab and Production Are Different Architectures

The lab below creates one public and one private subnet in a selected AZ. One NAT gateway lets a private EC2 instance reach Systems Manager and approved HTTPS services. There is no public

IP on that instance and no inbound SSH rule.

Production commonly uses at least two AZs, separate application and database subnets, and an application entry point across AZs. A NAT gateway per AZ avoids making outbound access depend on another AZ and can reduce cross-AZ traffic, but increases recurring charges. Do not copy that cost into a weekend lab without a reason.

SSM Session Manager needs an instance profile, the SSM agent, service connectivity, and operator authorization. An IAM policy alone does not create the network path. VPC interface endpoints can replace selected internet paths, but require endpoint security groups, DNS configuration, and any additional endpoints needed for logging, image retrieval, or package sources.

No ingress does not mean no administration. It means administration uses an authenticated, audited control path rather than an exposed SSH listener.

Provision the Lab Root

Use Chapter 2's pinned AWS provider and Chapter 3's independently bootstrapped backend. This example is a separate root, not another file inside the backend bootstrap. Choose a currently supported AMI with SSM Agent installed, and verify the instance architecture matches it.

network.tf

```
variable "ami_id" {
  type = string
  validation {
    condition     = can(regex("^ami-[0-9a-f]+$", var.ami_id))
    error_message = "Use a reviewed regional AMI ID."
  }
}
variable "availability_zone" {
  type = string
  # Supply an AZ in the provider's region; discovery is not an architecture.
}
resource "aws_vpc" "lab" {
  cidr_block           = "10.40.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = "orders-lab" }
}
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.lab.id
  cidr_block              = "10.40.0.0/24"
  availability_zone       = var.availability_zone
  map_public_ip_on_launch = false
}
resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.lab.id
  cidr_block        = "10.40.10.0/24"
  availability_zone = var.availability_zone
}
resource "aws_internet_gateway" "lab" {
  vpc_id = aws_vpc.lab.id
}
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.lab.id
}
resource "aws_route" "internet" {
```

```
  route_table_id         = aws_route_table.public.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id             = aws_internet_gateway.lab.id
}
resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
resource "aws_eip" "nat" {
  domain = "vpc"
}
resource "aws_nat_gateway" "lab" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public.id
  # The subnet reference does not express internet-route readiness.
  depends_on = [aws_route.internet]
}
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.lab.id
}
resource "aws_route" "outbound" {
  route_table_id         = aws_route_table.private.id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id         = aws_nat_gateway.lab.id
}
resource "aws_route_table_association" "private" {
  subnet_id      = aws_subnet.private.id
  route_table_id = aws_route_table.private.id
}
```

Explicit subnet associations prevent surprises from the VPC’s main route table. Reserve address ranges for future AZs, endpoints, and containers. Avoid choosing the same range for another cloud; Chapter 20 will make overlapping ranges an expensive mistake.

access.tf

```
resource "aws_security_group" "admin" {
  name_prefix = "orders-admin-"
  description = "No inbound ports; SSM administration"
  vpc_id      = aws_vpc.lab.id
}
resource "aws_vpc_security_group_egress_rule" "https" {
  security_group_id = aws_security_group.admin.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = 443
  to_port           = 443
  # Lab baseline: production narrows destinations with endpoints/proxy controls.
}
resource "aws_iam_role" "instance" {
  name_prefix = "orders-ssm-"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
      Action = "sts:AssumeRole"
    }]
```

```
  })
}
resource "aws_iam_role_policy_attachment" "ssm" {
  role       = aws_iam_role.instance.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}
resource "aws_iam_instance_profile" "instance" {
  role = aws_iam_role.instance.name
}
resource "aws_instance" "admin" {
  ami                         = var.ami_id
  instance_type               = "t3.micro"
  subnet_id                   = aws_subnet.private.id
  associate_public_ip_address = false
  vpc_security_group_ids      = [aws_security_group.admin.id]
  iam_instance_profile        = aws_iam_instance_profile.instance.name
  metadata_options {
    http_tokens = "required" # Require IMDSv2 session tokens.
  }
  root_block_device {
    encrypted = true
  }
  depends_on = [
    aws_iam_role_policy_attachment.ssm,
    aws_route.outbound,
    aws_route_table_association.private
  ]
}
output "instance_id" { value = aws_instance.admin.id }
```

AWS-provided DNS resolution has special behavior; a missing arbitrary UDP-53 internet rule is not permission to assume every DNS design works. Test resolution from the actual instance. If using your own resolver, explicitly allow that resolver and document forwarding paths.

Verify Behavior, Not Just Objects

Run formatting, initialization, validation, and a reviewed saved plan before applying. IAM propagation can be eventual; do not hide persistent authorization errors behind unlimited retries. Verify which instance role was actually attached.

```
terraform output -raw instance_id
aws ssm describe-instance-information
# Replace the ID with this root's output.
aws ssm start-session --target i-REPLACE
```

An operator needs separate Session Manager permissions. The instance role does not authorize your human identity to start sessions. Restrict operator access to tagged lab instances, require MFA through your identity provider, and configure session audit destinations. Port forwarding has different logging capabilities from an interactive shell; review it as a separate access mode.

Inside the session, verify DNS resolution and HTTPS reachability. Confirm the instance has no public IPv4 address and the security group has no inbound rules. Check the private route table points to the intended NAT, not directly to the Internet Gateway.

For the future database, authorize port 5432 from the application security group, not from the entire VPC range. Put administrative tooling behind a separate identity and network boundary. A compromised application must not inherit database administrator privileges merely because its packets originate inside a VPC.

Record the cost of the NAT gateway, allocated public IPv4 address, instance, EBS storage, and logs. Stop or destroy disposable lab resources after verification; stopping EC2 does not stop NAT or retained-storage charges. Inspect the destroy plan and never combine the lab root with protected state infrastructure.

Diagnose the First Broken Hop

When SSM does not register the instance, divide the investigation by boundary. First confirm the instance is running with the expected AMI and role. Then check the agent process and its logs through an approved out-of-band lab access method. Next inspect DNS, outbound route, NAT health, and endpoint connectivity. Finally review service authorization.

Do not add inbound SSH from the internet as the default diagnostic action. That creates another exposure path without identifying the original failure. If an emergency access mechanism is required, predefine its restricted source, short lifetime, approval owner, and removal verification.

For application connectivity, write a hop table: caller security group, source subnet route, destination subnet route, destination security group, service listener, TLS identity, and database role. A timeout suggests a different boundary from an authentication rejection. Preserve that distinction in alerts and runbooks.

Use VPC Flow Logs where appropriate to inspect accepted or rejected flows, understanding that they are metadata rather than full packet captures. Their delivery and retention have cost and privacy implications. An entry showing accepted packets does not prove PostgreSQL accepted the transaction.

The lab acceptance record should contain both a successful administrative session and evidence that unsolicited inbound access remains unavailable. This makes the security claim measurable instead of relying on a diagram.

Key Takeaways

- **Routing, addressing, and authorization are separate controls.**
- **A private subnet needs an intentional outbound and administrative path.**
- **SSM requires identity, agent health, and connectivity.**
- **Keep databases private and restrict callers by workload identity and security group.**
- **Label single-AZ labs honestly; design multi-AZ production deliberately.**
- **Test both permitted and denied paths, then clean up chargeable resources.**

References

- AWS VPC route tables: Documentation
- Session Manager prerequisites: Documentation

Chapter 5: Modules with Boundaries: Reuse Infrastructure Without Coupling Everything

A Development Tag That Replaced Production

The engineer changed `environment = "dev"` to clean up inconsistent tags. The same root generated names for development and production load balancers. A poorly chosen `for_each` key included the environment value, so Terraform interpreted the change as removing old resource addresses and creating new ones.

The reviewed plan contained replacements. The pipeline applied them with production credentials because everything shared one state. A development housekeeping request became a production outage.

A tag update does not inherently destroy a load balancer. The failure was the surrounding address and naming design, combined with an overly broad ownership boundary. **Do not make harmless metadata define resource identity unless replacement is intentional.**

Root Modules Are Operational Boundaries

A root module is the directory where Terraform executes. It defines provider configuration, backend selection, child-module calls, variables, and outputs. Its state establishes the resources that execution owns.

A child module groups reusable declarations behind an input/output contract. It inherits or explicitly receives provider configurations. Calling it does not create an independent state boundary: its resources remain owned by the calling root.

A child module named `production` inside the development root is still part of that root's state. Directory names do not prevent a development credential from operating on production resources if the provider identity permits it.

Choose roots around change cadence, blast radius, identity, and recovery ownership. Shared networking may deserve one root, while application infrastructure lives elsewhere. Avoid a root for every resource: excessive fragmentation creates dependency orchestration and recovery problems of its own.

Workspaces Are State Namespaces, Not Security Walls

CLI workspaces provide multiple states for the same configuration. They do not automatically change credentials, isolate cloud accounts, protect backend access, or require a different reviewer.

A workspace can be useful for ephemeral environments with equivalent trust requirements. It is a poor substitute for separating production and developer permissions. An engineer who can select the production workspace and assume its role already crosses the boundary.

This book uses separate development and production roots, accounts, state keys, and apply identities. Shared child modules provide reuse without granting shared administrative scope. The configuration contracts remain similar; the authority does not.

Keep Modules Small Enough to Explain

A network module should own networking. It should not create a database, configure Jenkins, obtain certificates, and deploy Helm charts because all four eventually use that network.

A stable module has explicit inputs, meaningful outputs, predictable ownership, and a documented upgrade path. Avoid dozens of boolean switches that create incompatible internal modes. If two configurations share only a filename, they may not belong in one module.

Expose identifiers consumers need rather than entire provider objects. Returning a whole subnet object makes callers depend on provider details your module should be free to change. Useful contracts include a VPC ID, subnet-ID map, and address-space documentation.

Reuse implementation; keep execution and authorization boundaries explicit.

Repository Contract

```
platform/
  modules/
    network/
      versions.tf
      variables.tf
      main.tf
      outputs.tf
  environments/
    dev/
      main.tf
      providers.tf
      backend.tf
      terraform.tfvars.example
    prod/
      main.tf
      providers.tf
      backend.tf
      terraform.tfvars.example
```

Each environment has a different backend key and allowed account ID. Neither commits credentials or local state. A separate root deploys the backend itself.

modules/network/versions.tf

```
terraform {
  required_version = ">= 1.10.0, < 2.0.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 6.66.0, < 7.0.0"
      # Child contract permits compatible callers; root pins selection.
```

```
    }
  }
}
```

modules/network/variables.tf

```
variable "network" {
  nullable = false
  type = object({
    name = string
    cidr = string
    subnets = map(object({
      cidr = string
      az   = string
    }))
  })
  validation {
    condition     = can(cidrnetmask(var.network.cidr))
    error_message = "Provide a valid IPv4 VPC CIDR."
  }
  validation {
    condition = alltrue([
      for subnet in values(var.network.subnets) :
      can(cidrnetmask(subnet.cidr))
    ])
    error_message = "Every subnet requires a valid IPv4 CIDR."
  }
  validation {
    condition     = length(var.network.subnets) > 0
    error_message = "Define at least one subnet."
  }
}
```

These validations check syntax, not full allocation correctness. Add tests for containment, overlap, and AZ placement. AWS also rejects invalid allocation; discovering those errors at apply time is an expensive feedback loop.

modules/network/main.tf

```
resource "aws_vpc" "this" {
  cidr_block           = var.network.cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = var.network.name }
}
resource "aws_subnet" "this" {
  # Stable keys express topology, never mutable display labels.
  for_each = var.network.subnets
  vpc_id   = aws_vpc.this.id
  cidr_block        = each.value.cidr
  availability_zone = each.value.az
  map_public_ip_on_launch = false
  tags = { Name = "${var.network.name}-${each.key}" }
}
```

modules/network/outputs.tf

```
output "vpc_id" { value = aws_vpc.this.id }
output "subnet_ids" {
  value = {
    for key, subnet in aws_subnet.this : key => subnet.id
  }
}
```

This module deliberately does not establish outbound connectivity. Subnets alone are not a complete application platform. Route-table ownership belongs to an explicitly chosen extension or separate routing module; use Chapter 4 to understand the resources before adding them.

environments/dev/main.tf

```
module "network" {
  source = "../../modules/network"
  network = {
    name = "orders-dev"
    cidr = "10.40.0.0/16"
    subnets = {
      app_a = { cidr = "10.40.10.0/24", az = "ap-south-1a" }
    }
  }
}
output "vpc_id" { value = module.network.vpc_id }
```

environments/prod/main.tf

```
module "network" {
  source = "../../modules/network"
  network = {
    name = "orders-prod"
    cidr = "10.50.0.0/16"
    subnets = {
      app_a = { cidr = "10.50.10.0/24", az = "ap-south-1a" }
      app_b = { cidr = "10.50.20.0/24", az = "ap-south-1b" }
    }
  }
}
output "vpc_id" { value = module.network.vpc_id }
```

AZ letters are account-specific mappings; cross-account coordination should use AZ IDs where physical placement matters. The displayed inputs are regional examples, not universal allocations.

Provider and State Isolation

In each root, keep the exact AWS provider pin from Chapter 2. Configure `allowed_account_ids` with that environment's approved account. Configure backend access to its own state key, and have CI obtain only the corresponding role.

A developer role should fail authorization if pointed at production state. Conversely, production execution must not silently fall back to developer credentials. Validate identity before initialization and again before apply when credential sessions may change.

Local module sources follow the checked-out repository commit. Registry modules require an explicit version. Git module sources should use a reviewed immutable commit reference. The provider dependency lock file does not lock module content.

Maintain an upgrade change that states what changed in the contract, what resources move or replace, and how rollback works. A module refactor can move Terraform addresses without changing AWS infrastructure; use reviewed `moved` blocks instead of destroying and recreating the objects.

```
# Keep this migration in the affected root for upgrading existing state.
moved {
  from = aws_vpc.legacy
  to   = module.network.aws_vpc.this
}
```

Acceptance and Recovery

Plan development first, then production with production inputs and identity. Do not infer that production is safe because development applied. Capacity, policies, existing resources, and network dependencies can differ.

Test that changing `network.name` updates tags without changing the stable `for_each` keys. Test that deleting `app_b` proposes deleting only the intended subnet and reveals dependent resources before execution. Exercise the account allow-list using a sandbox identity, not a production mutation.

Keep plans and state snapshots protected. If a module upgrade partially applies, inspect which resources succeeded and re-plan from the updated authoritative state. Do not restore an old state merely to make old code appear consistent.

Document which root owns each route, subnet, security group, and endpoint. When transferring ownership between roots, freeze both writers and use a reviewed state-transfer or import procedure. Two roots must never independently reconcile the same object.

Cleanup follows the dependency graph: workloads first, then network consumers, then networking. Backend administration remains independent. A reusable module is successful when another engineer can predict its plan without reading every resource implementation.

Review a Module as a Product Contract

A module review should answer four questions: what resources it owns, what assumptions its inputs make, what its outputs promise, and what changes can replace existing infrastructure. Put those answers in the module README with a minimal example and migration notes.

Test invalid CIDRs and missing topology entries before testing a happy path. Then exercise a metadata change and a topology change separately. The first should have the documented non-destructive effect; the second should reveal the expected replacement or removal. Preserve the actual plan as restricted evidence.

Treat output changes as interface changes. Renaming a subnet key can break downstream configuration even if AWS resources remain intact. Publish deprecation windows or migration instructions rather than expecting every caller to discover the change during apply.

Ownership is also a contract. If the network module begins creating routes previously managed by another root, coordinate the transfer before either writer applies. The source-code refactor is the easy part; resource adoption and exclusive authority determine whether production survives it.

Key Takeaways

- **Child modules organize code; roots establish execution ownership.**
- **Separate production identities, backends, and approvals.**
- **Workspaces do not create security boundaries.**
- **Stable keys preserve identity during metadata changes.**
- **Pin module sources independently of providers.**
- **Review address migrations and recovery paths before upgrades.**

References

- Terraform modules: Documentation
- Workspaces: Documentation

Chapter 6: Build Once, Run Consistently: Containerize the Application

Friday's Different Runtime

The release worked on the developer's laptop. Production fails during startup because the deployment server resolves a newer dependency, uses a different interpreter, and lacks a shared library. Someone installs packages directly on the host while customers retry their orders.

The next deployment fails differently. Yesterday's emergency packages remain, but nobody recorded them. The server is now an undocumented build environment and runtime simultaneously.

Containers address packaging consistency; they do not eliminate architecture, kernel, networking, or configuration differences. **Build a reviewed artifact once, then deploy that artifact by immutable digest.** Do not build production from whichever source happens to be on the server.

Define the Application Contract

This chapter uses a small Python HTTP API with the standard library, keeping the example executable without fabricating dependency hashes. The API demonstrates readiness, health, environment configuration, and termination. It is a teaching service, not the full order API: database transactions and authentication arrive in a real application implementation.

A production dependency set must be locked and verified separately. For Python packages, use an approved lock workflow and hashed requirements for the selected platform. Native dependencies can differ by architecture; testing on an ARM laptop does not validate an AMD64 release.

The process listens on port 8080, exposes `/healthz` and `/readyz`, writes no persistent data locally, and reads configuration at startup. The real order service must make readiness represent its ability to serve requests without turning every transient database slowdown into mass restart behavior.

app.py

```
import json
import os
import signal
import threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
```

```
ready = True

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path not in ('/healthz', '/readyz', '/'):
            self.send_error(404)
            return
        code = 200 if self.path != '/readyz' or ready else 503
        body = json.dumps({'status': 'ok', 'service': 'orders-api'}).encode()
        self.send_response(code)
        self.send_header('Content-Type', 'application/json')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

server = ThreadingHTTPServer(('0.0.0.0', int(os.getenv('PORT', '8080'))), Handler)

def stop(signum, frame):
    global ready
    ready = False
    # shutdown must run outside serve_forever's thread to avoid deadlock.
    threading.Thread(target=server.shutdown, daemon=True).start()

signal.signal(signal.SIGTERM, stop)
signal.signal(signal.SIGINT, stop)
try:
    server.serve_forever()
finally:
    server.server_close()
```

This fixture is intentionally small. Replace the server with your reviewed application framework and production serving stack when introducing real traffic. Set bounded request timeouts, authentication, input limits, and dependency pools; packaging does not supply those controls automatically.

Separate Build Tools from Runtime

Multi-stage builds separate tools needed to construct software from tools needed to execute it. Compilers, source-control clients, and development dependencies should not automatically enter the runtime image.

The example compiles Python sources in the build stage and copies only the application into runtime. Both stages use the same approved base reference to avoid an interpreter mismatch. Since this application has no external packages, a dependency lock file is unnecessary here; adding packages changes that obligation.

Dockerfile

```
# Supply a verified digest, e.g. python:<reviewed-tag>@sha256:<digest>.
ARG PYTHON_BASE
FROM ${PYTHON_BASE} AS build
WORKDIR /src
COPY app.py ./app.py
RUN python -m compileall -q /src

FROM ${PYTHON_BASE} AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 \
```

```
    PYTHONUNBUFFERED=1 \
    PORT=8080
WORKDIR /app
# Numeric identity works without relying on host user names.
RUN groupadd --gid 10001 app \
 && useradd --uid 10001 --gid 10001 --no-create-home app
COPY --from=build --chown=10001:10001 /src/ /app/
USER 10001:10001
EXPOSE 8080
STOPSIGNAL SIGTERM
# Exec form makes Python receive the container's termination signal.
CMD ["python", "/app/app.py"]
```

Select a Debian-based official Python image with the `groupadd` and `useradd` tools; do not substitute an Alpine base without changing those commands. The required build argument avoids silently using a floating default. Store the approved digest in reviewed build configuration.

.dockerignore

```
.git
.env
.env.*
!.env.example
*.pem
*.key
__pycache__
.venv
backups
terraform.tfstate*
*.tfplan
```

A small build context reduces accidental inclusion and improves caching. Ignore rules are not a secret scanner. A credential copied into an earlier layer can remain recoverable after deletion in a later layer. Use BuildKit secret mounts for build-time authentication, and ensure build tools never echo or persist those values.

Local Development Without Public Databases

Compose provides a reproducible local topology. It does not provide production high availability, cloud identity, or a disaster-recovery plan. Separate local convenience from deployment expectations.

docker-compose.yml

```
services:
  api:
    build:
      context: .
      args:
        PYTHON_BASE: ${PYTHON_BASE:?Set an approved Python digest}
    ports:
      - "127.0.0.1:8080:8080" # Restrict the development listener.
    read_only: true
    tmpfs:
      - /tmp:rw,noexec,nosuid,size=16m
```

```
    cap_drop: [ALL]
    security_opt:
      - no-new-privileges:true
    stop_grace_period: 30s
    environment:
      PORT: "8080"
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8080/healthz',
↪  timeout=2)"
      interval: 10s
      timeout: 3s
      retries: 3
```

There is no bind mount over `/app` in this release-like topology. A developer may add an explicit override with source mounting and reload behavior, but must understand that it bypasses the built artifact. Never test a bind-mounted tree and assume the separately built image contains identical files.

Build, Test, and Record the Artifact

Use an actual approved digest, not the placeholder text below. A digest establishes identity, not trust: review the publisher, provenance, architecture, and vulnerability findings.

```
export PYTHON_BASE='python:REVIEWED_TAG@sha256:REVIEWED_DIGEST'
docker compose config --quiet
docker compose build --pull
docker compose up -d
curl --fail http://127.0.0.1:8080/readyz
docker compose logs --no-color api
```

Verify the container executes as UID 10001 and writing to `/app` fails. Confirm `/tmp` is the only writable temporary location you intended. Test termination with an in-flight request and check the process exits before the grace period expires.

```
docker compose exec api id
docker compose stop -t 30 api
docker compose up -d
```

A production API must stop accepting new work, drain active requests, and release connections. Workers must stop claiming new jobs, complete or safely abandon current work, and acknowledge only after durable side effects. SIGTERM handling and idempotency are linked: a forced shutdown must not silently lose an acknowledged order.

Promote, Do Not Rebuild

CI builds the image, tests it, scans it, and pushes it to a registry. Record the registry digest and deploy `repository@sha256:...`. A Git commit tag is useful traceability but can still be overwritten unless registry policy prevents it.

Maintain an evidence record containing source commit, base digest, build platform, dependency lock revision, test results, scan policy, and final image digest. A release approval covers that record and its configuration, not just a human-readable tag.

Multi-platform releases require testing each supported platform or a documented test strategy. Emulated tests may be slower and do not prove all native performance behavior. Build-cache reuse also must not introduce secret material or untrusted artifacts into a trusted release.

Keep runtime secrets outside images. Inject short-lived credentials or secret references through the deployment platform. Environment variables may be inspectable by administrators; mounted secret files still need access controls and rotation handling. Choose based on the actual threat model.

Persistent data belongs outside the writable layer. Application logs go to standard output or a configured collection path, with retention owned by the logging system. Avoid writing database files, uploaded customer documents, or release identity into a disposable container filesystem.

Cleanup with `docker compose down`; this example has no persistent volumes. Once a database is added, never casually add `--volumes`. The next chapter explains why that option belongs in a deliberate data-destruction procedure.

A Dependency Change Is a New Release

When the application gains external packages, introduce a reviewed dependency lock and build from that lock. A file listing only broad version ranges allows package resolution to change independently of the application commit. That weakens the claim that the same source produces the same artifact.

For packages fetched from a private index, supply authentication through a build secret mount and confirm the installer does not write credentials into configuration copied to runtime. Inspect the resulting image filesystem and history using a synthetic credential during a controlled test. Do not experiment with a real production token.

Native extensions require compatible runtime libraries. A builder may compile successfully against a library absent from the final image. Run the actual final image's import and startup tests instead of concluding success from the build stage.

Track rebuild triggers separately from application changes. A base-image security update creates a new artifact and needs verification even if business code is unchanged. Retain the old approved digest until rollback and recovery policies permit deletion.

Image size is an operational input: larger images increase cold-start transfer time and registry dependence during recovery. Measure compressed pull size and startup duration on the intended node environment. Do not optimize size by removing certificate authorities or diagnostic capability without understanding the consequences.

The release fixture deliberately avoids pretending that a fabricated lock file proves dependency integrity. Once dependencies are real, their ownership, updates, and verification become part of the repository's acceptance gates.

Key Takeaways

- **Containers package an execution contract; they do not prove production readiness.**
- **Pin the base image and promote the tested final digest.**
- **Keep build tools, secrets, and durable data outside runtime layers.**
- **Run non-root with explicit writable locations and minimal capabilities.**
- **Test signal handling, readiness, and the actual built image.**
- **Treat dependency and architecture selection as reviewed inputs.**

References

- Docker multi-stage builds: Documentation
- Build secrets: Documentation

Chapter 7: Containers Die; Data Must Survive: Understand Docker Storage

The Image Update That Deleted Orders

The engineer pulls a newer PostgreSQL image, removes the old container, and launches a replacement. Startup succeeds. The application connects successfully. Every customer order is gone.

PostgreSQL had written its data directory into the old container's writable layer. Removing that container deleted the only copy. The team mistook a running database process for protected data.

Restarting a container ordinarily preserves its writable layer; deleting and replacing it does not. Distinguish those operations before writing a runbook. **Data survival must be demonstrated across replacement, not inferred from restart success.**

Three Storage Lifecycles

The writable layer belongs to one container. It is convenient for disposable process state but unsuitable as the sole durable copy of a database. Image layers are read-only inputs; runtime database writes do not become part of the image.

A named volume is managed independently by Docker. Removing a container normally leaves the volume. A replacement container can mount the same volume, provided it runs on the same storage host or a deliberately configured volume driver.

A bind mount maps a specific host path into the container. It gives explicit control over placement, backup tools, and permissions, but makes host filesystem assumptions part of the deployment contract. An empty or wrong directory can mask the expected container path.

Neither named volumes nor bind mounts automatically protect against disk failure, host termination, malicious deletion, or application corruption. They solve a lifecycle boundary, not every recovery requirement.

Storage	Survives container replacement	Survives host loss by itself	Typical use
Writable layer	No	No	Disposable cache or temporary state
Local named volume	Yes, if retained	No	Local database lab
Bind mount	Yes, if directory retained	No	Deliberately managed host storage

Storage	Survives container replacement	Survives host loss by itself	Typical use
External durable service	Depends on service	Depends on architecture	Production state with independent recovery

Establish the Data Contract

This lab uses PostgreSQL 17 and its `/var/lib/postgresql/data` mount path. Later major image families can change their recommended volume layout; inspect the selected image documentation before upgrading. Pin a reviewed image digest through `POSTGRES_IMAGE`.

Create a password file locally, protect it, and keep it outside Git. The image reads it during initial database setup. Changing that file does not automatically rotate an existing database password.

```
mkdir -p secrets
chmod 700 secrets
umask 077
# Generate locally; do not print the secret into shared logs.
openssl rand -base64 32 > secrets/postgres-password
```

docker-compose.yml

```
services:
  db:
    image: ${POSTGRES_IMAGE:?Set a reviewed PostgreSQL 17 digest}
    restart: unless-stopped
    environment:
      POSTGRES_DB: orders
      POSTGRES_USER: orders_admin
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - orders_data:/var/lib/postgresql/data
    # No host port: only approved local containers need this database.
    healthcheck:
      test: [CMD-SHELL, "pg_isready -U orders_admin -d orders"]
      interval: 5s
      timeout: 3s
      retries: 10
    stop_grace_period: 60s
volumes:
  orders_data:
    # A stable name makes project-directory changes less surprising.
    name: orders_lab_pgdata
secrets:
  db_password:
    file: ./secrets/postgres-password
```

Compose secrets in this local configuration are mounted files, not an encrypted cloud secret-management system. Administrators with access to the file or Docker daemon can retrieve them. Do not represent this lab mechanism as production credential isolation.

The initialization user is powerful. A real application requires a separate least-privilege database role with only the permissions its transactions need. Keep schema migration privileges separate when possible. The lab administrator account simplifies the demonstration, not the production

threat model.

Permissions Must Match the Image

The official PostgreSQL image may start its entrypoint with elevated privilege to prepare the data directory and then switch identity. Hard-coding `user: 10001` from the API example can break initialization.

Inspect the selected image's documented identity and entrypoint behavior. For bind mounts, set ownership for that image and host policy; do not assume PostgreSQL always uses a universal numeric UID. Rootless Docker and user-namespace remapping add another translation layer.

Never solve a permission failure with `chmod -R 777`. That permits unrelated processes to modify database files and hides an incorrect ownership contract. Document the mount source, filesystem, required identity, and permitted backup identity.

Prove Container Replacement Is Safe

Start the database after setting a real reviewed digest. Keep one Compose project directory and verify the volume attachment.

```
docker compose config --quiet
docker compose up -d --wait
docker compose exec -T db psql -U orders_admin -d orders \
  -v ON_ERROR_STOP=1 -c \
  "CREATE TABLE survival_test (id integer PRIMARY KEY, note text);"
docker compose exec -T db psql -U orders_admin -d orders \
  -v ON_ERROR_STOP=1 -c \
  "INSERT INTO survival_test VALUES (1, 'survives replacement');"
```

Now remove only the container. This is a controlled lab exercise using synthetic data; do not substitute a production database name.

```
docker compose stop db
docker compose rm -f db
docker volume inspect orders_lab_pgdata
docker compose up -d --wait db
docker compose exec -T db psql -U orders_admin -d orders \
  -v ON_ERROR_STOP=1 -c "SELECT * FROM survival_test;"
```

The row must remain. Record the volume mount before and after replacement. If a new empty database appears, inspect the actual mount and project naming before touching either volume; your old data may still exist under a different name.

`docker compose down --volumes`, `docker volume rm`, and some cleanup workflows intentionally delete volume data. Routine application deployment must not include those operations. A broad pruning command can also remove unused volumes; an unmounted recovery volume is still valuable.

Backup Is an Application Operation

Copying a live PostgreSQL data directory with a filesystem tool is not a generally valid database backup. PostgreSQL has specific physical backup and write-ahead-log requirements. For this small lab, use a logical dump.

```
mkdir -p backups
chmod 700 backups
```

```
umask 077
docker compose exec -T db pg_dump \
  -U orders_admin -d orders -Fc \
  > backups/orders.dump
# Inspect the archive catalog; successful inspection is not a restore test.
docker compose exec -T db pg_restore --list \
  < backups/orders.dump
```

A custom-format dump supports structured restoration, but one database dump does not include every cluster-wide role or configuration. Capture required roles, extensions, settings, and encryption dependencies separately. Protect the dump as customer data, not as an innocuous build artifact.

Restore into a disposable database on the lab instance, not over the primary. Use a unique test name and verify row counts and representative application queries.

```
docker compose exec -T db createdb \
  -U orders_admin orders_restore_test
docker compose exec -T db pg_restore \
  -U orders_admin -d orders_restore_test \
  --exit-on-error --no-owner < backups/orders.dump
docker compose exec -T db psql \
  -U orders_admin -d orders_restore_test \
  -c "SELECT * FROM survival_test;"
```

A restore on the same host demonstrates dump usability, not disaster recovery. Copy encrypted backups to independently protected storage and rehearse restoration onto a replacement host with recovered credentials. Measure elapsed recovery and the newest recoverable transaction.

Upgrades and Destruction Need Their Own Workflow

Do not attach PostgreSQL 17 data files to an arbitrary later major image and expect a safe upgrade. Major upgrades require a supported migration process such as `pg_upgrade`, logical migration, or managed-service procedures. Test extensions, collations, clients, and rollback boundaries.

Before upgrading, capture an application-consistent recovery point and prove it can restore. Prefer a tested rehearsal using a representative dataset. A successful container startup is insufficient: check migrations, query results, application writes, and background processing.

For cleanup, stop the lab containers first. Decide whether to retain the named volume and backup. If deletion is intentional, record exactly which volume will be removed and verify it does not belong to another project. Keep destructive cleanup separate from commands readers run to demonstrate replacement survival.

Persistence is a property of the storage lifecycle; recoverability is a property of the entire tested recovery procedure. The next chapter applies that distinction to Jenkins, where configuration, encrypted credentials, and history have different backup requirements from application orders.

Detect an Empty Mount Before Writing

A database that unexpectedly initializes a fresh cluster may indicate a wrong mount rather than deleted data. Stop application writes, inspect the container's mount source, and compare it with the expected volume identity. Do not copy new files over the old volume until ownership is understood.

Record the initialized database identifier and schema revision in operational inventory. Startup checks can detect a missing expected schema and fail closed rather than accepting an empty database as healthy. Such checks must distinguish a deliberate new environment from an accidental replacement mount.

This protects against a common failure: the system looks available while silently accepting new orders into a different database. Recovery then becomes reconciliation of two write histories rather than restoration of one missing service.

Key Takeaways

- **Container restart and container replacement have different data consequences.**
- **Named volumes outlive containers, not necessarily hosts.**
- **Match mount paths and ownership to the selected image version.**
- **Do not copy live database files casually or use world-writable permissions.**
- **Prove both replacement survival and restoration from backup.**
- **Treat volume deletion and database upgrades as deliberate operations.**

References

- Docker volumes: Documentation
- PostgreSQL backup: Documentation

Chapter 8: Jenkins That Survives Monday: Docker Volume Persistence on EC2

The Spot Termination That Erased CI

A Spot interruption terminates the EC2 instance hosting Jenkins. The replacement image contains Docker and Jenkins, but no jobs, plugins, credentials, or build history. Every deployment pipeline depended on files inside the terminated instance's root disk.

The root EBS volume was configured to delete on termination. The team had a container restart policy and believed it had persistence. That policy restarts a process on a surviving host; it does not recover a deleted disk.

Ephemeral compute requires separately owned storage and independently recoverable backups. A retained EBS volume addresses one failure boundary. An AZ outage, encryption-key loss, or compromised administrator still demands a different recovery path.

Own the Controller's Data

Jenkins stores its configuration under `/var/jenkins_home` in the official container image. Jobs, plugin binaries, build metadata, credentials, and encryption material live beneath that home. The exact contents matter: restoring encrypted credentials without the corresponding key material does not restore usable credentials.

Use encrypted EBS for the controller's persistent home, retained independently of the root disk. Prefer predictable On-Demand controller capacity unless the interruption and restore procedure is genuinely acceptable. Spot can be valuable for disposable agents, where interrupted work is retried safely.

Place the controller in a private subnet. Use SSM for administration and an authenticated TLS entry point for users. Do not expose port 8080 to the internet. The command below binds only to host loopback; a separate trusted reverse proxy must provide the external authenticated route.

Set built-in executors to zero in Chapter 9. Controller persistence does not justify executing arbitrary builds there. Docker-daemon access is highly privileged and belongs only on deliberately isolated build infrastructure.

Select the Disk Before Formatting

On Nitro instances, EBS devices appear as NVMe devices, and Linux names may change. Never assume `/dev/nvme1n1` is the new empty disk because a blog used that path.

Identify the provisioned volume using the EC2 attachment, NVMe serial information, and expected size. Verify it has no mounted filesystem or existing data. The following commands use a variable to force explicit device selection.

```
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS,SERIAL
sudo nvme list
# Replace only after matching the approved EBS volume ID and size.
export JENKINS_DEVICE=/dev/nvmeREPLACE
sudo wipefs --no-act "$JENKINS_DEVICE"
sudo blkid "$JENKINS_DEVICE" || true
```

`wipefs --no-act` inspects signatures without erasing them. If a filesystem exists, stop and investigate. A restored EBS volume should be mounted, not formatted. Formatting is a destructive one-time action on a confirmed blank lab disk.

```
# DESTRUCTIVE: only for a verified new empty EBS volume.
sudo mkfs.ext4 "$JENKINS_DEVICE"
sudo mkdir -p /srv/jenkins
sudo mount "$JENKINS_DEVICE" /srv/jenkins
sudo blkid -s UUID -o value "$JENKINS_DEVICE"
```

Capture the returned UUID. It identifies the filesystem across device-name changes. Check cloud volume encryption separately; an ext4 UUID says nothing about KMS access or EBS encryption.

Mount Reliably and Fail Closed

Add one reviewed entry to `/etc/fstab`. Back up the existing file and avoid adding duplicate entries during repeated provisioning.

```
sudo cp /etc/fstab /etc/fstab.before-jenkins
# Replace UUID_FROM_BLKID with the actual value.
printf '%s\n' \
  'UUID=UUID_FROM_BLKID /srv/jenkins ext4 defaults 0 2' \
  | sudo tee -a /etc/fstab
sudo findmnt --verify
sudo mount -a
findmnt /srv/jenkins
```

The omission of `nofail` is intentional: silently starting Jenkins on an unmounted directory can create a second home on the root disk. Boot behavior should match your operational preference, but the service must always fail closed when the intended filesystem is unavailable.

For automated startup, use a service dependency that requires the mount. A plain Docker restart policy can race storage mounting after reboot. Use systemd to control the container lifecycle and omit Docker's automatic restart policy when systemd owns retries.

```
# /etc/systemd/system/jenkins-container.service
[Unit]
Description=Persistent Jenkins controller
Requires=docker.service
After=docker.service
RequiresMountsFor=/srv/jenkins

[Service]
EnvironmentFile=/etc/jenkins-container.env
ExecStartPre=/usr/bin/mountpoint -q /srv/jenkins
ExecStartPre=-/usr/bin/docker rm jenkins
ExecStart=/usr/bin/docker run --name jenkins --init --publish 127.0.0.1:8080:8080 --mount
 ↪  type=bind,src=/srv/jenkins/home,dst=/var/jenkins_home --env
 ↪  JAVA_OPTS=-Djenkins.install.runSetupWizard=true ${JENKINS_IMAGE}
```

```
ExecStop=/usr/bin/docker stop --time 60 jenkins
Restart=on-failure
RestartSec=10
TimeoutStopSec=90

[Install]
WantedBy=multi-user.target
```

Set `JENKINS_IMAGE` in the environment file to an approved official Jenkins LTS image digest. This file contains image configuration, not cloud keys. Match the Docker binary path to the distribution before enabling the unit.

Match Ownership to the Image

The official Jenkins image normally uses UID/GID 1000. Verify the actual selected image rather than trusting a universal number.

```
export JENKINS_IMAGE='jenkins/jenkins:REVIEWED_LTS@sha256:REVIEWED_DIGEST'
docker run --rm --entrypoint id "$JENKINS_IMAGE"
sudo mkdir -p /srv/jenkins/home
# Use 1000:1000 only after confirming the selected image's identity.
sudo chown 1000:1000 /srv/jenkins/home
sudo chmod 750 /srv/jenkins/home
```

For a manual first start, use the following command instead of starting the systemd service simultaneously:

```
mountpoint -q /srv/jenkins || exit 1
docker run -d --name jenkins --init \
  --publish 127.0.0.1:8080:8080 \
  --mount type=bind,src=/srv/jenkins/home,dst=/var/jenkins_home \
  "$JENKINS_IMAGE"
```

The `--mount` form rejects a nonexistent bind source rather than conveniently creating an unexpected directory. The mountpoint check prevents writing into the root filesystem when EBS is absent. Do not mount the host Docker socket into the controller to make a pipeline work.

Retrieve the initial administrator secret through the protected administrative session. Do not paste it into CI logs or the repository. Complete setup through the approved private access route, then configure authentication, authorization, TLS, and controller executors before onboarding users.

Demonstrate Replacement Survival

Create a disposable test job, stop Jenkins cleanly, remove only the container, and start the exact same image with the same mount. Verify the job and configuration remain.

```
docker stop --time 60 jenkins
docker rm jenkins
# Repeat the manual docker run command using the same mount and image.
```

Reboot the instance after configuring the systemd unit. Confirm the EBS filesystem mounts before Jenkins starts. Deliberately test a missing-volume scenario in a lab: Jenkins must fail to start, not create a new empty home.

Moving a retained EBS volume requires a replacement instance in the same AZ. Stop the old writer and detach cleanly before attaching elsewhere. Never start two controllers against the

same home. EBS volume attachment does not implement an active-active Jenkins architecture.

Backup and Recovery Are Separate

For this single-controller lab, stop Jenkins to take a consistent home backup, or use an explicitly supported consistency procedure. EBS snapshots alone do not guarantee application-consistent content unless quiescing and filesystem behavior are addressed.

Protect the entire required home, including credential encryption material, while restricting who can read backups. Prefer configuration-as-code and versioned job definitions to reduce the amount of irreplaceable manual configuration. Plugins need a reviewed version inventory; restoring a home with incompatible runtime or plugins can fail even when files exist.

Rehearse restoring an encrypted snapshot to a new EBS volume, attaching it to a replacement host, mounting its existing filesystem, and starting the approved Jenkins image. Confirm authentication, one test job, artifact access, and credential retrieval without executing production deployments.

Cross-AZ recovery uses a snapshot restored into the destination AZ, not direct cross-AZ attachment of the original volume. Cross-region copies and KMS policies introduce additional dependencies. Document which recovery identity can decrypt the destination volume and obtain image artifacts when the primary environment is unavailable.

Measure RTO from interruption to a validated pipeline. Measure RPO from the newest recoverable controller changes, not the snapshot creation request timestamp alone. Preserve the bootstrap procedure outside Jenkins; restoring Jenkins must never require Jenkins to run first.

Retention Is a Cloud Resource Setting

Verify the EBS attachment's deletion-on-termination setting through the infrastructure definition and actual EC2 attachment. Naming a disk persistent does not change that flag. Separate the volume resource's lifecycle from the disposable instance and record its AZ, encryption key, size, and backup policy.

Plan what happens if the instance disappears during a filesystem write. Retained storage may need filesystem recovery and Jenkins consistency checks; do not confuse retention with guaranteed clean shutdown. Review disk utilization and inode exhaustion because a full home can break jobs, plugin updates, and backup operations.

Monitor snapshot completion and restoration capability. A snapshot request that never completes does not meet the controller's RPO. Keep enough versions to recover from delayed discovery of corruption, and restrict snapshot-sharing permissions.

For disk expansion, increase the cloud volume and extend the filesystem through a tested distribution-specific procedure. Confirm the mounted device and current size first. Expansion should not require replacing Jenkins home or recursively changing ownership on unrelated directories.

Finally, record the image digest and plugin inventory used with each recovery point. Data persistence without runtime compatibility can leave the team with intact files and an unusable controller.

A restore drill should also verify filesystem ownership after attachment; matching the volume ID does not guarantee Jenkins can read its home.

Key Takeaways

- **A restart policy is not persistence or disaster recovery.**
- **Retain encrypted controller storage separately from disposable compute.**
- **Identify a blank disk before formatting; never format restored data.**
- **Require the real mount before starting Jenkins.**
- **Keep controller access private and build execution isolated.**
- **Test container, host, and AZ recovery as different failure scenarios.**

References

- Jenkins Docker installation: Documentation
- EBS volume attachment: Documentation

Chapter 9: Secure the Build Factory: Separate Controllers, Agents, and Credentials

A Dependency Script Finds the Deployment Keys

A package's installation script runs during a Jenkins build. It reads the controller's home directory and retrieves an AWS credential with administrative permissions. The attacker creates a new cloud identity, disables selected logging, and copies database snapshots.

The job ran on the built-in node because it was convenient. That node combined orchestration, credential storage, source execution, and cloud administration. A package installation became a control-plane compromise.

Treat build code as untrusted execution, even when it came from your own repository. Review can reduce risk, but dependency scripts, build tools, and contributed pull requests still need constrained authority.

Controller and Agent Are Different Trust Domains

The controller schedules work, stores configuration, and coordinates plugins. Agents execute job steps. Set built-in executors to zero so jobs cannot casually run beside controller secrets.

Agent isolation is not achieved merely by adding a label. A build that can mount the controller's home, access its Docker daemon, or reach an unrestricted administrative endpoint can still cross the boundary. Give agents separate hosts or appropriately hardened ephemeral execution environments.

Separate untrusted verification from trusted release. Pull-request jobs test code without production credentials or publishing rights. Release jobs execute only approved revisions with narrowly scoped identities. Do not let an arbitrary repository Jenkinsfile request a privileged label and receive a deployment role.

Use folder authorization, node-use restrictions supported by reviewed plugins, and trusted pipeline definitions to enforce those decisions. Labels express scheduling intent; authorization determines who may use the capacity.

Configuration as Code Must Be Versioned

JCasC makes controller configuration reproducible. It is not a substitute for plugin version management, credential storage, or authorization review. Pin plugin versions through the supported image-building workflow and test configuration compatibility before controller upgrades.

The following example requires JCasC, the matrix-authorization strategy plugin, and core inbound-agent support. It creates a permanently defined logical agent; the actual process must run on separate infrastructure. An ephemeral provisioning plugin can replace this approach when lifecycle ownership is established.

jenkins.yaml

```yaml
jenkins:
  systemMessage: "Orders CI - approved revisions only"
  numExecutors: 0 # The controller never executes builds.
  mode: EXCLUSIVE
  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: "bootstrap-admin"
          password: "${BOOTSTRAP_ADMIN_PASSWORD}"
  authorizationStrategy:
    globalMatrix:
      entries:
        - user:
            name: "bootstrap-admin"
            permissions:
              - "Overall/Administer"
  nodes:
    - permanent:
        name: "isolated-verification"
        remoteFS: "/home/jenkins/agent"
        numExecutors: 1
        mode: EXCLUSIVE
        labelString: "untrusted-verification"
        launcher:
          inbound:
            workDirSettings:
              disabled: false
              internalDir: "remoting"
              failIfWorkDirIsMissing: true
  remotingSecurity:
    enabled: true
```

This is a bootstrap configuration, not the final enterprise access model. Supply the administrator password through a protected secret source, never a committed `.env` file. Replace local authentication with your reviewed identity-provider integration, MFA policy, and group-to-role mapping before normal use.

JCasC environment substitution does not make secrets invisible to administrators or process inspection. Avoid export endpoints and logs that disclose interpolated configuration. Test recovery with the secret source available independently of the controller.

Run the inbound agent with the approved agent image and a node-specific connection secret. Use WebSocket over the controller's TLS endpoint where appropriate. Keep the secret out of command history, rotate it after suspected exposure, and avoid sharing one agent identity across unrelated machines.

The controller-to-agent connection carries work but does not grant a secure sandbox automatically. Restrict access to controller ports and monitor anomalous remoting behavior. The agent's workspace is untrusted data; never copy it into controller configuration as an administrative action.

Short-Lived Credentials Still Need Separation

An EC2 agent can receive an instance-profile role. Every build on that agent may be able to reach its metadata credentials, so the instance role must reflect the least-trusted workload using that host. IMDSv2 improves metadata access controls but is not a magical barrier against code executing on the instance.

Use an untrusted agent role without deployment rights. Place trusted releases on separately authorized capacity. A release-agent base role may assume a narrowly scoped deployment role; ordinary verification agents must not have that permission.

agent-roles.tf

```
variable "account_id" { type = string }
resource "aws_iam_role" "release_agent" {
  name = "orders-release-agent"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
      Action = "sts:AssumeRole"
    }]
  })
}
resource "aws_iam_instance_profile" "release_agent" {
  role = aws_iam_role.release_agent.name
}
resource "aws_iam_role" "deployment" {
  name                 = "orders-production-deployment"
  max_session_duration = 3600
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { AWS = aws_iam_role.release_agent.arn }
      Action = "sts:AssumeRole"
    }]
  })
}
resource "aws_iam_role_policy" "assume_deployment" {
  role = aws_iam_role.release_agent.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"
      Action   = "sts:AssumeRole"
      Resource = aws_iam_role.deployment.arn
    }]
  })
}
```

These declarations define trust, not useful deployment permissions. Attach a separate reviewed permission policy to `deployment` for the intended ECR, EKS, or Terraform operations. Do not attach `AdministratorAccess` to complete the example quickly.

Also restrict who can pass the instance role and attach the instance profile. An attacker who can launch an instance with the release-agent role may inherit its deployment path. `iam:PassRole` is part of the trust model, not an incidental provisioning permission.

A CI OIDC workflow requires an actual trusted issuer and constrained subject/audience claims. Jenkins does not automatically issue an AWS-trusted OIDC token merely because a Jenkinsfile requests one. Use an established federation implementation or EC2 role sessions; do not invent a token exchange.

Credentials Belong to Stages and Identities

Request credentials only in the stage that needs them. Use short session durations and identifiable role-session names. Avoid `set -x`, environment dumps, and shared workspaces containing credential files.

A pipeline's secret masking is a display feature, not a prevention boundary. Malicious code can encode a credential, send it over the network, or use it without printing it. Prevent exposure by keeping secrets away from untrusted execution.

Separate registry publishing, infrastructure apply, and application deployment where their permissions differ. A builder needs permission to push an image, not to delete production data. An application deployer may need a constrained Kubernetes identity, not Terraform backend access.

State and Recovery Controls

Back up controller configuration and required credential-encryption material under separate access control. Version pipeline definitions and JCasC, preserve the reviewed plugin inventory, and test restoration into an isolated controller.

Do not restore a compromised controller and immediately resume production jobs. Revoke suspicious sessions, rotate affected secrets, inspect identity changes, and establish a clean trusted configuration source. Recovery must address attacker persistence as well as uptime.

Ephemeral agents should discard workspaces between trust domains. Clear caches deliberately: dependency and build caches can contain modified artifacts or secret material. Use trusted cache ownership and verification instead of sharing an unrestricted writable cache across all jobs.

Record agent image digest, plugin versions, job revision, identity role, and session timestamps as build evidence. Audit who changes labels, node definitions, credential bindings, and pipeline source references.

Prove the Boundary

Create a synthetic pull-request job that attempts to discover credentials and contact protected services. It must fail without retrieving real secrets. Confirm built-in executors remain zero and the job cannot select release capacity through a modified Jenkinsfile.

On a trusted test agent, obtain a short-lived role session and verify only the intended operation succeeds. Test a denied production action in a sandbox with explicit authorization. Review CloudTrail for the expected role-session identity.

Stop the controller and confirm existing applications continue running. Destroy a disposable agent and verify retry behavior does not duplicate releases. Disconnect the secret source during a recovery rehearsal and confirm the runbook reports the dependency clearly rather than substituting static keys.

A secure build factory has explainable trust transitions. If a reviewer cannot identify which identity executes each stage and what it can change, the pipeline is not ready for production.

Limit Agent Network Authority

An isolated agent should reach only the source, dependency, artifact, and control services its job needs. Unrestricted outbound access makes exfiltration easy even when logs mask credentials. Use approved proxies, destination policies, or private endpoints where the environment supports them.

Avoid sharing one host between trusted and untrusted jobs merely because they execute at different times. Container cleanup may leave daemon state, writable caches, or host modifications. Recreate agents from a reviewed image when crossing trust domains.

Test whether the verification role can read Terraform state or assume the release role. Both should fail. Then verify the trusted release role can perform only its documented publishing or deployment action. Record effective policies, including inherited grants; a narrowly written policy does not cancel another broad allow.

This turns agent isolation into an observed property of filesystem, network, and identity access rather than a label attached to a Jenkins node.

Review inherited credential bindings whenever a job moves folders; organizational convenience must not silently expand the new job's effective authority.

Key Takeaways

- **Keep build execution off the controller.**
- **Labels schedule work; authorization enforces trust boundaries.**
- **Separate untrusted verification from credential-bearing releases.**
- **Short-lived credentials reduce exposure, not excessive privilege.**
- **Version JCasC and plugins, and protect recovery secrets.**
- **Test agent isolation and role assumption explicitly.**

References

- Jenkins controller isolation: Documentation
- JCasC: Documentation

Chapter 10: From Commit to Trusted Image: Write the Jenkinsfile

Latest Was Not the Release

A pipeline pushes an untested image as `orders-api:latest`. Production restarts a pod and pulls that tag. Some replicas run yesterday's image, others run the new one. The deployment record still says `latest`, so nobody can identify the actual code consistently.

The team rolls back the deployment YAML. It still points at `latest`. The new image is fetched again, and the incident continues.

A tag is a name that can change. A registry digest identifies content. **Build evidence must name the exact image that passed the checks and the exact image deployed.** Git-commit tags help humans navigate, but immutable registry policy and digest promotion establish stronger identity.

Define the Pipeline's Contract

This pipeline executes on a trusted isolated release agent, not the controller. Untrusted pull requests use a separate verification pipeline without registry publishing credentials. Jenkins authorization must prevent arbitrary code from choosing this privileged execution path.

The agent needs a reviewed Docker builder, Python, AWS CLI, Trivy, and the Jenkins Pipeline plugins used below. Pin the toolchain in the agent image. Docker-daemon access is privileged; use dedicated agents and keep their trust boundary away from other tenants.

The repository includes an application test suite and the Chapter 6 Dockerfile. The test suite must contain meaningful tests, not just successful discovery of zero tests. The pipeline checks syntax and executes unit tests before image construction, then performs a runtime health test.

ECR must have an immutable-tag policy for this repository. Its lifecycle policy must retain images referenced by production and recovery records. The assumed agent role may authenticate to ECR and publish only to the approved repository; it must not delete production state.

A Complete Declarative Pipeline

The following values are examples to replace in reviewed pipeline configuration. `PYTHON_BASE` must contain a verified digest. It is not a user-supplied parameter that an untrusted job can alter.

Jenkinsfile

```
pipeline {
  agent { label 'trusted-image-builder' }
  options {
    skipDefaultCheckout(true)
    disableConcurrentBuilds()
    timestamps()
    timeout(time: 30, unit: 'MINUTES')
  }
  environment {
    AWS_REGION = 'ap-south-1'
    AWS_ACCOUNT_ID = '123456789012'
    ECR_REPOSITORY = 'orders-api'
    PYTHON_BASE = 'python:REVIEWED_TAG@sha256:REVIEWED_DIGEST'
  }
  stages {
    stage('Checkout') {
      steps {
        deleteDir() // Do not inherit an unrelated workspace.
        checkout scm
        script {
          env.REVISION = sh(
            script: 'git rev-parse HEAD',
            returnStdout: true
          ).trim()
          if (!(env.REVISION ==~ /[0-9a-f]{40}/)) {
            error('Unexpected Git revision')
          }
          env.REGISTRY = "${env.AWS_ACCOUNT_ID}.dkr.ecr.${env.AWS_REGION}.amazonaws.com"
          env.IMAGE = "${env.REGISTRY}/${env.ECR_REPOSITORY}:${env.REVISION}"
        }
      }
    }
    stage('Test source') {
      steps {
        sh '''#!/usr/bin/env bash
set -euo pipefail
python3 -m compileall -q app.py tests
python3 - <<'PY'
import unittest
suite = unittest.defaultTestLoader.discover('tests')
if suite.countTestCases() == 0:
    raise SystemExit('Refusing a zero-test release')
result = unittest.TextTestRunner(verbosity=2).run(suite)
raise SystemExit(not result.wasSuccessful())
PY
'''
      }
    }
    stage('Build and runtime test') {
      steps {
        sh '''#!/usr/bin/env bash
set -euo pipefail
docker build --pull --build-arg PYTHON_BASE="$PYTHON_BASE" -t "$IMAGE" .
cid=$(docker run -d --read-only --cap-drop ALL --security-opt no-new-privileges:true "$IMAGE")
trap 'docker rm -f "$cid" >/dev/null' EXIT
# Test inside the container without exposing an ephemeral host listener.
for attempt in {1..20}; do
  if docker exec "$cid" python -c "import urllib.request;
 ↪  urllib.request.urlopen('http://127.0.0.1:8080/readyz', timeout=2)"; then
```

```
    exit 0
  fi
  sleep 1
done
docker logs "$cid"
exit 1
'''
      }
    }
    stage('Scan and SBOM') {
      steps {
        sh '''#!/usr/bin/env bash
set -euo pipefail
mkdir -p evidence
# Scanner failure and policy failure both fail the build.
trivy image --exit-code 1 --severity HIGH,CRITICAL \
  --format json --output evidence/vulnerabilities.json "$IMAGE"
trivy image --format cyclonedx \
  --output evidence/sbom.json "$IMAGE"
'''
      }
    }
    stage('Publish approved artifact') {
      steps {
        sh '''#!/usr/bin/env bash
set -euo pipefail
actual=$(aws sts get-caller-identity --query Account --output text)
[ "$actual" = "$AWS_ACCOUNT_ID" ]
# Keep registry tokens out of the agent's shared home directory.
export DOCKER_CONFIG
DOCKER_CONFIG=$(mktemp -d)
trap 'rm -rf "$DOCKER_CONFIG"' EXIT
aws ecr get-login-password --region "$AWS_REGION" \
  | docker login --username AWS --password-stdin "$REGISTRY"
docker push "$IMAGE"
# Resolve registry identity, rather than treating the tag as the release.
aws ecr describe-images --region "$AWS_REGION" \
  --repository-name "$ECR_REPOSITORY" \
  --image-ids imageTag="$REVISION" \
  --query 'imageDetails[0].imageDigest' --output text \
  > evidence/image-digest.txt
grep -Eq '^sha256:[0-9a-f]{64}$' evidence/image-digest.txt
printf '%s\n' "$REVISION" > evidence/source-commit.txt
printf '%s\n' "$PYTHON_BASE" > evidence/base-image.txt
'''
      }
    }
  }
  post {
    always {
      archiveArtifacts artifacts: 'evidence/*',
        allowEmptyArchive: true, fingerprint: true
      // Agent destruction, not deleteDir alone, ends its trust lifecycle.
      deleteDir()
    }
  }
}
```

The build refuses to publish if testing or scanning fails. `allowEmptyArchive` preserves a failure's available evidence without pretending a missing scan report means success. Configure artifact

retention and access independently; reports can expose dependency and infrastructure details.

Understand the Scanner Gate

Severity is a policy input, not a complete risk model. A critical finding in an unreachable component and a moderate finding in an exposed authentication path need different analysis. Start with a conservative gate and define a reviewed exception mechanism with owner, scope, expiry, and compensating controls.

Do not add `|| true` when scanner infrastructure is unavailable. That transforms an unavailable security check into permission to release. Cache vulnerability databases under trusted ownership and monitor freshness. Record scanner version and database revision alongside results.

An SBOM describes components; it does not prove authenticity. Add provenance and signing through a reviewed signing identity, then verify signatures and policy at deployment. Signing an untested image merely authenticates a bad release.

Publication Is Not Deployment

The pipeline publishes a candidate artifact. Environment promotion consumes its recorded digest and approval evidence without rebuilding. A release record should include repository, digest, source revision, test results, configuration revision, and schema compatibility.

If an ECR immutable tag already exists for a retried commit, do not delete it automatically. Determine whether the recorded artifact already passed the required checks. A retry can produce a different image from changing external inputs even when source is unchanged.

Prevent registry cleanup from deleting deployed digests. Keep a separate retention policy for recoverable releases, and verify a replacement cluster can still pull the previous approved artifact. Registry availability and authentication are part of recovery, not just build convenience.

Multi-platform images introduce a manifest-list digest plus platform-specific digests. Record the top-level identity and prove each supported architecture meets the test contract. The single-platform pipeline here does not claim multi-platform verification.

Boundaries and Failure Handling

A failed build must leave production unchanged. A failed push can leave partial registry upload state but must not create an approved deployment record. A scan report from a previous workspace must never be reused as current evidence; clean checkout and isolated agents prevent that confusion.

Cloud credentials should be short-lived and scoped. Publishing requires ECR authorization plus upload operations on the intended repository. Application deploy permissions and Terraform backend access belong to different identities and stages.

Jenkins pipeline definitions are code with administrative consequences. Protect the source revision used by this trusted pipeline, review shared-library updates, and restrict who can edit the job’s branch reference. The most secure shell command is irrelevant if a contributor can replace the whole pipeline with credential exfiltration.

Exercise failures deliberately: make a unit test fail, inject a synthetic scanner policy failure, and deny publishing to a sandbox repository. Confirm each prevents publication and preserves diagnostic evidence. Never introduce a real exploitable package into production to test the gate.

Measure build duration, scan freshness, agent queue time, and retry causes. Excessive flakiness drives engineers to bypass controls. Fix unstable dependencies and execution environments instead of weakening the definition of a trusted image.

Bind the Published Digest to the Tested Image

A registry query after push identifies the stored artifact, but the evidence chain must also show that the tested local image produced that publication. Record the local image identifier and build metadata before testing, keep the agent isolated, and prevent another job from retagging the image during the pipeline.

After publication, pull the recorded digest on a clean verification environment and repeat a bounded smoke test. This detects registry-reference mistakes and missing runtime content without rebuilding. Preserve the distinction between verifying a copied artifact and constructing a new one.

Use an explicit platform when architecture matters. A manifest-list publication can include multiple images, so testing one architecture does not prove the entire list meets the contract. Record supported platforms and their verification evidence.

If signing is added, sign the resolved digest rather than a mutable tag. Restrict the signing identity and require deployment-side verification against the approved issuer or key. Keep the private key or token out of the repository and build context.

These controls close a practical gap: a good test report is useful only if the deployed bytes are the bytes that generated it.

Retain the registry verification result with the build record so recovery operators can retrieve the artifact without relying on mutable branch names.

Key Takeaways

- **Tags are labels; digests identify the promoted artifact.**
- **Testing, runtime checks, scanning, and publication have separate gates.**
- **Zero tests and unavailable scanners must not silently pass.**
- **Keep publishing credentials off untrusted pull-request agents.**
- **Preserve evidence and retain deployed image digests.**
- **Provenance and signing complement testing; neither replaces it.**

References

- Jenkins declarative pipeline: Documentation
- ECR tag immutability: Documentation
- Trivy: Documentation

Chapter 11: Terraform Through CI: Approve the Change You Actually Apply

Approval Covered a Different Plan

The engineer reviewed a local plan showing one security-group change. Before CI ran, another administrator modified the database configuration. The pipeline generated a fresh plan and applied it automatically, including a database replacement nobody approved.

The dangerous action came from executing a newly generated plan, not from Terraform magically changing an approved saved plan. A saved plan whose state has become stale can be rejected. External changes outside state still require coordination and reconciliation.

An approval must identify the exact execution artifact, source revision, environment, and identity. Infrastructure approval is not a generic permission to run the latest branch.

Plan Files Are Sensitive Execution Artifacts

`terraform plan -out=tfplan` creates a binary plan that apply can consume. It includes configuration and values required for execution, potentially including sensitive material. Human-readable masking does not turn the binary into a public artifact.

Store the plan under restricted access and retention. Avoid a public pull-request attachment containing plan JSON or state. `terraform show -json` can expose values in machine-readable form; use protected policy checks rather than publishing the entire output.

Pair the plan with the source commit, provider lock file, CLI version, backend key, account identity, timestamp, and a cryptographic hash. Require approval after those details are established. Apply only that plan in the matching execution context.

Separate Review from Authority

A planning role normally needs read access to resources and state plus permissions for backend lock operations. An applying role needs the exact write operations the root manages. Separate credentials where practical, but test that both roles can initialize the same backend correctly.

A plan approval does not replace IAM. The apply identity should be unable to modify unrelated accounts, read other state keys, or delete protected backups. Terraform can only act within effective cloud permissions, so an overly broad root and role remain dangerous even with a manual button.

Backend locking coordinates state operations. Pipeline serialization coordinates this pipeline. Neither prevents a human console change or a different pipeline using another state key. Establish an environment write policy and a drift-response procedure.

A Plan, Approve, Apply Pipeline

The example requires a trusted isolated agent with pinned Terraform and a scoped instance role. It plans and applies using one authorized lab identity for simplicity; production can split stages onto distinct protected agents with reviewed credential transitions.

Jenkins authorization must restrict job edits, source references, and the `platform-approvers` group. Adapt the submitter to your configured security realm. A string in a Jenkinsfile does not create that group.

Jenkinsfile

```
pipeline {
  agent { label 'trusted-terraform' }
  options {
    skipDefaultCheckout(true)
    disableConcurrentBuilds()
    timestamps()
    timeout(time: 60, unit: 'MINUTES')
  }
  environment {
    TF_IN_AUTOMATION = 'true'
    TF_INPUT = 'false'
    AWS_ACCOUNT_ID = '123456789012'
    TF_ROOT = 'infra/environments/prod/aws/orders'
  }
  stages {
    stage('Checkout and identify') {
      steps {
        deleteDir()
        checkout scm
        sh '''#!/usr/bin/env bash
set -euo pipefail
[ "$(aws sts get-caller-identity --query Account --output text)" = "$AWS_ACCOUNT_ID" ]
git rev-parse HEAD > revision.txt
terraform version
'''
      }
    }
    stage('Plan') {
      steps {
        dir("${env.TF_ROOT}") {
          sh '''#!/usr/bin/env bash
set -euo pipefail
umask 077
terraform init -lockfile=readonly
terraform fmt -check -recursive
terraform validate
terraform plan -lock-timeout=120s -out=tfplan
terraform show -no-color tfplan > plan-review.txt
sha256sum tfplan > tfplan.sha256
'''
          // Restrict job/artifact readers; these are sensitive artifacts.
          archiveArtifacts artifacts: 'tfplan,tfplan.sha256,plan-review.txt',
```

```
            fingerprint: true
        }
      }
    }
    stage('Approve exact plan') {
      steps {
        timeout(time: 30, unit: 'MINUTES') {
          input message: 'Approve this build\'s protected plan and environment?',
            submitter: 'platform-approvers'
        }
      }
    }
    stage('Apply approved plan') {
      steps {
        dir("${env.TF_ROOT}") {
          sh '''#!/usr/bin/env bash
set -euo pipefail
[ "$(aws sts get-caller-identity --query Account --output text)" = "$AWS_ACCOUNT_ID" ]
sha256sum --check tfplan.sha256
# No fresh planning command and no -auto-approve are needed here.
terraform apply -lock-timeout=120s tfplan
'''
        }
      }
    }
    stage('Verify convergence') {
      steps {
        dir("${env.TF_ROOT}") {
          sh '''#!/usr/bin/env bash
set -euo pipefail
# This stage fails if further changes remain; investigate partial convergence.
terraform plan -lock-timeout=120s -detailed-exitcode
'''
        }
      }
    }
  }
  post {
    always { deleteDir() }
  }
}
```

A checksum stored beside the plan detects accidental modification, not an attacker who can replace both files. Protect the workspace and artifact service, and bind approval to an authenticated build record. Stronger signing or immutable artifact storage can extend that trust chain.

Holding one agent during approval is simple but consumes capacity. A production workflow may archive the plan and release the agent, then retrieve it on a separate applying agent. That requires authenticated artifact retrieval, matching source and lock file, compatible CLI, and exact backend configuration. Do not casually reconstruct inputs from a moving branch.

Handle Staleness Explicitly

Approval expires when its reviewed assumptions no longer hold. Define a maximum plan age and reject an approval after relevant configuration, state, or administrative changes. A fixed thirty-minute approval timeout is only one part of that policy.

If apply reports a stale plan, stop. Generate a new plan and obtain new approval. Do not regen-

erate and auto-apply because the reviewer “already approved the intent.” Intent and concrete resource effects are different review objects.

An external console change may not alter Terraform state until refreshed. Coordinate writers and use post-apply verification. Saved plans are not a guarantee that remote services remain unchanged between review and execution.

Plan policy should flag deletions, replacements, public access, IAM privilege expansion, encryption changes, and movement across network boundaries. For destructive database changes, require a separate recovery checkpoint and database-owner approval. Do not rely on reviewers visually scanning thousands of unchanged resources.

Partial Apply Is an Incident Condition

Terraform is not a transaction that rolls back every AWS operation when one resource fails. Some resources may be created or updated before another operation errors. The state can record successful operations while the overall apply reports failure.

Preserve logs and inspect the authoritative state. Determine which actions completed, identify the failed dependency, and re-plan after correcting the cause. Do not blindly rerun the original saved plan or restore an older state snapshot to hide partial progress.

Application rollback and infrastructure repair are separate decisions. If a network change broke traffic, a safe corrective plan may restore connectivity. If a database replacement deleted data, a configuration rollback does not reconstruct the missing records; the tested recovery procedure must do that.

Protect the Approval System

Jenkins job configuration, shared libraries, credentials, and agent images are part of the execution trust chain. A contributor who can edit the production pipeline or replace its trusted toolchain may bypass the approval gate entirely.

Keep untrusted PR planning separate. Public plans can expose resource names, network ranges, and secrets; even read-only planning can execute data sources or interact with external tools. Review providers and external execution paths before granting planning credentials to contributed code.

Retain the approved plan and execution evidence according to a defined policy. Do not keep sensitive artifacts indefinitely just because storage is cheap. Log the approver, build ID, source revision, target account, plan hash, apply result, and functional verification result.

Test the Process

Use a disposable root to demonstrate a waiting lock, an expired approval, a rejected stale plan, and a failed IAM write. Confirm none causes an automatic transition to a new unapproved apply.

Modify a synthetic plan artifact in a test workspace and verify the checksum gate fails. Test that an unauthorized Jenkins user cannot approve the production stage or edit its source reference. Check that secrets do not appear in archived review text or shared logs.

Finally, verify the workload after infrastructure changes: DNS, connection paths, health, identity, and representative transactions. Terraform convergence is an infrastructure observation. The business service still needs a functional acceptance check.

Plan Review Must Explain Replacements

Terraform marks some attribute changes as requiring replacement. The reviewer should understand whether that is a provider limitation, an immutable service property, or a resource-address change introduced by configuration. The word replacement is a recovery question, especially for databases and storage.

For each replacement, record the affected service, expected interruption, preserved data, reconnect behavior, and rollback boundary. Check whether create-before-destroy is feasible: uniqueness constraints, address capacity, quotas, or dependent resources can make it impossible.

Do not add lifecycle settings as a generic promise of zero downtime. They change graph behavior but do not establish application compatibility or spare capacity. Verify the replacement sequence in a disposable representative environment.

The approval record should explain why destructive actions are acceptable and which recovery evidence supports that decision. A reviewer who cannot answer those questions should reject the plan rather than approve it because the number of changed resources is small.

Require the approver to identify the protected build record explicitly; approval in a separate chat message is not the execution gate.

Key Takeaways

- **Approve a concrete saved plan, not a moving branch or general intent.**
- **Protect plans as sensitive artifacts.**
- **Re-plan and re-approve when assumptions become stale.**
- **Serialize execution and coordinate every other writer.**
- **Partial apply requires investigation, not assumed rollback.**
- **Verify application behavior after infrastructure convergence.**

References

- Terraform plan: Documentation
- Terraform apply: Documentation

Chapter 12: Kubernetes Beyond “It Runs”: Orchestrate the Application

The Pods That Never Came Back

The team deploys three standalone Pods and calls the application highly available. A node fails, and the Pods assigned to it stop serving. There is no Deployment to create replacements on healthy capacity.

A container restart policy can restart a process within a surviving Pod environment; it does not give a standalone Pod a replacement controller after node loss. The team confuses process recovery with workload reconciliation.

Declare the workload you want maintained, not just a process you want started. Kubernetes needs a controller, a service contract, and schedulable capacity before it can replace failed application instances.

Deployments Own Replica Intent

A Pod is a scheduling unit containing one or more cooperating containers. A Deployment manages ReplicaSets, which maintain the requested Pod count. Deployment changes create a controlled rollout rather than requiring manual Pod replacement.

A Service provides stable discovery and forwards traffic to matching endpoints. Its selector must match the intended Pod labels. A working Pod with the wrong label is invisible to the Service; a wrong broad selector can send traffic to an unrelated workload.

Namespaces organize API objects and policy scope. They are not complete tenant isolation by themselves. RBAC, NetworkPolicy enforcement, admission controls, workload identity, and resource quotas establish additional boundaries.

Use a local disposable cluster before paying for managed capacity. Pin its Kubernetes and node-image versions through your chosen local tool. This chapter’s manifests use stable APIs; confirm compatibility with the actual cluster and admission policies.

The Deployment Contract

Use the Chapter 6 image published by Chapter 10. Replace the image placeholder with its verified registry digest. The manifest deliberately does not embed a database password or make the API publicly accessible.

namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: orders
  labels:
    pod-security.kubernetes.io/enforce: restricted
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: orders-api
  namespace: orders
automountServiceAccountToken: false
```

Restricted Pod Security checks container security settings, not application authentication or all network behavior. Pin admission policy versions when managing cluster upgrades so enforcement changes are reviewed rather than accidental.

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-api
  namespace: orders
spec:
  replicas: 2
  revisionHistoryLimit: 3
  progressDeadlineSeconds: 120
  selector:
    matchLabels:
      app: orders-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: orders-api
    spec:
      serviceAccountName: orders-api
      automountServiceAccountToken: false
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
        fsGroup: 10001
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: api
          image: REGISTRY/orders-api@sha256:REPLACE_DIGEST
          ports:
            - name: http
```

```
            containerPort: 8080
          env:
            - name: PORT
              value: "8080"
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 128Mi
          startupProbe:
            httpGet:
              path: /healthz
              port: http
            periodSeconds: 2
            failureThreshold: 30
          readinessProbe:
            httpGet:
              path: /readyz
              port: http
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /healthz
              port: http
            periodSeconds: 10
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: [ALL]
          volumeMounts:
            - name: temporary
              mountPath: /tmp
      volumes:
        - name: temporary
          emptyDir:
            sizeLimit: 16Mi
```

A digest placeholder is not deployable configuration. Replace it deliberately and validate the resulting manifest. The fixture has no database dependency; a real order API must define how readiness behaves when its transaction path is unavailable.

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: orders-api
  namespace: orders
spec:
  type: ClusterIP # Keep exposure inside the cluster for this lab.
  selector:
    app: orders-api
  ports:
    - name: http
      port: 80
```

```
    targetPort: http
```

A ClusterIP Service is not an internet endpoint. Public routing requires an intentionally configured ingress or gateway, TLS, DNS, and authentication. Do not switch to NodePort to bypass an unresolved exposure design.

Probes Have Different Jobs

Startup probes protect a slow initialization period from premature liveness failure. Readiness controls whether an endpoint should receive Service traffic. Liveness decides whether the process should restart.

Do not make liveness depend on every external service. A temporary database outage could restart every API replica, add connection storms, and erase useful diagnostic context. Readiness also needs care: removing every endpoint during a recoverable dependency incident may change client behavior and amplify retries.

Set probe timeouts and thresholds from measured startup and response behavior. A default that happened to work on a laptop is not a capacity-tested contract. Monitor probe failures separately from application request failures.

Requests Are Scheduling Inputs

CPU and memory requests tell the scheduler how much capacity the workload needs. Limits constrain execution behavior: memory exhaustion can trigger an OOM kill, and CPU limits can throttle work. These values are starting measurements, not a universal production recommendation.

A rollout with `maxSurge: 1` needs spare schedulable capacity for an additional Pod. If the cluster has no headroom, the new Pod may remain Pending while old replicas continue running. Check scheduling events rather than assuming the deployment is stuck mysteriously.

Two replicas on the same node do not provide node-failure resilience. Add topology-spread or anti-affinity rules and enough nodes to satisfy them. Hard constraints can also block scheduling, so design the cluster capacity and placement rules together.

Apply and Inspect

```
kubectl config current-context
kubectl apply --dry-run=server -f namespace.yaml
kubectl apply -f namespace.yaml
kubectl apply --dry-run=server -f deployment.yaml -f service.yaml
kubectl apply -f deployment.yaml -f service.yaml
kubectl -n orders rollout status deployment/orders-api --timeout=120s
kubectl -n orders get pods -o wide
kubectl -n orders get endpointslices
```

Server-side dry-run needs the Namespace to exist before validating namespaced objects against it. It validates API and admission behavior, not image availability or functional readiness. Confirm the image can be pulled with the configured registry authentication.

Use a temporary local port forward for a controlled test:

```
kubectl -n orders port-forward service/orders-api 8080:80
# In another terminal:
curl --fail http://127.0.0.1:8080/readyz
```

Port forwarding demonstrates access through an administrative tunnel, not public routing, load balancing under real traffic, or firewall enforcement. Test the intended user path separately when adding ingress.

Delete one synthetic API Pod and verify the ReplicaSet creates a replacement. Then test a node failure in a disposable multi-node cluster. Compare the recovery delay with your availability objective and record unschedulable conditions.

Termination Is Part of Correctness

Kubernetes sends termination signals and eventually forces process exit after the grace period. The application must stop taking new work and handle unfinished operations safely. Endpoint updates and external load balancers introduce propagation delays; graceful shutdown must account for real routing behavior.

Workers need durable claims, bounded lease behavior, and idempotent side effects. A replacement worker may encounter work interrupted by a forced termination. A replica count alone does not prevent duplicate execution or lost acknowledgement.

Secrets should come from a reviewed secret-management integration or protected Kubernetes Secrets. Base64 encoding is not encryption. Restrict read permissions, configure encryption at rest where required, and avoid granting the API broad access to the Kubernetes API.

Configuration changes need explicit rollout behavior. Updating a ConfigMap does not universally restart a Deployment or make every process reread its values. Record image and configuration revisions together.

State and Ownership

Keep workload manifests in Git, cluster runtime state under Kubernetes control, and cloud resource ownership under Terraform. Do not have Terraform and a separate deployment system fight over the same field without a documented ownership contract.

A namespace deletion can remove namespaced workload objects; it is not a safe generic rollback. Persistent data and cloud resources may survive or be deleted depending on their policies. Review cleanup across API, volume, and cloud boundaries.

The acceptance result is a maintained workload with tested service discovery, readiness, replacement, and shutdown. “The Pod is Running” is only a process observation.

Troubleshoot from Ownership to Traffic

When an API is unavailable, start with the Deployment’s desired and available replica counts. Inspect rollout conditions and Pod events before looking at application logs. An unschedulable Pod, failed image pull, admission rejection, and crashing process need different fixes.

Next inspect readiness and Service selectors. Confirm EndpointSlices contain the expected ready addresses. A Service with no endpoints can be a label problem even when every container runs normally. A Service with endpoints can still fail because network policy or destination ports are wrong.

Then test the actual application path with an approved diagnostic identity. Keep administrative port forwarding separate from the user traffic path. It can help isolate process behavior but can bypass parts of the network architecture being investigated.

Finally inspect configuration and dependencies. Compare image digest and configuration revision against the release record. Avoid editing a running Pod to repair configuration: the next replacement will erase that change and restore the declared mistake.

Capture evidence before deleting everything. A restart can remove useful timing and crash context without fixing the underlying dependency. The controller will reproduce the same failure until its desired configuration or operating environment becomes correct.

A good runbook follows these boundaries in order and names the owner responsible for each one.

Key Takeaways

- **Use controllers for workloads that must be replaced.**
- **Services discover matching ready endpoints; labels are part of the contract.**
- **Probes, scheduling requests, and limits solve different problems.**
- **Replica count needs placement and capacity to provide resilience.**
- **Validate termination, identity, and configuration behavior.**
- **Keep cloud and cluster ownership boundaries explicit.**

References

- Deployments: Documentation
- Pod lifecycle: Documentation

Chapter 13: AWS Kubernetes: Build an EKS Platform with Terraform

The Cluster Only Its Creator Understood

The engineer who created EKS through the console leaves the company. Months later, nodes are full and a control-plane upgrade is required. Nobody knows which IAM roles, subnet tags, add-ons, or access entries were changed manually.

The platform team can see the cluster but cannot reliably reproduce or administer it. A managed control plane reduced server maintenance; it did not remove ownership decisions.

EKS manages selected control-plane responsibilities. You still own identity, worker capacity, networking, upgrades, application state, and recovery. Record those decisions in code and test them before the original administrator becomes a dependency.

Split Cloud Provisioning from Workload Deployment

Terraform owns the EKS cluster, node groups, IAM relationships, and surrounding AWS resources. Kubernetes manifests own application workloads. A separate platform configuration layer owns reviewed add-ons and cluster policies.

Avoid a single apply that creates a private cluster and immediately tries to deploy every Kubernetes object using a provider that cannot yet authenticate or reach the endpoint. Separate execution phases and identities make bootstrap failures understandable.

Use the multi-AZ network contract from earlier chapters. Private node subnets need outbound paths to required services or configured endpoints. Nodes must retrieve images, communicate with the control plane, and use identity services. Do not assume a private endpoint makes all required traffic private automatically.

The Module Is Community Maintained

The widely used `terraform-aws-modules/eks/aws` module is community maintained, not an AWS-official Terraform module. The AWS provider supplies the underlying managed resources. Use an exact module version and review its inputs rather than mixing examples from different major releases.

This example targets module 21.3.0, whose inputs include `name`, `kubernetes_version`, and `addons`. Older examples using `cluster_name` or `cluster_addons` should not be pasted into this configuration unchanged.

eks.tf

```
variable "vpc_id" { type = string }
variable "private_subnet_ids" { type = list(string) }
variable "platform_admin_role_arn" { type = string }
variable "kubernetes_version" {
  type = string
  # Supply a currently supported version verified for this region.
}
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "21.3.0"

  name               = "orders-platform"
  kubernetes_version = var.kubernetes_version
  vpc_id             = var.vpc_id
  subnet_ids         = var.private_subnet_ids

  endpoint_private_access = true
  endpoint_public_access  = false
  enable_irsa             = true
  authentication_mode     = "API"
  enable_cluster_creator_admin_permissions = false

  # Explicit administrative identity, independent of the original creator.
  access_entries = {
    platform_admin = {
      principal_arn = var.platform_admin_role_arn
      policy_associations = {
        admin = {
          policy_arn = "arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy"
          access_scope = { type = "cluster" }
        }
      }
    }
  }
  addons = {
    vpc-cni = { before_compute = true }
    coredns = {}
    kube-proxy = {}
  }
  eks_managed_node_groups = {
    application = {
      ami_type       = "AL2023_x86_64_STANDARD"
      instance_types = ["m6i.large"]
      min_size       = 2
      max_size       = 4
      desired_size   = 2
      disk_size      = 40
    }
  }
  tags = { Project = "orders", Environment = "lab" }
}
output "cluster_name" { value = module.eks.cluster_name }
output "oidc_provider_arn" { value = module.eks.oidc_provider_arn }
```

Keep the approved AWS provider pin and remote backend in this root. The subnet list must span appropriate AZs and contain enough IP capacity. This configuration is a foundation, not a claim that every production control is complete.

Review add-on versions explicitly before adoption. Omitting versions lets selection behavior depend on module and API defaults. For a reproducible platform release, record compatible tested versions, node-image selection, and the control-plane version in the release inventory.

The private endpoint requires an agent or operator with network reachability through the VPC or approved connected network. A laptop's AWS credentials do not provide that path. Plan how emergency administrators authenticate and connect before disabling public access.

IRSA Connects a Specific Service Account

Enabling an OIDC provider does not grant every Pod AWS permissions. IRSA maps a projected service-account token to an IAM role through issuer, subject, and audience conditions.

The example below trusts only `orders/orders-worker`. Give its role a separate policy granting the intended object-store operations; do not attach broad storage administration.

worker-identity.tf

```
locals {
  issuer = replace(module.eks.cluster_oidc_issuer_url, "https://", "")
}
resource "aws_iam_role" "worker" {
  name = "orders-worker-irsa"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Federated = module.eks.oidc_provider_arn }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "${local.issuer}:sub" = "system:serviceaccount:orders:orders-worker"
          "${local.issuer}:aud" = "sts.amazonaws.com"
        }
      }
    }]
  })
}
output "worker_role_arn" { value = aws_iam_role.worker.arn }
```

worker-service-account.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: orders-worker
  namespace: orders
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/orders-worker-irsa
```

Replace the ARN with the output and set the worker Pod's `serviceAccountName`. Compatible SDKs must use the projected identity correctly. A Pod configured with `automountServiceAccountToken: false` needs deliberate review of the IRSA webhook's token projection rather than assuming the Chapter 12 non-cloud fixture settings apply unchanged.

EKS Pod Identity is another supported approach with different prerequisites and associations.

Do not mix mechanisms accidentally. Record which identity path the workload uses and test its effective AWS identity from an authorized diagnostic workload.

Access Is Not One Permission

AWS authorization to call `DescribeCluster`, Kubernetes authentication, Kubernetes authorization, and endpoint connectivity are separate requirements. `aws eks update-kubeconfig` writes connection configuration; it does not grant cluster administrator rights.

```
aws eks update-kubeconfig --region ap-south-1 \
  --name orders-platform
kubectl config current-context
kubectl auth can-i get pods -n orders
kubectl get nodes -o wide
kubectl -n kube-system get pods
```

Use the explicit approved role and private connectivity. Inspect node readiness, DNS, CNI health, available IPs, and image retrieval before deploying the API. A healthy control plane with broken node networking does not constitute a working platform.

Upgrades Are Platform Releases

Rehearse upgrades with representative workloads and admission policies. Check deprecated APIs, add-on compatibility, node AMIs, client versions, and disruption budgets. A control-plane upgrade and worker replacement are related but distinct operations.

Terraform's desired node count can conflict with an autoscaler that owns runtime capacity. Decide which controller owns which fields and document ignored changes carefully. Do not add broad `ignore_changes` rules merely to suppress unexplained drift.

Drain nodes only after reviewing capacity and PDB behavior. A two-node pool may not have enough room for a surge workload during maintenance. Prove scheduling headroom and storage placement rather than relying on minimum node count alone.

Recovery and Cleanup

Cloud state records platform ownership; it does not contain PostgreSQL records, Kubernetes Secret recovery material, or every dynamically created volume. Inventory add-on-created resources and their owner before dismantling the cluster.

Remove application load balancers and disposable workloads through their owning controllers before destroying cloud networking. Retain protected database volumes according to policy. Destroying the EKS cluster first can orphan controller-managed AWS resources and leave chargeable infrastructure without its reconciliation engine.

Test a node-group replacement, lost administrative session, denied workload identity, and failed image pull in the lab. Record whether existing orders remain durable and whether diagnostics identify the correct failed boundary.

A reproducible EKS platform is one another engineer can administer and upgrade using approved identities and documented connectivity. It is not simply a green Terraform apply.

Version the Entire Platform Envelope

An EKS platform release includes more than the cluster version. Record the Terraform CLI, AWS provider, EKS module, CNI, DNS, proxy, CSI components, node image, admission configuration, and diagnostic clients. Compatibility exists across that envelope, not between two version numbers alone.

For add-ons, select the tested release supported by the target cluster version and region. Observe existing configuration before upgrading so an update does not reset intentional settings. Keep identity changes and binary upgrades separately reviewable where possible.

Node bootstrap can fail while the control plane remains healthy. Inspect instance launch, security groups, outbound paths, instance roles, bootstrap logs, and cluster authentication. Do not create public nodes simply to make registration work without understanding which private dependency failed.

Test subnet exhaustion using a capacity model rather than allocating addresses until production breaks. Container networking can consume addresses beyond the number of nodes. Include surge nodes, endpoint interfaces, maintenance, and future workloads in the subnet budget.

Account for capacity during recovery. If the selected instance family is unavailable in one AZ, a multi-AZ design may still need alternate compatible instance types and correct architecture selection. A list of types should reflect tested workload compatibility rather than arbitrary availability guesses.

Finally, validate the emergency administration role from its actual access path and record that exercise. An explicit access entry is useful only if the identity can authenticate, reach the endpoint, and perform the documented repair actions without inheriting unrelated platform privileges.

This inventory becomes the baseline for upgrades and incident comparison: when behavior changes, the team can identify which part of the operating envelope changed.

Test a denied administrative action as well as a permitted read, confirming the chosen identity respects the intended namespace and cluster boundaries.

Key Takeaways

- **Managed control planes still require explicit platform ownership.**
- **Pin the community module and use its version-specific inputs.**
- **Separate cloud bootstrap from cluster configuration.**
- **IRSA trusts a constrained issuer, subject, and audience.**
- **Private endpoints require a tested administration path.**
- **Upgrade and cleanup plans must include add-ons and persistent resources.**

References

- EKS module 21.3.0: Documentation
- IRSA: Documentation

Chapter 14: Traffic Has Boundaries: TLS, Network Policies, and FortiGate

One Container, Every Database

An attacker exploits an exposed API dependency. From that container, the attacker connects to billing, administration, and the database because the cluster has no enforced traffic policy. Network reachability expands a single application flaw into a platform incident.

A VPC perimeter firewall did not see the Pod-to-Pod path. The security diagram showed a firewall icon but never demonstrated packet traversal. **Controls protect traffic only where they are actually enforced.**

North-South and East-West Are Design Categories

Ingress or gateway infrastructure commonly handles traffic entering the application from outside the cluster. NetworkPolicies constrain selected Pod ingress and egress. They can also affect external destinations; they are not restricted to east-west traffic alone.

A Kubernetes Ingress object needs an installed controller and an appropriate ingress class. It is a routing declaration, not an automatic TLS endpoint or authorization layer. Configure DNS, certificate issuance, redirect behavior, request limits, and authentication explicitly.

NetworkPolicies require a network implementation that enforces them. Applying YAML to an unsupported cluster can succeed while traffic remains unrestricted. Verify implementation compatibility, then run positive and negative connectivity tests.

Policy rules are additive. A default-deny policy selects traffic for isolation; another allow policy can permit it. There is no general ordering where a later deny overrides all earlier allows. Labels are security-relevant selectors and must be protected from arbitrary mutation.

Establish Default Deny

Apply this to the backend namespace after inventorying its dependencies. An unplanned production default deny can cause an outage by blocking DNS, secret retrieval, monitoring, or health paths.

default-deny.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: backend
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

This does not cover every possible host-network or infrastructure packet path. Review your CNI's behavior, node traffic exceptions, and cloud security-group interactions. NetworkPolicy is one layer in the trust model.

allow-frontend.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-api
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: orders-api
  policyTypes: [Ingress]
  ingress:
    - from:
        # One peer containing both selectors means namespace AND Pod label.
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: frontend
          podSelector:
            matchLabels:
              app: orders-frontend
      ports:
        - protocol: TCP
          port: 8080
```

If the namespace and Pod selectors appear as separate entries under `from`, they become alternative allowed peers rather than the intended intersection. This small YAML difference can widen access substantially.

The frontend namespace also needs an outbound permission if it has default-deny egress. Policy permission at the backend does not override a denied sender path.

allow-api-egress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-required-egress
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: orders-api
  policyTypes: [Egress]
```

```
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - {protocol: UDP, port: 53}
        - {protocol: TCP, port: 53}
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: data
          podSelector:
            matchLabels:
              app: postgres
      ports:
        - {protocol: TCP, port: 5432}
```

Verify the DNS labels and topology in your cluster. Node-local DNS and managed DNS paths may differ. A managed external database is not selected by a Pod label; use an appropriate network design and policy mechanism for its actual addresses.

The database namespace needs matching ingress permission. Default-deny policy on both ends creates an explicit communication contract rather than relying on one namespace's behavior.

TLS Is an Identity Boundary

Terminate TLS at a reviewed entry point and decide whether to re-encrypt to the backend. Validate certificates and service identities on internal connections where the threat model requires it. "Private network" does not justify disabling database TLS verification.

Certificate lifecycle includes issuance, renewal, trust-store updates, and emergency replacement. Monitor expiry and renewal errors. A certificate controller's Kubernetes permissions and cloud DNS permissions should be narrowly scoped.

Keep application authorization separate: TLS authenticates selected endpoints and protects transport, while the API must still authenticate callers and enforce order access. A valid encrypted request can still be malicious.

FortiGate Must Be on the Actual Route

FortiGate-VM can provide inspected routing at the VPC edge or between connected networks. Its deployment requires appropriate licensing, throughput sizing, interface configuration, high availability, and cloud routing integration. Treat these as tested prerequisites, not an automatic free feature.

A conceptual routed inspection path is:

Source	Destination	Next hop	Required return behavior
Application subnet	Approved external network	Inspection appliance or supported endpoint	Return traffic through the same inspected flow

Source	Destination	Next hop	Required return behavior
Appliance external path	Internet or VPN peer	Approved gateway	Session-aware return routing
Connected cloud	Application subnet	Inspection path	No bypass route to workload subnet

In AWS, direct appliance routing may target an appliance ENI with source/destination checks disabled when required by the design. Other architectures use Gateway Load Balancer endpoints and appliance mode. These are different implementations; do not mix their routes or assume one example configures both.

High-level FortiGate policy intent is: trusted application zone to approved destination objects, specified services only, logging enabled, and deny everything else. Build address and service objects from the actual allocation plan. Use a separate management interface or path with restricted administrators.

Deep packet inspection of encrypted traffic requires an explicit TLS-inspection design and trusted certificates, with privacy, application compatibility, and failure behavior reviewed. A firewall cannot inspect plaintext inside arbitrary TLS without that arrangement.

Test Packet Paths

Choose synthetic test Pods in frontend, backend, and an unauthorized namespace. Use a reviewed diagnostic image digest and a nonprivileged identity. Verify frontend-to-API succeeds and unauthorized-to-API fails. Check both UDP and TCP DNS where needed.

```
kubectl -n backend get networkpolicy
kubectl -n backend describe networkpolicy allow-frontend-to-api
kubectl -n backend get pods --show-labels
kubectl get namespaces --show-labels
```

A failed request alone does not prove policy enforcement: the destination may be down. Confirm the same destination is reachable from the approved caller during the test. Record firewall logs, CNI observations, and application outcomes together.

For appliance routes, capture the effective route table and demonstrate traffic appears in inspection logs. Test an intentional bypass attempt and asymmetric return route in an isolated environment. Measure MTU, connection establishment, and throughput rather than assuming VPN plus inspection preserves application performance.

State and Recovery

Version policies, route definitions, certificates' configuration references, and firewall configuration backups. Secret keys and private certificates belong in protected storage. Do not commit exported firewall configurations containing passwords or VPN pre-shared keys.

A policy rollback must identify the exact previous reviewed revision. Removing all NetworkPolicies may restore traffic but also remove the security boundary; use a targeted emergency procedure with owner and expiry.

Recover the firewall's configuration, licenses, routing integration, and administrative identity together. An appliance that boots with default routes is not a restored inspection path. A cluster policy that exists without enforcement is not a restored security boundary.

Application teams own dependency declarations; platform and security teams own enforcement contracts. Keep a reviewed communication matrix so changing a service endpoint does not silently require broadening the network to everywhere.

Make Dependency Changes Reviewable

Maintain a communication matrix with caller identity, destination identity, protocol, port, purpose, and verification method. When a team adds an external API or moves a database, the change should update that contract and its enforcement together.

Avoid permanent temporary allowances. An emergency broad rule needs a named owner, expiry, monitoring, and removal verification. The incident record should explain what narrower rule replaces it. Otherwise the next audit inherits a network whose real trust boundaries exist only in old chat messages.

For service discovery, distinguish names from enforcement addresses. Standard NetworkPolicy does not universally provide portable DNS-name egress matching. If a platform uses FQDN-aware policy extensions, document their controller, DNS behavior, and cloud-specific portability limits.

Observe both directions when troubleshooting. Source egress, destination ingress, cloud security groups, route tables, and appliance policy may all participate. A policy change at the wrong layer can broaden access without affecting the actual blocked path.

Treat namespace and Pod label mutation as a security capability. A user able to assign trusted labels may place a workload inside an allow selector. Admission and RBAC should align with the label-based model rather than letting any tenant impersonate a permitted caller.

During recovery, validate the restored policy against current allocation and service identity. A firewall configuration from last month can point to retired addresses. Configuration backup is necessary, but successful restoration also requires correct current routes, identities, and tests.

The final evidence should show a working permitted transaction, a denied unauthorized transaction, and the expected enforcement logs. That combination supports the boundary more strongly than either successful access or an isolated timeout alone.

Repeat the negative connectivity test after policy upgrades; successful enforcement last month does not verify the current network implementation.

Key Takeaways

- **Prove enforcement, not just successful YAML creation.**
- **NetworkPolicies are additive and selector placement matters.**
- **Default deny needs explicit DNS and dependency allowances.**
- **TLS, routing, and application authorization are separate controls.**
- **FortiGate inspection requires tested forward and return paths.**
- **Version and rehearse recovery of policies and appliance configuration.**

References

- NetworkPolicies: Documentation
- Fortinet AWS documentation: Documentation

Chapter 15: Stateful Kubernetes: Storage, Databases, and Safe Migrations

The Database Could Not Follow the Pod

A node in AZ A fails. The database replacement is expected to run in AZ B, but its existing EBS volume belongs to AZ A. The scheduler reports a volume topology conflict, and the database remains unavailable.

Kubernetes does not normally solve that conflict by moving an EBS volume across zones. EBS is zonal; changing Pod placement does not change that storage boundary. The outage came from assuming process replacement implied data mobility.

Schedule state with its storage constraints, and recover data through a tested replication or restore procedure. Availability Zones are not interchangeable mount points.

PV, PVC, and StorageClass Have Different Owners

A PersistentVolume represents storage capacity available to the cluster. A PersistentVolumeClaim requests capacity with access and class requirements. A StorageClass describes how dynamic provisioning should obtain that capacity.

The CSI driver connects Kubernetes storage operations to the underlying cloud service. Installing its manifests without the required identity, permissions, compatible version, and node components does not establish a working storage platform.

For EBS, use a supported EBS CSI driver with a narrowly scoped cloud identity. Record which deployment system owns the driver and its version. The following lab assumes it is already installed and functioning; the database manifest does not install it.

`ReadWriteOnce` means a volume can be mounted read-write by a single node in the relevant semantics, not that it is a universal one-Pod security boundary. `ReadWriteOncePod` provides different behavior with compatible drivers. Understand the selected access mode before attempting failover.

Provision with Scheduling Context

`WaitForFirstConsumer` delays provisioning until scheduling requirements are known. That helps avoid creating a volume in a zone incompatible with the eventual Pod. It does not make an existing zonal volume portable across zones.

storage-class.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: orders-ebs-gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
reclaimPolicy: Retain # Claim deletion must not automatically erase data.
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

For customer-managed encryption, supply the approved KMS configuration supported by the driver and verify its IAM/key-policy requirements. Encryption flags do not guarantee the recovery role can decrypt a restored volume.

A Retain policy preserves the storage object after claim deletion, but administrators can still delete cloud volumes. Retained resources require ownership inventory and cleanup procedures; otherwise the result is both a recovery hazard and a recurring storage bill.

A Single-Instance Database Lab

This PostgreSQL StatefulSet demonstrates stable identity and persistent claims. It does not provide database replication, automatic primary election, or cross-AZ database availability. Do not increase `replicas` and expect three independent PostgreSQL containers to become one replicated database.

Create a protected `postgres-auth` Secret containing `password` through your approved secret workflow. Avoid putting literal credentials in the manifest or shell history. The image reference must point to the reviewed PostgreSQL 17 digest.

postgres.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: data
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - name: postgres
      port: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: data
spec:
  serviceName: postgres
  replicas: 1
  selector:
```

```
  matchLabels:
    app: postgres
template:
  metadata:
    labels:
      app: postgres
  spec:
    terminationGracePeriodSeconds: 60
    containers:
      - name: postgres
        image: postgres:17@sha256:REPLACE_DIGEST
        ports:
          - name: postgres
            containerPort: 5432
        env:
          - name: POSTGRES_DB
            value: orders
          - name: POSTGRES_USER
            value: orders_admin
          - name: POSTGRES_PASSWORD
            valueFrom:
              secretKeyRef:
                name: postgres-auth
                key: password
          - name: PGDATA
            value: /var/lib/postgresql/data/pgdata
        resources:
          requests:
            cpu: 250m
            memory: 256Mi
          limits:
            memory: 1Gi
        readinessProbe:
          exec:
            command: [pg_isready, -U, orders_admin, -d, orders]
          periodSeconds: 5
        volumeMounts:
          - name: data
            mountPath: /var/lib/postgresql/data
volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: orders-ebs-gp3
      resources:
        requests:
          storage: 20Gi
```

The official image's entrypoint can need elevated permissions to initialize storage before switching to its database user. This lab manifest does not claim compliance with a namespace's restricted Pod Security policy. Review the chosen image and ownership initialization before enforcing non-root settings; do not copy the API's UID blindly.

The readiness command checks whether PostgreSQL accepts connections. It does not prove application credentials, schema compatibility, or transaction correctness. The admin initialization user is not the role your API should use.

Verify Claim and Zone

```
kubectl -n data get statefulsets,pods,pvc
kubectl get pv
kubectl -n data describe pod postgres-0
kubectl -n data describe pvc data-postgres-0
```

Inspect the PV's node affinity, the node's zone labels, and the underlying EBS volume. Create a synthetic row, replace the Pod, and verify persistence. Then simulate loss of the node in a lab with compatible replacement capacity in the same zone.

A forced detach while the old writer is alive can risk data corruption. Confirm fencing and instance state before authorizing storage takeover. "The node is unreachable" is not proof that its database process cannot write.

Managed Services Are a Tradeoff

RDS removes many operational tasks associated with hosting PostgreSQL on Kubernetes, but does not remove schema design, credentials, connection management, backup policy, or application migration responsibilities.

Compare the real requirements: supported extensions, patch control, failover behavior, recovery objectives, staffing, cost, and compliance. In-cluster databases need operators or explicit procedures for replication, primary selection, backups, and upgrades. A StatefulSet alone supplies none of those database protocols.

Managed endpoints introduce DNS and connection-pool behavior during failover. Clients must reconnect correctly, respect TLS validation, and retry only safe operations. The application cannot assume a persistent TCP connection survives primary replacement.

Migrate Schemas Safely

Use an expand-and-contract sequence. Add compatible structures, deploy consumers capable of working with both versions, backfill with monitored batches, then remove old structures in a later approved release.

A migration job must be serialized and identifiable. Multiple application replicas running startup migrations can contend or execute destructive operations unexpectedly. Give the migration identity only the required schema privileges and revoke unused elevated access.

Capture migration revision, image digest, database version, start/end timestamps, and validation results. A migration that commits successfully can still violate application expectations. Test representative queries and background processing before declaring success.

Rollback must account for data written under the new schema. Reverting the application image does not recreate dropped columns or translate new records automatically. Define a forward-fix or recovery decision before approving destructive changes.

Backup and Recovery

Use PostgreSQL-aware backups and retained WAL where point-in-time recovery is required. Volume snapshots need a reviewed application-consistency mechanism; filesystem capture alone is not a complete database recovery contract.

Restore to isolated storage and validate database startup, schema revision, row counts, roles, and representative order workflows. Record the latest recoverable transaction to measure RPO.

Include secret retrieval, KMS access, image retrieval, and network policy in the RTO measurement.

Cross-AZ recovery creates a new volume from a snapshot in the destination zone or uses database replication designed for that boundary. Update the storage ownership carefully; do not edit PV topology to pretend an old EBS volume moved.

Cleanup and Ownership

Before deleting a StatefulSet or namespace, inspect PVC retention settings, PV reclaim policy, external snapshots, and recovery records. Keep names and volume IDs in the asset inventory. Never let a generic application cleanup job delete database storage.

Choose one owner for CSI installation and cloud identities. Terraform, Helm, and manually installed add-ons must not reconcile the same driver simultaneously. Recovery procedures should specify which layer is repaired first.

The lab is complete when Pod replacement preserves data and a separate restore exercise meets its measured target. Cross-AZ database availability remains unverified until a real replication or recovery architecture is tested.

Storage Expansion Is an Operational Change

Increasing a PVC's requested size requires driver and StorageClass support. The cloud volume and mounted filesystem may complete expansion at different times, so monitor claim conditions and actual available capacity. Do not replace a database volume merely to enlarge it.

Before expansion, record the claim, PV, cloud volume, zone, encryption key, and backup checkpoint. Confirm the request changes the intended resource rather than creating a new empty claim with a similar name. Review whether the filesystem or database requires additional handling.

Database capacity planning includes IOPS, throughput, WAL growth, temporary files, and retained recovery data, not just table size. A volume with spare gigabytes can still be saturated by I/O. Observe service latency and queueing during growth.

Storage ownership must survive namespace or controller changes. If a migration creates a new claim, document which one becomes authoritative and how the old copy is retained or retired. Two writable database copies cannot be treated as harmless backup variants.

The acceptance result should include the expanded mounted capacity, unchanged representative records, and a successful recovery-point verification. This makes storage maintenance part of the same data-preservation discipline as release and failover.

Check retained cloud volumes after cleanup and assign an owner, so recovery data does not become an unidentified storage resource.

Key Takeaways

- **Zonal storage does not follow arbitrary Pod placement.**
- **WaitForFirstConsumer improves provisioning placement, not volume mobility.**
- **A StatefulSet supplies identity and claims, not database replication.**
- **Retain policies and encryption require recovery ownership.**
- **Schema migration and application rollback are different operations.**
- **Validate restoration with application transactions, not just Pod status.**

References

- StatefulSets: Documentation
- EBS CSI: Documentation

Chapter 16: Release Without Roulette: Deployments, Promotion, and Rollback

Green Pods, Unusable Service

The new release starts successfully, and its readiness endpoint returns 200 before configuration and database initialization finish. During a rolling update, Kubernetes considers the new replicas ready and removes old capacity. User requests fail even though the deployment reports progress.

A readiness endpoint pointing at a nonexistent path would normally block progress instead; the catastrophic case is a probe that incorrectly reports usable capacity. **Readiness must describe the serving contract, not merely the existence of a process.**

Package Configuration Without Rebuilding

Helm templates package Kubernetes configuration. Values select environment behavior, but the application image remains the same tested digest. Keep chart version, image digest, configuration revision, and schema compatibility in the release record.

Do not rebuild an image for each environment. If production needs a different endpoint, configure it at runtime. Rebuilding creates a new artifact that did not pass the same staging evidence.

Charts need explicit contracts. A values schema can reject missing image digests, invalid replica counts, or unsupported configuration shapes before deployment. Template checks do not prove live admission compatibility, sufficient capacity, or application correctness.

Environment Values

values-dev.yaml

```
image:
  repository: registry.example/orders-api
  digest: sha256:REPLACE_REVIEWED_DIGEST
replicaCount: 1
resources:
  requests: {cpu: 100m, memory: 64Mi}
  limits: {cpu: 500m, memory: 128Mi}
rollout:
  maxSurge: 1
  maxUnavailable: 0
```

```
config:
  logLevel: debug
```

values-prod.yaml

```
image:
  repository: registry.example/orders-api
  digest: sha256:REPLACE_SAME_REVIEWED_DIGEST
replicaCount: 3
resources:
  requests: {cpu: 250m, memory: 256Mi}
  limits: {cpu: 1000m, memory: 512Mi}
rollout:
  maxSurge: 1
  maxUnavailable: 0
config:
  logLevel: info
```

The production values are capacity hypotheses, not benchmark results. Debug logging should not expose secrets even in development. Secrets must remain references to protected sources rather than plaintext values in Git or Helm release history.

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  progressDeadlineSeconds: 180
  minReadySeconds: 10
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: {{ .Values.rollout.maxSurge }}
      maxUnavailable: {{ .Values.rollout.maxUnavailable }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      terminationGracePeriodSeconds: 30
      containers:
        - name: api
          image: "{{ .Values.image.repository }}@{{ .Values.image.digest }}"
          ports:
            - name: http
              containerPort: 8080
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
          readinessProbe:
            httpGet:
              path: /readyz
```

```
            port: http
          periodSeconds: 5
        livenessProbe:
          httpGet:
            path: /healthz
            port: http
          periodSeconds: 10
```

This template excerpt must be combined with the security context, service account, temporary volume, and startup behavior from Chapter 12 before deployment. The repository labels partial templates explicitly; do not apply this excerpt as if it were the entire secure chart.

Rolling Updates Need Headroom

`maxUnavailable: 0` keeps requested old serving capacity while replacements become ready. `maxSurge: 1` permits one extra Pod. Neither guarantees zero downtime: readiness accuracy, endpoint propagation, connection draining, topology, and actual capacity still matter.

A surge Pod that cannot schedule can stall the release. A cluster autoscaler may add capacity, but node startup and image retrieval take time. Reserve or prove sufficient headroom and define a deployment timeout based on measured behavior.

`minReadySeconds` requires a period of stable readiness before treating a replica as available. It catches selected startup instability, not every business regression. Post-deployment checks must exercise order creation and processing.

Choose a Traffic Strategy

Rolling updates replace replicas incrementally using the ordinary Service. They are simple but temporarily mix versions, requiring API and schema compatibility.

Blue-green releases maintain two versions and switch traffic deliberately. They need additional capacity and still require safe database coexistence. Switching a Service selector does not drain every existing connection instantly or update every external load balancer at the same moment.

Canary releases expose a small share of real traffic to the candidate and compare behavior. Kubernetes replica ratios alone do not guarantee exact traffic percentages. Use a reviewed ingress, gateway, or rollout controller capable of the intended traffic weighting.

The canary gate should compare error rates, latency, business transactions, and relevant downstream pressure. A healthy candidate serving only simple read requests may still fail order writes. Define the request mix and sample size before concluding it is safe.

Release Gates

Before promotion, verify trusted image evidence, signature policy where configured, configuration validity, migration compatibility, spare capacity, and recovery readiness. The approver must know what is changing and how the old version can still run.

```
helm lint ./charts/orders-api
helm template orders-api ./charts/orders-api \
  -f values-prod.yaml > rendered.yaml
# Validate in the intended existing namespace and context.
kubectl apply --dry-run=server -f rendered.yaml
helm upgrade --install orders-api ./charts/orders-api \
```

```
  --namespace orders -f values-prod.yaml \
  --wait --timeout 5m --atomic
```

The full chart must exist and placeholders must be replaced. These commands demonstrate the release workflow, not an invitation to bypass chart completeness checks.

Helm `--atomic` attempts rollback when an upgrade fails according to Helm's checks. It cannot reverse external side effects, every hook action, or committed database migrations. Readiness failure detection and rollback policy remain application-aware decisions.

Rollback Has a Compatibility Boundary

An image rollback can restore application code. It cannot recover deleted rows or restore a dropped database column. If the new application wrote incompatible records, the old code may fail after an apparently successful rollback.

Use additive schema changes and a compatibility window. Execute destructive migrations only after older consumers are retired and the recovery decision is explicit. A migration backup is valuable only when restoration and subsequent write loss are understood.

Record Helm history and the image/configuration identities of each revision. Do not assume a chart rollback recreates secret values that changed outside Helm. Runtime secret rotation and external feature flags need their own historical record and recovery procedures.

Verify User Behavior

After rollout, test through the actual user entry point rather than only port forwarding into a Pod. Create a synthetic order with an idempotency key, observe worker processing, and verify expected data persistence. Use a dedicated synthetic identity and avoid polluting real billing or fulfillment systems.

Compare metrics before, during, and after release. Distinguish startup noise from sustained regression, and define the observation window. A pipeline that finishes immediately after readiness may miss failures triggered by scheduled jobs or connection-pool saturation.

Exercise a failing image pull, insufficient capacity, false readiness, and an incompatible migration in staging. Confirm the release halts, alerts identify the right cause, and rollback or forward repair follows the documented compatibility boundary.

State and Authority

Helm release records live in the cluster and can contain rendered values. Restrict their access and avoid storing secrets unnecessarily. Git stores chart definitions and non-secret values; the registry stores immutable images; the database owns durable records.

Give the deployment identity only the required namespace-scoped permissions where possible. Cluster-wide custom resources or admission components need separately reviewed authority. A chart that installs cluster administration should not run under an ordinary application release role.

Destroying the namespace is not a release rollback. It can remove secrets, claims, and release records. Keep destructive cleanup separate from version promotion and incident repair.

A Release Needs a Decision Window

Define how long to observe the candidate and what evidence causes promotion, rollback, or a pause. Some failures occur only after caches expire, scheduled workers run, or connection pools reach steady state. A thirty-second smoke test may miss all three.

Use a release checklist that names the old and new image digests, configuration revision, migration revision, expected traffic mix, and the compatible rollback target. Keep the observation period bounded so an ambiguous canary does not become a permanent mixed-version state.

Compare candidate and baseline under similar traffic. If the canary receives a different customer segment or only cached requests, lower latency may reflect the request mix rather than better behavior. Preserve trace context and bounded labels to analyze the difference without leaking customer data.

For blue-green switching, identify sessions and background workers that remain on the old version. Traffic routing to a new frontend does not stop an old worker from writing. Disable or retire competing consumers only through a workflow that preserves in-flight work and idempotency.

A rollback gate should consider data compatibility before automation acts. Automatically reverting code after a destructive migration can make the outage worse. Where compatibility is uncertain, pause traffic or apply an approved forward fix rather than guessing.

Record the final decision and its supporting evidence in the release inventory. If the release is paused, specify who owns it and which version currently serves each path. Operational ambiguity itself is a risk during the next incident.

This discipline makes deployment a controlled change in service behavior, rather than a sequence of Kubernetes objects that happened to reconcile.

Record observation-window results before promotion, including whether background workers completed successfully under the candidate configuration and schema.

Key Takeaways

- **Promote the same image digest with reviewed environment configuration.**
- **Readiness must reflect usable serving capacity.**
- **Rolling updates need spare capacity and graceful termination.**
- **Canary weighting requires an actual traffic-control mechanism.**
- **Application rollback does not reverse database side effects.**
- **Verify complete order workflows through the real entry point.**

References

- Helm upgrade: Documentation
- Deployment strategies: Documentation

Chapter 17: Scale What You Can Measure: Reliability and Observability

Fifty Replicas, One Exhausted Database

The API becomes CPU-bound during a promotion. HPA adds replicas, each opening a large PostgreSQL connection pool. The database reaches its connection limit, queries queue, and every new Pod increases the pressure.

The autoscaler followed its CPU signal. The architecture failed to bound downstream demand. **Scaling the caller is not the same as increasing end-to-end capacity.** Observe the bottleneck before deciding which component should grow.

Define the Service's Evidence

Metrics summarize rates, distributions, and resource use. Logs explain individual events and errors. Traces connect work across services. None is a complete substitute for the others.

For the order API, measure request rate, errors, latency distribution, active requests, database pool occupancy, query latency, and timeout rates. For the worker, measure queue age, pending work, processing duration, retries, and dead-letter or failed-work counts.

Business completion matters: accepting an order while the worker never processes it is not a healthy order service. Define an objective for completion latency and durability separately from HTTP availability.

Protect telemetry as operational data. Logs can contain customer identifiers, tokens, SQL parameters, and exception details. Redact deliberately, restrict readers, and establish retention. Do not log authorization headers to simplify debugging.

Capacity Is a Budget Across Dependencies

Suppose the database allows 200 application connections after reserving administrative and recovery capacity. If each API replica can open ten connections, twenty replicas already consume that budget before workers are counted.

Choose a maximum replica count together with pool limits, worker concurrency, query behavior, and proxy or pooling architecture. Database CPU, I/O, locks, and memory can become bottlenecks before the connection ceiling is reached.

Retries also consume capacity. Use bounded exponential backoff with jitter and idempotency. A timeout does not prove the transaction failed; retrying blindly can duplicate work. Admission

control and load shedding may protect the database better than adding more callers.

HPA Requires Requests and Metrics

CPU utilization percentages are relative to CPU requests. Missing requests or unavailable metrics can prevent the intended scaling behavior. The cluster needs a supported metrics pipeline and sufficient node capacity.

hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: orders-api
  namespace: orders
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: orders-api
  minReplicas: 3
  maxReplicas: 10 # Chosen with the database connection budget.
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 65
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
        - type: Pods
          value: 2
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 20
          periodSeconds: 60
```

These values are a controlled starting point, not a universal tuning answer. Measure how quickly a new replica becomes useful and whether scale-up increases database pressure. A slow image pull or startup path can make aggressive scaling ineffective.

The Deployment still needs its CPU requests. When HPA owns replica count, do not repeatedly apply a static replica value that fights it. Helm and GitOps ownership must account for that field.

CPU is not always the right signal. A worker waiting on I/O can accumulate backlog while using little CPU. Queue age or pending work may require a custom or external metrics adapter. Validate adapter failure behavior and metric semantics before making them release gates.

PDBs Protect Selected Maintenance

A PodDisruptionBudget constrains voluntary disruptions using the eviction API. It does not guarantee survival during node crashes, network partitions, or every rollout mechanism.

pdb.yaml

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: orders-api
  namespace: orders
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: orders-api
```

With three healthy replicas, this permits selected maintenance while retaining two. If readiness falls or capacity is absent, drains can block. Do not remove the PDB automatically to make an upgrade finish; investigate why the availability requirement cannot be met.

PDBs and Deployment rollout settings control different operations. Match them with topology-spread rules, surge capacity, and node-group maintenance behavior. A strict budget on a single replica can block maintenance without providing real redundancy.

Alerts Need Actionable Semantics

Alert on customer impact and imminent exhaustion rather than every harmless spike. Example conditions include sustained error rate above the SLO threshold, oldest pending order exceeding its completion objective, database pool saturation, and unavailable ready capacity.

Define each alert's owner, severity, evidence, and first diagnostic steps. Include service identity and environment without embedding secrets. Test the actual notification route; an alert rule that fires only in a dashboard nobody watches is incomplete.

Use latency histograms with appropriate buckets rather than averaging durations. An acceptable mean can hide a severe tail. Choose bucket boundaries around the service objective and inspect aggregation across replicas carefully.

High-cardinality labels such as order IDs can overwhelm the metrics system. Put per-order context in logs or traces and use bounded labels for metrics. Telemetry ingestion must have its own resource and retention budgets.

Controlled Load Testing

Use an isolated environment and synthetic requests. Define the request mix, arrival model, payload distribution, data size, and duration. A loop of cached health requests does not benchmark order processing.

Observe baseline, gradual increase, peak, sustained load, and recovery. Record throughput alongside errors and latency; a system serving fewer requests can appear faster while failing the workload.

```
kubectl -n orders get hpa
kubectl -n orders describe hpa orders-api
kubectl top pods -n orders
```

```
kubectl -n orders get pdb
kubectl -n orders get pods -o wide
```

These commands require working metrics and appropriate permissions. Inspect HPA conditions when desired replicas do not change. Separate metric unavailability, replica limits, insufficient node capacity, and readiness failures.

During the test, compare database connections with the predicted maximum. Verify worker concurrency and retry traffic remain bounded. Observe scale-down: killing idle-looking replicas can still interrupt long requests or claimed work if termination handling is wrong.

Recovery and Telemetry State

Version dashboards, alert rules, collectors, and retention configuration. Time-series data and logs need explicit storage ownership; they do not automatically survive collector or cluster replacement.

Keep an independent incident path when the primary monitoring stack fails. Cloud service events, external probes, and protected operational access can help distinguish application failure from observability failure. Do not make every alert delivery depend on the same cluster being monitored.

Back up configuration and test recovery of the minimum diagnostic capability. Recovery objectives for telemetry can differ from order data, but losing all evidence during an incident may significantly increase service RTO.

Tune with Evidence

Change one important scaling parameter at a time and compare equivalent tests. Record why the maximum replica count, pool size, and stabilization windows were chosen. Preserve before-and-after evidence in the architecture decision record.

A larger node or managed database tier may help, but first identify locks, inefficient queries, unbounded pools, and retry amplification. Capacity purchases without diagnosis can merely move the failure threshold.

The successful result is a measured envelope: within a defined request rate and data profile, the service meets its latency and completion objectives; beyond it, the system degrades predictably without losing committed orders.

Protect the Database During Scale-Up

A connection pool should have bounded size, acquisition timeout, and clear behavior when exhausted. Avoid allowing every waiting request to open another connection. Expose pool occupancy and wait duration so operators can see whether the bottleneck is connection admission or query execution.

Apply similar bounds to the worker. Increase concurrency only when database and external side-effect capacity support it. Queue depth alone is not permission to launch unlimited processing. Long-running transactions and locked rows can keep CPU low while progress stops.

Measure retry amplification as its own signal. If one customer request produces several database attempts, apparent workload can rise faster than incoming traffic. Use time budgets across the request path so nested retries do not exceed the original service deadline.

HPA scaling needs a stable relationship between the metric and useful capacity. Observe whether adding a replica actually reduces per-replica pressure without increasing total failures. If not, the chosen signal or bottleneck model is wrong.

Separate resource exhaustion from correctness failures. A fast API returning incorrect order status is not successful capacity. Synthetic checks should validate durable state and worker completion alongside latency.

After the load ends, verify backlog drains within the expected window and scaling returns to its minimum without stranding work. Watch memory behavior and connection release across repeated cycles; a single short test can hide gradual leaks.

Preserve the dataset size, index state, and query mix used in the benchmark. Performance results from an empty database do not support a mature production workload. Capacity decisions should cite the actual test envelope and known limits rather than one impressive throughput number.

Use the measured safe envelope to set admission limits before traffic exceeds it, rather than waiting for database exhaustion.

Key Takeaways

- **Observe the complete transaction path before scaling callers.**
- **Bound replicas and connection pools together.**
- **HPA needs meaningful requests, metrics, and capacity.**
- **PDBs constrain voluntary disruption, not every failure.**
- **Alert on actionable impact and test notification delivery.**
- **Record a measured capacity envelope and recovery behavior.**

References

- HPA: Documentation
- Disruption budgets: Documentation

Chapter 18: Azure as a Second Home: Port the Platform Without Pretending Clouds Match

Renaming Resources Did Not Port the Network

The engineer replaces `aws_vpc` with an Azure-looking resource name and expects the same subnet, identity, and routing behavior. Terraform rejects the configuration. The next attempt creates AKS but fails because the cluster identity lacks permission on its selected subnet.

Cloud portability is not text substitution. Similar architectural roles have different APIs, identity models, defaults, and operational constraints. **Reuse application contracts where they are stable; implement cloud-specific infrastructure honestly.**

Map Responsibilities, Not Resource Names

Responsibility	AWS	Azure
Network address domain	VPC	Virtual Network
Managed Kubernetes	EKS	AKS
Image registry	ECR	ACR
Resource authorization	IAM	Azure RBAC and Microsoft Entra identities
Pod cloud identity	IRSA or EKS Pod Identity	Microsoft Entra Workload ID
Terraform backend	S3	Azure Blob backend

Microsoft Entra ID is the current name for the directory platform previously called Azure AD. Directory identity, Azure resource authorization, and Kubernetes RBAC are related but distinct layers. Do not assume authenticating to Azure grants access to every AKS namespace.

Use separate Azure roots, provider locks, backend credentials, and subscriptions for environment boundaries. Do not make the Azure apply depend on an AWS state backend that would be unavailable during an AWS outage unless that dependency has an independently tested recovery plan.

A Deliberate AKS Foundation

This example uses Azure CNI overlay with Cilium for network policy enforcement. Pod address ranges and service ranges must not overlap the VNet or connected networks. Confirm the chosen region, version, and network configuration are supported before deployment.

The example assumes an existing delegated application subnet and an existing user-assigned cluster identity authorized to use it. That keeps network ownership separate. Grant the identity the required network role at the intended scope through a reviewed bootstrap step, not an unrestricted subscription-wide assignment.

aks.tf

```
terraform {
  required_version = ">= 1.10.0, < 2.0.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0" # Commit lock selection; review upgrades.
    }
  }
}
variable "subscription_id" { type = string }
variable "subnet_id" { type = string }
variable "cluster_identity_id" { type = string }
variable "kubernetes_version" { type = string }
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
resource "azurerm_resource_group" "platform" {
  name     = "orders-platform-lab"
  location = "centralindia"
}
resource "azurerm_kubernetes_cluster" "platform" {
  name                = "orders-aks"
  location            = azurerm_resource_group.platform.location
  resource_group_name = azurerm_resource_group.platform.name
  dns_prefix          = "orders-aks"
  kubernetes_version  = var.kubernetes_version

  private_cluster_enabled   = true
  oidc_issuer_enabled        = true
  workload_identity_enabled = true
  role_based_access_control_enabled = true

  identity {
    type         = "UserAssigned"
    identity_ids = [var.cluster_identity_id]
  }
  default_node_pool {
    name           = "system"
    node_count     = 2
    vm_size        = "Standard_D2s_v5"
    vnet_subnet_id = var.subnet_id
  }
  network_profile {
    network_plugin      = "azure"
    network_plugin_mode = "overlay"
    network_data_plane  = "cilium"
    pod_cidr            = "10.81.0.0/16"
    service_cidr        = "10.82.0.0/16"
    dns_service_ip      = "10.82.0.10"
  }
}
```

```
output "oidc_issuer" {
  value = azurerm_kubernetes_cluster.platform.oidc_issuer_url
}
```

The provider range is not an exact binary selection; the committed lock file establishes that selection, and the deployment baseline must record it. Generate the lock from the actual trusted registry, not by inventing checksums. A future provider upgrade requires a reviewed plan.

A private AKS endpoint needs private DNS and operator connectivity. Test the execution agent's DNS resolution and route to the API before deployment steps. As on AWS, credentials do not create network reachability.

Separate Cluster and Workload Identities

The cluster's managed identity performs platform operations. The kubelet identity retrieves selected node resources, including registry images where authorized. Application Pods should obtain their own scoped workload identity rather than inherit broad node access.

For a worker identity, create a user-assigned managed identity and federated credential constrained to the AKS issuer and service-account subject:

worker-identity.tf

```
resource "azurerm_user_assigned_identity" "worker" {
  name                = "orders-worker"
  location            = azurerm_resource_group.platform.location
  resource_group_name = azurerm_resource_group.platform.name
}
resource "azurerm_federated_identity_credential" "worker" {
  name                = "orders-worker-aks"
  resource_group_name = azurerm_resource_group.platform.name
  parent_id           = azurerm_user_assigned_identity.worker.id
  audience            = ["api://AzureADTokenExchange"]
  issuer              = azurerm_kubernetes_cluster.platform.oidc_issuer_url
  subject             = "system:serviceaccount:orders:orders-worker"
}
```

Grant the worker only the required Azure resource role at the intended scope. The federation declaration enables token exchange; it does not automatically grant storage or secret permissions.

The Kubernetes service account needs the managed identity client-ID annotation, and the Pod template needs the workload-identity use label supported by the integration. Use a compatible SDK and verify effective identity. Keep application credential code behind a cloud-specific adapter rather than spreading token assumptions across business logic.

Registry and Release Contracts

ACR image retrieval requires the correct identity and repository permissions. An image available in ECR is not automatically available in Azure. Copy the approved artifact without rebuilding and verify the resulting registry digest and signature policy.

Registry translation can affect references and supported metadata. Record source and destination identities, preserve provenance, and test image pulls from the secondary cluster while the primary cloud is unreachable.

Keep release values cloud specific: ingress class, load-balancer configuration, secret integration, storage class, identity annotations, and registry location differ. Application ports, HTTP routes, idempotency behavior, and compatible schema contracts can remain stable.

State and Security Boundaries

Configure Azure remote state using its documented backend identity and locking behavior. Blob leases provide backend coordination; do not assume the S3 lock-object IAM policy applies to Azure. Scope storage access, encryption, versioning, and recovery separately.

Backend state may contain sensitive platform outputs. Restrict who can retrieve cluster credential material, avoid exporting kubeconfig indiscriminately, and use supported Entra authentication instead of broadly sharing administrative credentials.

Use network policies and admission rules consistent with the application's contracts, then prove enforcement on Azure's actual data plane. A YAML policy tested only on AWS has not been operationally verified in AKS.

Validate Differences Explicitly

Connect through the intended private path and obtain normal user credentials:

```
az account show --query '{subscription:id,tenant:tenantId}'
az aks get-credentials --resource-group orders-platform-lab \
  --name orders-aks
kubectl config current-context
kubectl get nodes -o wide
kubectl auth can-i get pods -n orders
```

Check that the selected subscription and tenant match the environment matrix. Run image-pull, DNS, workload-identity, allowed/denied traffic, storage, and graceful-shutdown tests. Compare results with AWS without pretending their implementation details match.

A successful Azure deployment does not prove disaster recovery. The database, secret source, image registry, DNS authority, and release evidence must also be available during a primary-cloud failure. Chapter 21 will define that exercise.

Upgrade and Cleanup

AKS upgrades, node images, network plugins, and identity integrations have their own compatibility requirements. Record the tested provider, Kubernetes version, CLI, and add-on inventory. Do not derive the Azure release schedule from EKS's version availability.

Before destruction, review node-resource-group ownership and controller-created resources. Retain data and backups under explicit policy, and remove application load balancers through their owner. Keep the recovery cloud's backend independent of the environment being retired.

The result is a second platform with equivalent application behavior and explicit cloud-specific operational contracts. That is a useful multi-cloud capability; a large boolean-heavy abstraction is not required to achieve it.

Model Azure Egress and Administration Separately

The network profile selects container networking behavior; it does not document every permitted destination. Decide whether outbound traffic uses the supported default path, NAT, or a user-

defined inspected route, and configure dependencies consistently with that choice.

For a private cluster, record the private DNS-zone ownership and how connected administrators resolve it. A recovery agent in another network may need different resolver forwarding from the ordinary Azure operator. Test that path while primary AWS services are unavailable.

Subscription permissions can be inherited from management groups or broad role assignments. Review effective authorization, not just the narrow role created for this chapter. A constrained worker identity does not help if the application falls back to a privileged credential embedded in configuration.

If using ACR with repository-scoped permissions or attribute-based access features, choose the registry permission model deliberately and verify the kubelet's actual pull behavior. Do not assume an old AcrPull example covers every registry authorization mode.

Back up the non-secret identity mapping and resource inventory, while keeping token and secret recovery outside Git. Changing a federated credential's subject can break workload authentication even when the managed identity still exists.

Before accepting Azure as the recovery site, prove its control path is independent: operator authentication, private API access, image retrieval, DNS authority, secrets, and database write ownership. A working deployment that relies on a live AWS service for any of those functions needs an explicit exception or redesign.

Verify the recovery operator can resolve the private endpoint from the approved network without using primary-cloud DNS dependencies.

Key Takeaways

- **Map architectural responsibilities, not API names.**
- **Keep Azure state, subscriptions, and identities independently owned.**
- **Workload federation and resource permissions are separate steps.**
- **Private AKS access requires tested DNS and connectivity.**
- **Copy approved images without rebuilding and verify destination identity.**
- **Test cloud-specific networking, storage, upgrades, and recovery.**

References

- AKS Workload ID: Documentation
- Azure CNI overlay: Documentation

Chapter 19: GCP Without a Rewrite: Test the Limits of Portability

The Friday Migration Mandate

The CTO asks for a GCP deployment by Friday. The team assumes containers make the migration straightforward. The API starts, then fails because business code expects S3-specific signed URLs, IAM credential discovery, and AWS event payloads.

The image was portable. The application contract was not. **Container portability does not imply service portability.** Identify provider-specific assumptions before estimating the migration, especially in storage, identity, events, and database features.

Build a Capability Map

Capability	AWS implementation	GCP implementation	Application consequence
Managed Kubernetes	EKS	GKE	Similar workload APIs; platform behavior differs
Image registry	ECR	Artifact Registry	Different identity and repository references
Object storage	S3	Cloud Storage	API and signed-URL semantics may differ
PostgreSQL service	RDS	Cloud SQL	Connection, identity, extensions, operations differ
Workload authorization	IAM role integration	Workload Identity Federation for GKE	Token and SDK integration changes

Keep HTTP APIs, event contracts, idempotency keys, and transaction semantics stable where possible. Put object-storage behavior behind a small application interface with explicit error and consistency expectations. Do not build an abstraction that hides every cloud feature indiscriminately.

An adapter needs tests for upload, download, metadata, deletion, retries, signed-URL expiry, and authorization failure. Equivalent method names do not prove equivalent behavior. Review required features such as object versioning, retention, encryption, and event delivery separately.

Autopilot Changes the Node Ownership Contract

GKE Autopilot manages substantial node provisioning and configuration responsibilities. You still own workload resource requests, placement requirements, supported security settings, identity, network boundaries, data, and cost behavior.

Autopilot admission can enforce or adjust workload settings. A privileged diagnostic DaemonSet that worked on your EKS lab may be rejected. Validate supported workload patterns before deciding the platform is interchangeable.

This example provisions a custom VPC, subnet with secondary ranges, and private Autopilot cluster. It is a separate GCP Terraform root with its own credentials and remote backend. Enable the required project services before provisioning and verify quota and billing.

gke.tf

```
terraform {
  required_version = ">= 1.10.0, < 2.0.0"
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 6.0" # Commit the reviewed provider selection.
    }
  }
}
variable "project_id" { type = string }
provider "google" {
  project = var.project_id
  region  = "asia-south1"
}
resource "google_compute_network" "platform" {
  name                    = "orders-platform"
  auto_create_subnetworks = false
}
resource "google_compute_subnetwork" "platform" {
  name                     = "orders-platform"
  region                   = "asia-south1"
  network                  = google_compute_network.platform.id
  ip_cidr_range            = "10.90.0.0/20"
  private_ip_google_access = true
  secondary_ip_range {
    range_name    = "pods"
    ip_cidr_range = "10.92.0.0/16"
  }
  secondary_ip_range {
    range_name    = "services"
    ip_cidr_range = "10.93.0.0/20"
  }
}
resource "google_container_cluster" "platform" {
  name             = "orders-autopilot"
  location         = "asia-south1"
  enable_autopilot = true
  network          = google_compute_network.platform.id
  subnetwork       = google_compute_subnetwork.platform.id
  deletion_protection = true

  ip_allocation_policy {
    cluster_secondary_range_name  = "pods"
```

```
    services_secondary_range_name = "services"
  }
  private_cluster_config {
    enable_private_nodes    = true
    enable_private_endpoint = true
    master_ipv4_cidr_block  = "172.20.0.0/28"
  }
  release_channel {
    channel = "REGULAR"
  }
}
```

Autopilot's managed identity and networking behavior must be checked against the selected provider and service version. Do not add a manually managed node group to this example. A release channel is an upgrade policy, not an immutable forever-version guarantee.

The address allocation includes node, Pod, Service, and control-plane ranges. Ensure none overlaps connected networks. Private nodes need a deliberate route to required external services; Private Google Access addresses selected Google-service access, not arbitrary internet egress. Add approved Cloud NAT or other paths when required.

A private endpoint needs administration from a supported connected environment. Test the agent's route, DNS, identity, and authorization. A credentials command can succeed while the Kubernetes endpoint remains unreachable.

Authenticate Without Exported Keys

Use Application Default Credentials through an approved local identity or supported workload federation in CI. Avoid downloading a long-lived service-account JSON key as the default pipeline solution.

Separate Terraform provisioning authority from workload identity. A Pod retrieving an object should not receive permission to create networks or administer the project. Restrict who can impersonate privileged service accounts and who can change federation mappings.

Workload Identity Federation for GKE has GKE-specific identity syntax and resource policies. Translate the worker's required capability into a scoped GCP permission and prove the effective identity. Do not reuse the AWS role ARN annotation and expect it to authenticate.

Registry permissions likewise differ. Artifact Registry repositories are regional resources with explicit access policies. Copy the approved image, verify its resulting digest and provenance, and ensure the secondary environment can pull it independently of AWS.

Deploy the Application Contract

Retrieve cluster credentials from the approved environment:

```
gcloud config get-value project
gcloud container clusters get-credentials orders-autopilot \
  --region asia-south1 --project APPROVED_PROJECT
kubectl config current-context
kubectl auth can-i get pods -n orders
```

Confirm project identity before changing anything. Do not rely on an operator's default project left over from another task. Use separate configuration or explicit project arguments in automation.

Render cloud-specific workload values: registry path, secret integration, storage class, identity labels, and ingress configuration. Preserve the same application digest and HTTP behavior where possible. Validate API compatibility through server-side dry-run and functional tests.

Autopilot may require resource declarations and disallow host-level privileges. Test the actual admission response rather than adding exceptions blindly. Security restrictions can expose assumptions your earlier platform tolerated without review.

Data Portability Has a Recovery Cost

Database portability includes extensions, collation, privileges, connection limits, backup formats, encryption dependencies, and transaction behavior. A PostgreSQL-compatible service name is not proof that an existing dump and workload can move unchanged.

Object storage migration needs inventory, version handling, metadata translation, checksums, deletion semantics, and cutover policy. If URLs were persisted in database rows, moving buckets may require data transformation rather than a single environment-variable change.

Measure bulk migration time and incremental catch-up. Cross-cloud transfer can incur egress charges and take longer than compute provisioning. Data gravity often makes the database and object-store migration the dominant part of the project.

Test the Limits, Then Record Them

Create a portability matrix listing what ran unchanged, what required new configuration, and what required application changes. For each item, record a verification test and owner. This is more useful than claiming the whole platform is cloud agnostic.

Test allowed and denied network paths, workload identity, image retrieval, storage provisioning, synthetic order processing, and graceful shutdown. Also test what happens when a cloud service rate-limits or returns an authorization error; provider-specific failure behavior can leak through adapters.

Keep GCP state encrypted and scoped in an independently managed remote backend. Back up recovery configuration and test privileged access separately. Cloud provisioning state does not restore Cloud SQL records or object versions.

Cleanup and Upgrade Responsibility

Deletion protection requires a deliberate reviewed change before cluster destruction. Plan the retention of data volumes, registry images, external addresses, and backups. Managed node removal does not imply every chargeable resource disappears.

Track release-channel upgrades and application compatibility. Managed operations still need maintenance planning, supported API monitoring, and incident procedures. Record which platform actions Google performs and which your team must validate.

The deliverable is an honest GCP deployment and a tested portability boundary. If the migration requires rewriting storage integration, say so early; that is an architectural finding, not a failure of Docker.

Build Adapter Tests Around Failure Semantics

An object-store adapter should expose a small documented result model: success, missing object, authorization failure, retryable service failure, and invalid request. Test how each cloud SDK

maps its errors into that model. Otherwise provider exceptions can leak into business retry logic.

Signed URLs need explicit expiration, allowed method, content constraints, and trust boundaries. A URL generated on one provider may not permit the same headers or upload behavior on another. Validate the actual client workflow rather than only asserting a string was returned.

If storing provider identifiers in order records, distinguish durable business identity from storage location. A migration should be able to change location without changing which customer object the record represents. Record checksums and metadata needed to verify transfer completeness.

Event delivery also needs idempotency. Cloud notification systems can differ in ordering and retry behavior, so the worker must not treat one arrival as proof of one unique operation. Use durable deduplication or transactional state transitions.

These tests define the portability contract. They let the team add a cloud-specific implementation without pretending every managed-service feature is interchangeable or forcing business code to understand every provider's failure syntax.

Treat an adapter exception as a reviewable product limitation with an owner, rather than silently falling back to a different storage behavior.

Key Takeaways

- **Containers are portable artifacts; managed-service contracts may not be.**
- **Autopilot changes node ownership, not application responsibility.**
- **Use cloud-specific identity, networking, and storage configuration.**
- **Keep application adapters narrow and behaviorally tested.**
- **Include data migration time and egress cost in the plan.**
- **Document verified portability and explicit exceptions.**

References

- Autopilot: Documentation
- GKE Workload Identity Federation: Documentation

Chapter 20: Connect Clouds Carefully: Identity, Routing, and Data Boundaries

The Tunnel Was Up; Traffic Went Nowhere

The AWS-Azure VPN reports established tunnels. The application still cannot reach the recovery database. Both clouds use `10.0.0.0/16`, so hosts treat the destination as local rather than sending it across the intended route.

The network dashboard is green because tunnel establishment succeeded. The application path remains wrong. **Connectivity is an end-to-end route and authorization contract, not a tunnel-status icon.**

Allocate Addresses Before Connecting

Reserve non-overlapping ranges for each VPC/VNet, Pod network, Service network, administrative network, and control-plane range. Check existing offices, vendor networks, acquired environments, and future expansion, not just two new cloud diagrams.

Scope	Example allocation	Ownership
AWS production VPC	10.50.0.0/16	AWS platform root
Azure recovery VNet	10.80.0.0/16	Azure platform root
Azure overlay Pods	10.81.0.0/16	AKS configuration
Azure Services	10.82.0.0/16	AKS configuration
GCP nodes	10.90.0.0/20	GCP network root
GCP Pods	10.92.0.0/16	GKE configuration

These are book allocations, not permission to use them without checking your organization. VPN address translation can sometimes accommodate overlap, but adds complexity to routing, DNS, logging, and policy. Avoid it when deliberate address planning is possible.

Choose the Routing Protocol Explicitly

AWS Site-to-Site VPN can use static routing or BGP depending on the chosen architecture. Azure gateway settings, tunnel endpoints, and route propagation must match. Define who advertises which prefixes and how withdrawal behaves during failure.

AWS commonly provides two tunnels per VPN connection. A single static Azure public endpoint example does not automatically implement a fully redundant Azure active-active architecture. Engineer both peers and failover paths according to the selected gateway modes.

The AWS Customer Gateway resource describes the remote peer. It is not a running router and does not configure Azure. The VPN connection provisions the AWS side; Azure local-network gateway and connection configuration complete the other side.

AWS Static-Routing Example

Use a separate connectivity root with the approved AWS provider and remote state. The Azure gateway public IP must already exist and be stable. This example is a static-route lab, not a complete production redundant VPN.

vpn.tf

```
variable "vpc_id" { type = string }
variable "private_route_table_ids" { type = set(string) }
variable "azure_gateway_public_ip" { type = string }
variable "azure_network_cidr" {
  type    = string
  default = "10.80.0.0/16"
  validation {
    condition     = can(cidrnetmask(var.azure_network_cidr))
    error_message = "Supply a valid IPv4 Azure network CIDR."
  }
}
resource "aws_vpn_gateway" "aws" {
  vpc_id = var.vpc_id
  tags = { Name = "orders-cross-cloud" }
}
resource "aws_customer_gateway" "azure" {
  bgp_asn    = 65020 # Required API field; routing here remains static.
  ip_address = var.azure_gateway_public_ip
  type       = "ipsec.1"
}
resource "aws_vpn_connection" "azure" {
  vpn_gateway_id      = aws_vpn_gateway.aws.id
  customer_gateway_id = aws_customer_gateway.azure.id
  type                = "ipsec.1"
  static_routes_only  = true
  # Generated tunnel keys are sensitive outputs/state; do not log them.
}
resource "aws_vpn_connection_route" "azure" {
  destination_cidr_block = var.azure_network_cidr
  vpn_connection_id     = aws_vpn_connection.azure.id
}
resource "aws_route" "azure" {
  for_each               = var.private_route_table_ids
  route_table_id         = each.value
  destination_cidr_block = var.azure_network_cidr
  gateway_id             = aws_vpn_gateway.aws.id
}
```

Supply route-table identifiers as known configuration values or through a map with stable keys if outputs are unknown at plan time. A set of newly created unknown IDs may be unsuitable for `for_each`; preserve stable topology keys when integrating this snippet with network creation.

On Azure, configure the selected VPN gateway, remote AWS tunnel addresses, address spaces, and compatible IKE/IPsec settings. Deliver pre-shared keys through a protected process. Do not copy the entire AWS-generated customer-gateway configuration into a public issue.

DNS Is a Separate Cross-Cloud System

A network route does not teach an AWS resolver how to resolve a private Azure database name. Use supported inbound/outbound resolver endpoints and conditional forwarding for the required domains. Scope rules to avoid loops and accidental forwarding of unrelated names.

Test resolution from the real application environment. A laptop using a different resolver can produce a false positive. Record the returned address, resolver path, TTL, and expected private zone.

Keep service names stable where appropriate, but do not hide write ownership in a DNS alias. A database name resolving to a secondary does not prove it is writable or sufficiently current. Identity and promotion checks remain separate.

DNS availability is part of recovery. If conditional forwarding depends on the primary cloud, the secondary may lose name resolution during the exact incident it must survive. Maintain independently usable recovery names and procedures.

FortiGate and Return Paths

Cross-cloud traffic may traverse an inspected path, but only if forward and return routes enforce it. Place FortiGate or the selected inspection service deliberately and test asymmetric routing, failover, and bypass prevention.

Stateful inspection can fail when a return packet reaches another appliance without the required session state. Cloud gateway routing, appliance HA, and NAT choices must be compatible. Do not add a default route to an appliance and assume all peering or VPN traffic will traverse it.

Constrain management access separately from data-plane traffic. Back up firewall policy and VPN configuration securely; exports may contain keys. Test license and image availability in the recovery cloud.

Data Gravity and Cost

Every cross-cloud call adds latency, failure modes, and possible egress charges. A chatty API talking to a remote database can suffer even when individual round trips appear acceptable. Measure transactions, not just ping latency.

Prefer keeping tightly coupled transactional work near its data. Replicate or transfer selected data through a defined contract rather than making every request depend on a WAN database connection. Replication still needs encryption, identity, retention, and monitoring.

Budget bandwidth for steady-state replication, initial seeding, catch-up after interruption, backups, and game-day traffic. A VPN sized for ordinary requests can take days to catch up after a large outage. Review cloud gateway limits and data-transfer pricing using current regional details before purchase.

Architecture Checklist

```
# Cross-Cloud Connectivity Gate
- [ ] All VPC/VNet/Pod/Service ranges checked for overlap
- [ ] Forward and return routes documented at every hop
- [ ] Static routing or BGP selected and peer settings matched
- [ ] Tunnel redundancy and failure detection tested
- [ ] DNS forwarding domains and loop prevention verified
- [ ] Caller and destination permissions scoped
- [ ] Inspection path and bypass denial demonstrated
- [ ] MTU, MSS, throughput, latency, and reconnect behavior measured
- [ ] Pre-shared keys stored outside Git and rotation rehearsed
- [ ] Egress and catch-up bandwidth budget approved
- [ ] Recovery DNS and administration survive primary-cloud loss
```

A checked box needs evidence: command output, packet observation, service transaction, and owner. Keep the checklist in the repository, but store sensitive diagnostic material under restricted access.

Prove the Path and Its Failure Behavior

Test an allowed synthetic request and a denied request from an unauthorized identity. Verify routes and firewall logs, then interrupt one tunnel in a lab and measure reconnection. A successful ICMP test does not prove TLS, database authentication, or application writes.

Exercise MTU-sensitive payloads and long-lived connections. VPN encapsulation can expose packet-size problems that ordinary health probes miss. Observe application retry behavior to prevent reconnection storms.

Keep cloud state roots independent. Connectivity resources may depend on published endpoint identifiers, but recovery of one cloud should not require a live query to the failed cloud's backend. Record those values securely and refresh them through controlled changes.

Destroy connectivity only after stopping dependent replication and traffic. Preserve data checkpoints and configuration backups. A tunnel removal can silently invalidate the secondary's RPO if replication stops without an alert.

Rotation Can Interrupt the Data Path

A VPN pre-shared-key rotation changes both peers' configuration and can interrupt active sessions. Rehearse tunnel-by-tunnel changes using the selected redundancy design, monitor reconnection, and confirm route behavior before changing the remaining path.

Document which identity can retrieve or change keys on each cloud. Generated VPN configuration and Terraform state may contain sensitive material even when ordinary outputs are masked. Protect exports and delete temporary copies through an approved retention process.

During rotation, verify database replication resumes and catches up. A green tunnel does not prove the worker reconnected or the standby replayed missed WAL. Alert on the data consequence, not merely the network event.

Keep an independent rollback path with the prior approved configuration available only to authorized operators. Once rotation completes, revoke obsolete access and confirm the final configuration matches the intended state. Untracked manual peer changes will otherwise reappear as drift during the next Terraform apply.

Verify rotation alarms reach the operator before beginning; an interrupted replication path without notification can invalidate the recovery objective.

Key Takeaways

- **Tunnel establishment is only one part of end-to-end connectivity.**
- **Plan non-overlapping addresses across every network layer.**
- **Configure both peers and both route directions explicitly.**
- **DNS and inspection need independent tested paths.**
- **Measure transaction latency, replication catch-up, and egress cost.**
- **Keep recovery configuration usable without the failed cloud.**

References

- AWS Site-to-Site VPN: Documentation
- Azure VPN Gateway: Documentation

Chapter 21: Failover Is a Data Problem: Engineer Disaster Recovery

DNS Moved; Writes Still Failed

The AWS application region becomes unavailable. Global DNS changes to Azure, and the API begins receiving traffic there. Its database is still a read-only standby. Order creation fails while monitoring reports that the recovery endpoint is reachable.

The team prepared traffic routing but never rehearsed data promotion. Worse, nobody can prove the old primary is unable to resume writing. **A recovery endpoint is not a recovered service until write ownership and data correctness are established.**

Active-Passive Has One Writer

In active-passive architecture, one site owns writes and the secondary maintains a recoverable copy. Recovery requires choosing an acceptable recovery point, fencing the previous writer, promoting the secondary, and reconnecting clients.

Active-active requires conflict handling, globally meaningful identity, write routing, and data semantics that support concurrent ownership. Deploying the same API in two clouds against independent databases does not create an active-active order service.

For this runbook, use a deliberately configured self-managed PostgreSQL physical standby in Azure. The replication, TLS, WAL retention, storage, and monitoring must already have been tested. This is not a claim that an RDS instance can automatically become an Azure-managed read replica.

The example promotion command applies to self-managed PostgreSQL 17. Managed services require their own supported promotion APIs and documented replication compatibility. Do not substitute a managed database and reuse `pg_ctl` blindly.

Measure RPO and RTO Before the Incident

RPO is the maximum accepted recovery gap. Replication lag must be observed before failure because the primary may become unreachable during the incident. Monitor replay progress and retained WAL, not just a green replication connection.

RTO includes detection, decision-making, fencing, promotion, application rollout, DNS behavior, and validation. A database promotion taking ten seconds does not prove a ten-second service recovery.

For the book's target, validate that the newest recovered committed order falls within five minutes and that the complete service returns within sixty minutes. If lag exceeds the accepted RPO, the incident commander must explicitly approve the data-loss decision or select another recovery path.

Fence Before Promotion

Fencing prevents the former primary from accepting writes. Depending on the architecture, it can include shutting down verified primary compute, revoking a write lease, blocking the write path through independently controlled infrastructure, or another database-supported fencing mechanism.

DNS changes alone do not fence existing connections or internal workers. An unreachable primary is not necessarily stopped; it may still serve a subset of clients during a partition. Record positive evidence that the old writer cannot continue.

If your architecture cannot establish safe fencing during the planned failure, that is a design gap. Do not make simultaneous writes an undocumented emergency mode.

Recovery Checklist

```
# Active-Passive Recovery Record
Incident ID:
Commander and database owner:
Primary failure evidence:
Primary write-fencing evidence:
Newest observed replication checkpoint:
Accepted recovery gap and approver:
Secondary storage/identity/network verification:
Promotion start/end:
Application image and configuration revision:
DNS change identifier:
Synthetic order and worker result:
Service recovery timestamp:
Failback owner and restrictions:
```

Keep this record outside the failed site's control plane. It must remain available when the primary CI, identity session, or state backend cannot be reached.

Inspect the Secondary

Use an explicitly named Azure recovery context with independently available credentials and private connectivity. This example assumes the PostgreSQL container includes the official image's `gosu` utility and the configured `PGDATA` directory.

```
export RECOVERY_CONTEXT=azure-recovery
kubectl --context "$RECOVERY_CONTEXT" -n data get pod postgres-0
kubectl --context "$RECOVERY_CONTEXT" -n data exec postgres-0 -- \
  psql -U orders_admin -d orders -v ON_ERROR_STOP=1 \
  -c "SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn(), pg_last_xact_replay_timestamp();"
```

Replay timestamps can be old on an idle database without implying active replication lag. Interpret them with source activity, LSN monitoring, and known synthetic checkpoints. Do not turn one timestamp into a universal RPO calculation.

Confirm the standby has required WAL and is consistent at the selected recovery point. Protect the diagnostic output; it may include infrastructure and transaction metadata. Verify available capacity, secret retrieval, image pulls, storage access, and application connection routing.

Promote the Approved Standby

Only after documented fencing and data-loss approval:

```
kubectl --context "$RECOVERY_CONTEXT" -n data exec postgres-0 -- \
  gosu postgres pg_ctl \
  -D /var/lib/postgresql/data/pgdata promote -w -t 60

kubectl --context "$RECOVERY_CONTEXT" -n data exec postgres-0 -- \
  psql -U orders_admin -d orders -v ON_ERROR_STOP=1 \
  -c "SELECT pg_is_in_recovery();"
```

The final query must return false. Then execute a controlled write and verify it commits. Authentication success or TCP reachability alone is insufficient.

Promotion creates a new primary timeline. The old primary cannot simply resume and become its peer. Failback needs a deliberate re-seeding or supported reconciliation procedure, with source-of-truth ownership preserved.

If promotion fails, stop and inspect PostgreSQL logs and recovery status. Do not repeatedly run destructive recovery commands or create another independent primary without understanding the first attempt.

Deploy the Recovery Application

The Azure cluster must use Azure storage and identity configuration, not the AWS EBS StorageClass from Chapter 15. Its application values should reference the newly writable endpoint and the same approved image digest.

```
# recovery/ contains complete reviewed Azure-specific manifests, not snippets.
kubectl --context "$RECOVERY_CONTEXT" apply --dry-run=server -f recovery/
kubectl --context "$RECOVERY_CONTEXT" apply -f recovery/
kubectl --context "$RECOVERY_CONTEXT" -n orders \
  rollout status deployment/orders-api --timeout=180s
```

The recovery directory is an environment implementation prerequisite. Populate it from the complete application manifests and tested Azure integrations; the commands do not create that architecture automatically.

Reconnect pools deliberately. Verify the API and worker use the promoted endpoint, correct TLS identity, and least-privilege credentials. Keep external side effects disabled until duplicate-processing and write-ownership checks pass.

Update Independently Controlled DNS

Use a DNS control plane whose recovery access does not rely on the failed application region. The example uses a pre-existing Cloudflare CNAME record with known zone and record IDs. Adapt the operation to your actual authoritative provider; do not create duplicate records as an emergency shortcut.

```
# Set these from the protected recovery inventory; token never goes in Git.
export DNS_ZONE_ID=REPLACE_ZONE_ID
export DNS_RECORD_ID=REPLACE_RECORD_ID
```

```
export DNS_API_TOKEN=RETRIEVE_FROM_SECRET_SOURCE
umask 077
header_file=$(mktemp)
trap 'rm -f "$header_file"' EXIT
printf 'Authorization: Bearer %s\n' "$DNS_API_TOKEN" > "$header_file"
unset DNS_API_TOKEN
cat > dns-change.json <<'JSON'
{
  "type": "CNAME",
  "name": "orders.example.com",
  "content": "recovery.example.net",
  "ttl": 60,
  "proxied": false
}
JSON
curl --fail-with-body --silent --show-error \
  --request PATCH \
  --header "@$header_file" \
  --header 'Content-Type: application/json' \
  --data @dns-change.json \
  "https://api.cloudflare.com/client/v4/zones/$DNS_ZONE_ID/dns_records/$DNS_RECORD_ID"
```

Verify the API response indicates success. Replace names and ensure the record type matches the existing record. Proxied records have provider-specific TTL behavior; this example deliberately uses DNS-only routing.

Lowering TTL during an incident does not invalidate already cached records with older TTLs. Clients may cache beyond expectations, keep existing connections, or use different recursive resolvers. Measure real traffic convergence and maintain fencing throughout it.

Verify Service, Then Declare Recovery

Create a synthetic order using the recovery endpoint and an idempotency key. Confirm the row exists, the worker completes expected processing, and the result appears through the public API. Reconcile acknowledged requests near the failure boundary with recovered records.

Check error rates, latency, backlog, database saturation, and external dependencies. A recovered service operating beyond safe capacity can fail again immediately. Use admission controls and communicate any reduced service envelope.

Record actual RPO and RTO with evidence. If customer data was lost, report the gap precisely. Do not redefine the objective after the incident to make the exercise pass.

Failback Is Another Migration

Keep the former primary fenced. Rebuild it from the current authoritative data, establish replication, verify catch-up, and schedule an approved reverse cutover. Do not repoint DNS merely because AWS is healthy again.

Preserve incident logs and state snapshots. Terraform repairs infrastructure ownership; PostgreSQL recovery repairs data continuity. Neither should be used to hide the other's unresolved state.

Rehearse this process with synthetic data before relying on it. The successful deliverable is one writer, measured data continuity, and a functioning order workflow under the defined failure—not a dashboard showing two healthy clusters.

Preserve the Recovery Decision

Record who authorized promotion, which recovery point was accepted, and how primary fencing was established. These facts are part of data governance and incident reconstruction, not optional paperwork after service returns.

Keep customer-facing reconciliation separate from technical promotion. Some requests near the failure boundary may have committed without the caller receiving a response. Use idempotency records and order evidence to avoid treating all timeouts as lost transactions.

After recovery, compare the accepted gap with actual recovered records and identify any missing acknowledgements or duplicate side effects. Assign a remediation owner and preserve the evidence before logs expire.

This closes the recovery loop: uptime returns first, but correctness, reconciliation, and an independently verified failback plan complete the incident.

Keep the reconciliation owner assigned until ambiguous requests and duplicate external effects have been investigated and resolved.

Key Takeaways

- **Failover requires write ownership, not just traffic routing.**
- **Fence the old primary before promoting a secondary.**
- **Measure recovery gaps from transaction evidence.**
- **Use promotion commands appropriate to the actual database implementation.**
- **DNS caching and existing connections complicate cutover.**
- **Treat failback as a controlled data migration.**

References

- PostgreSQL standby operation: Documentation
- Cloudflare DNS API: Documentation

Chapter 22: Break It Before Production: The Multi-Cloud Capstone

Green Dashboards, Missing Packets

Launch day begins with healthy Pods and a successful deployment. Half the application's requests time out because an intermittent network path drops packets. The health endpoint uses a different path, and the monitoring dashboard stays green.

Staging had tested only successful startup. Nobody had interrupted a dependency, replaced a node under load, or verified that customer-impact alerts reached an operator.

Operational readiness requires observed behavior under defined failures. A platform is credible when another engineer can provision, diagnose, recover, and retire it using documented procedures and approved identities.

State the Experiment Before Creating the Fault

A game day begins with a hypothesis, scope, synthetic workload, expected observations, abort conditions, and rollback steps. The purpose is to learn whether a specific architectural claim holds—not to create random damage and hope monitoring explains it.

Use a dedicated lab account and disposable cluster. Exclude production customer data, real fulfillment actions, and production credentials. Confirm resource names and contexts before every fault command.

An experiment must have an independent stop mechanism. If the same network you are breaking carries your only administrative access, you have not designed a controlled exercise. Assign a fault operator, observer, incident commander, and recovery owner; one person can fill multiple roles only if the workload remains manageable.

The Game Day Record

```
# Experiment Record
Experiment ID and owner:
Approved account, cluster context, namespace:
Hypothesis:
Synthetic request mix and duration:
Fault and exact target:
Expected service behavior:
Expected metric/log/trace evidence:
```

```
Expected alert route and delivery deadline:
Abort conditions:
Independent stop mechanism:
Recovery commands and owner:
Start, detection, alert, recovery timestamps:
Observed RPO/RTO and data checks:
Unexpected behavior and follow-up owner:
```

Store evidence under restricted access and sanitize it before attaching to public book issues. Logs and packet captures can disclose addresses, tokens, and customer-like identifiers even in a test environment.

Experiment One: Lose a Worker Node

Use a disposable multi-node kind cluster with a known worker container. List actual container names and explicitly select a worker; do not target the control-plane container or another project's node.

```
kubectl config current-context
docker ps --format '{{.Names}}'
export LAB_WORKER=REPLACE_OBSERVED_KIND_WORKER
# After verifying that this is the approved disposable worker:
docker stop "$LAB_WORKER"
kubectl get nodes
kubectl -n orders get pods -o wide
# Independent rollback:
docker start "$LAB_WORKER"
```

Stopping the node container approximates abrupt node unavailability. It is different from draining, which uses voluntary eviction behavior. Compare both experiments because PDBs do not govern every involuntary failure.

Observe detection latency, endpoint removal, replacement scheduling, image pulls, and service impact. If all replicas were on the failed node, the experiment should reveal the placement error. If spare capacity is absent, replacement may remain Pending even though the controller requests it correctly.

Check committed orders and worker idempotency after recovery. A node failure can interrupt an operation after its side effect but before acknowledgement. The application must handle retries without silently duplicating fulfillment.

Experiment Two: Block the Database Dependency

Create a dedicated `gameday` namespace containing a reviewed clone of the application and synthetic database. Verify no existing additive allow policy defeats the intended experiment. The following policy isolates API egress and permits only the configured cluster DNS path; database TCP traffic is denied by omission.

deny-database-egress.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: experiment-deny-database
  namespace: gameday
```

```
spec:
  podSelector:
    matchLabels:
      app: orders-api
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - {protocol: UDP, port: 53}
        - {protocol: TCP, port: 53}
```

Verify your DNS implementation and enforcement first. The policy blocks other non-DNS egress too; in this isolated fixture the database must be its only application dependency. If testing a production-like dependency set, preserve explicit permissions for unrelated services instead of attributing every failure to PostgreSQL.

```
kubectl apply --dry-run=server -f deny-database-egress.yaml
kubectl apply -f deny-database-egress.yaml
kubectl -n gameday logs deployment/orders-api --tail=100
# Roll back only this experiment's policy.
kubectl -n gameday delete networkpolicy experiment-deny-database
```

An existing egress allow policy can permit traffic because policies are additive. Confirm the denied request and an approved control request before declaring the fault successful. Observe whether bounded timeouts, backoff, and admission behavior protect the database and clients.

Chaos Mesh or another reviewed fault controller can model delay and packet loss more precisely. Install it only in an authorized lab with pinned versions and documented privileges. Its custom-resource schemas and selectors must be validated against the installed release; do not paste unverified CRDs into production.

Experiment Three: Lose Jenkins

Stop the lab controller through its service manager and measure how existing workloads behave. The expected result is interrupted build/release capability with existing applications still serving.

```
# Execute on the approved lab controller host, not production.
sudo systemctl stop jenkins-container
# Restore the service only after the persistent mount is verified.
findmnt /srv/jenkins
sudo systemctl start jenkins-container
```

Verify jobs, configuration, and credentials survive. Then rehearse controller replacement from backup on isolated compute. Confirm no production job fires automatically during recovery testing. Disable or redirect deployment credentials until the recovery controller is trusted.

A pipeline retry must not repeat destructive infrastructure changes or duplicate release publication. Review build IDs, approved plan identity, and registry immutability before resuming interrupted work.

Experiment Four: Make State Unavailable

Do not delete a production state bucket to demonstrate resilience. In a dedicated backend lab, temporarily remove the test writer's access or introduce a controlled backend outage and verify plans/applies fail closed.

The expected result is existing workloads continue running while infrastructure changes stop. An agent must not fall back to local state and begin managing the same infrastructure as a fresh root.

Exercise a versioned-state restoration in the lab using Chapter 3's runbook. Deleting a whole bucket destroys a different boundary from deleting a current object version; choose the experiment deliberately and preserve independently protected recovery evidence.

Record how the team locates the correct backend, recovers credentials, verifies lineage and serial, and reconciles actual resources. A restored JSON file with unexpected delete proposals is not a successful state recovery.

Verify Alerts Actually Arrive

For each fault, define the expected alert and inspect delivery to the configured receiver. Record detection time, delivery time, acknowledgement, and diagnostic usefulness. A dashboard condition changing color does not demonstrate on-call notification.

Use external synthetic probes for the user path and internal metrics for diagnosis. A collector on the failed node can disappear with its target; independent observations help distinguish telemetry failure from service recovery.

Validate that alert labels identify the correct environment and experiment. Suppress noise only through reviewed temporary routing, not by disabling the entire alert system. Remove experiment suppression afterward and prove normal routing resumes.

Recovery and Data Verification

After each experiment, verify representative order creation, worker completion, stored records, and image/configuration identities. Compare observed outcomes with the architecture's stated RPO and RTO.

Test backups through restoration, not listing files. Check that recovery roles can decrypt retained data and retrieve images without primary-site services. Verify one writer after failover and keep the former primary fenced until failback is approved.

Measure the full recovery path, including human decisions and failed attempts. If the runbook requires knowledge absent from the repository, record and fix that gap before the next exercise.

Readiness Is a Reviewable Result

The capstone deliverable contains all infrastructure roots, workload definitions, image evidence, dependency maps, threat models, release procedures, recovery records, and game-day findings. Static examples are not automatically a deployed platform; distinguish code completeness from operational verification.

Classify findings by consequence and owner. Block release on unresolved data-loss, privileged-access, or unbounded failure-amplification problems. Other limitations may be accepted with explicit scope and deadline; hiding them behind "all tests passed" prevents informed operation.

Finish with a teardown rehearsal. Identify retained storage, snapshots, keys, registries, public addresses, gateways, and log retention. Destroy disposable resources through their owners and verify remaining charges are deliberate.

The platform's final claim should be specific: under the tested topology, load, and failure conditions, service and recovery objectives were met with recorded evidence. Anything broader requires another experiment.

Close Every Experiment

Remove only the experiment's fault objects and verify ordinary policy remains intact. Check for stopped lab nodes, temporary access rules, abandoned snapshots, altered routing, and muted alerts. Cleanup is a measured part of the exercise, not an assumption.

Assign each unexpected result an owner, severity, and verification test. Repeat the relevant experiment after the fix. A finding closed by changing a document alone does not demonstrate improved system behavior.

Keep the final readiness decision linked to those results so the next operator understands both tested guarantees and unresolved limits.

Key Takeaways

- **Define a hypothesis, target, abort condition, and rollback before injecting failure.**
- **Use dedicated lab environments and synthetic data.**
- **Test abrupt failures separately from voluntary maintenance.**
- **Prove alert delivery and data correctness, not just recovery of processes.**
- **Keep CI and state outages from stopping existing workloads.**
- **Publish operational evidence and unresolved limitations honestly.**

References

- kind: Documentation
- Chaos Mesh: Documentation